

Operating System: Unix

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Class details

- 27h: $2h \times 12$ + 1h mid term + 2h exam final
- Grade: 10% Participation
40% Mid term exam
50% Final exam
- For anonymous feedback: [Feedback - Google Docs](#)

Class organisation

- Lectures
- Labs during class

Syllabus

- **Session 1:** Introduction to Unix/Linux
- **Session 2:** Installation of Linux (WSL, VirtualBox, Mac)
- **Session 3:** Linux Architecture & File Systems
- **Session 4:** CLI Basics: Commands & Navigation
- **Session 5:** Advanced CLI: I/O & Text Processing
- **Session 6:** Shell Scripting I: Basics & Automation
- **Session 7:** Shell Scripting II: Advanced Scripts
- **Session 8:** Processes & Init Systems
- **Session 9:** Users, Groups & Permissions
- **Session 10:** Text Editors: vi/vim
- **Session 11:** Networking
- **Session 12:** Remote Access, Downloads & Shell Customization

History of Unix/Linux

Session 1

Objectives:

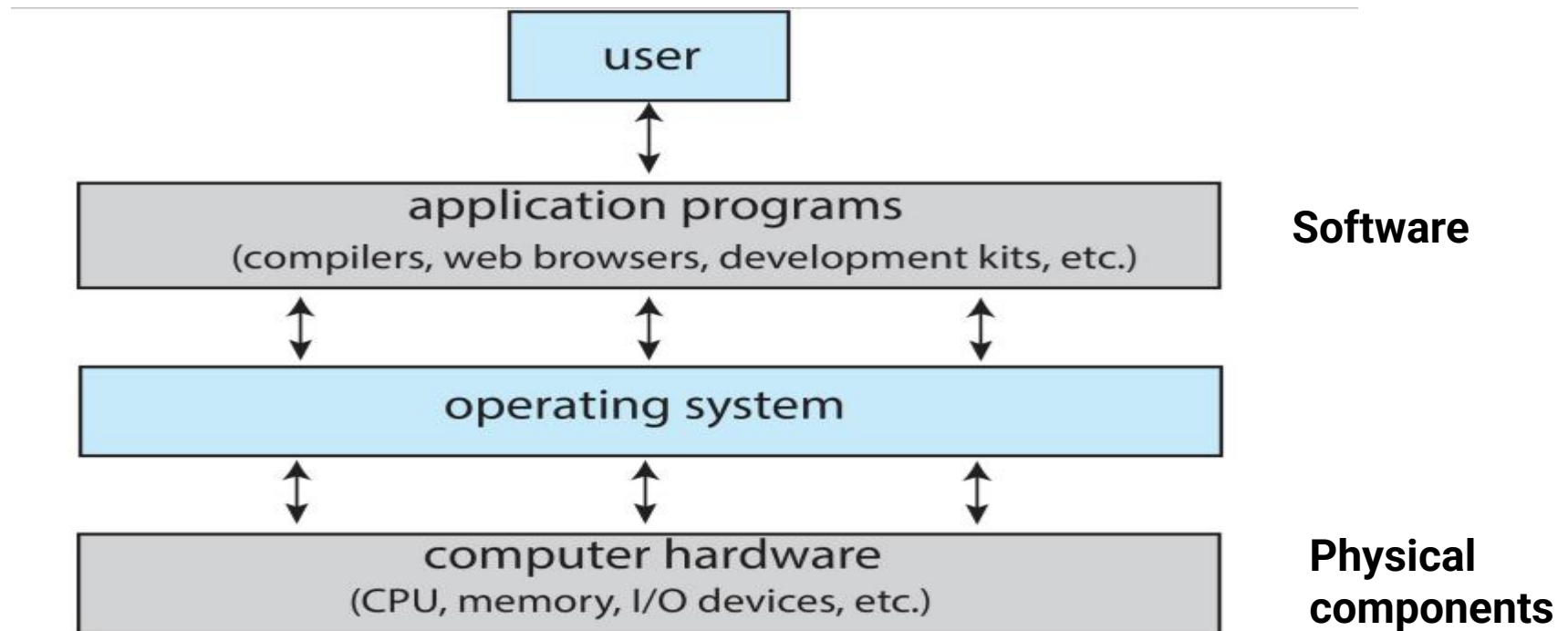
- Understand the origin, evolution, and impact of UNIX/Linux systems

History of Unix/Linux

What is an operating system?

Operating system

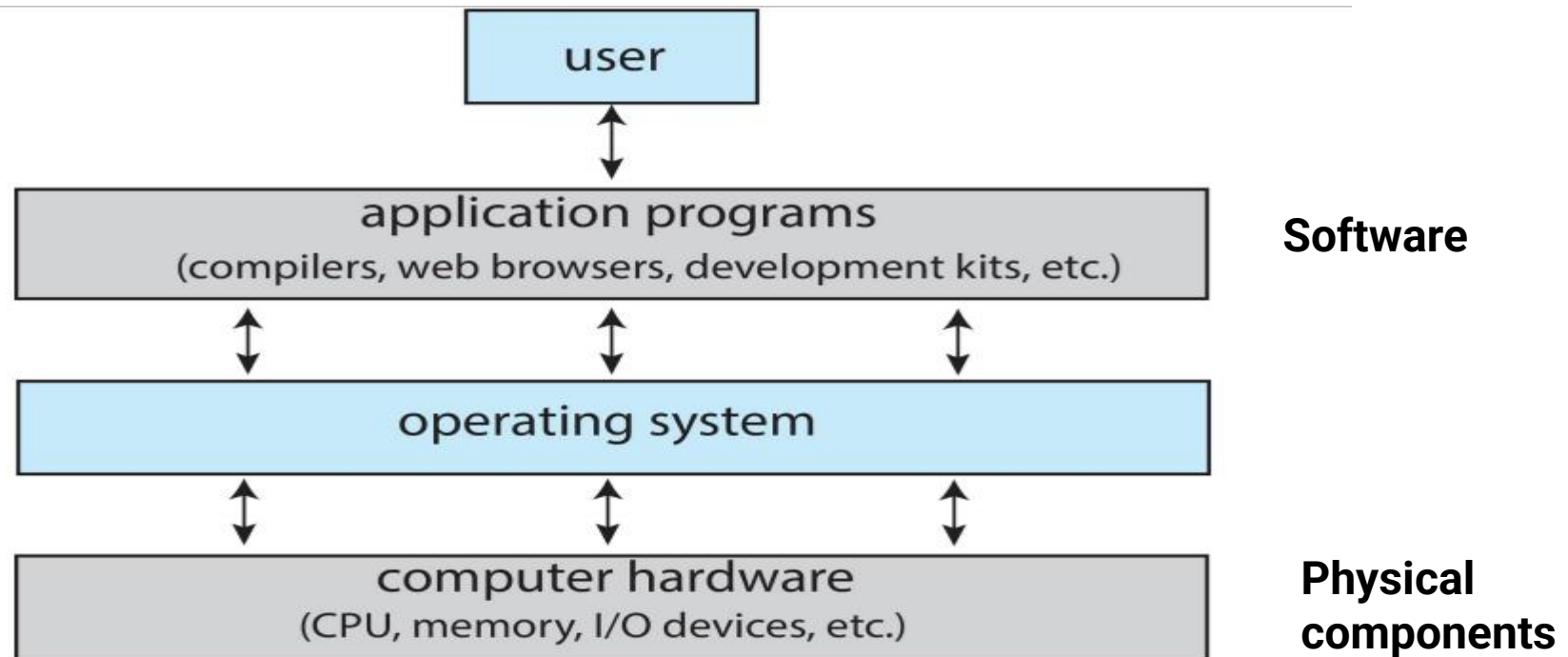
An OS is a layer of software that sits between your applications and your computer's hardware.



Operating system

An OS is a layer of software that sits between your applications and your computer's hardware. **It manages the computer's resources like the CPU, memory, and devices such as printers, keyboards, and screens.** Without an operating system, you'd need to program directly in machine language for each piece of hardware.

OS = resource manager
- CPU scheduling
- memory management
- device access



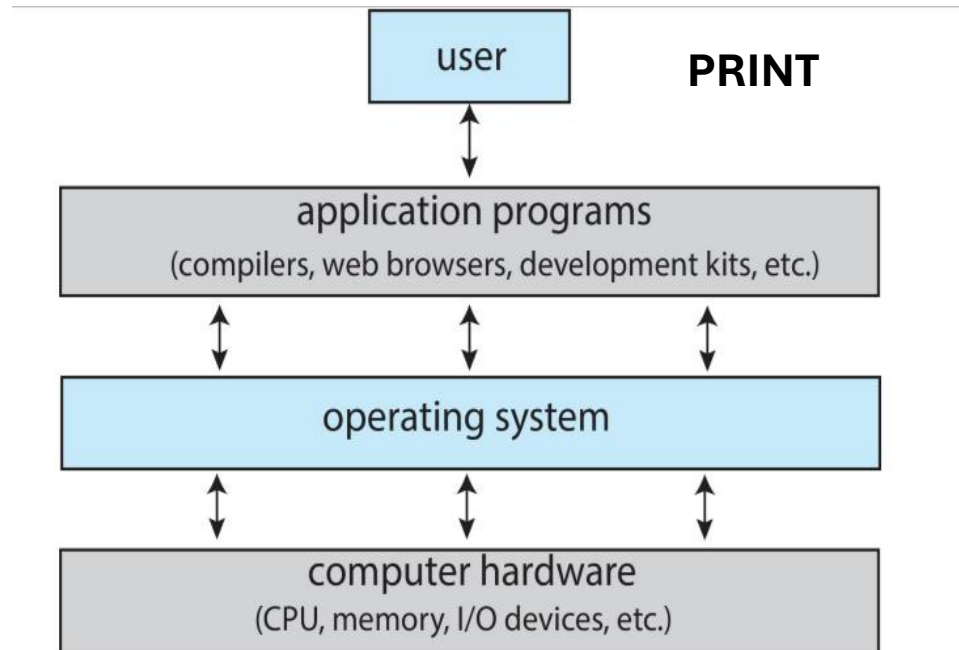
Operating systems

Two programs try to print at the same time.
What should happen?

Operating systems

Two programs try to print at the same time.
What should happen?

The OS steps in to manage that, ensuring only one gets access to the printer at once. It also makes sure that if a program is waiting for something (data from the network) and it doesn't block the whole system. Instead, it pauses that task and lets others continue. This efficient management makes multitasking possible.



Operating systems

Operating system goals:

- Execute user programs and make solving user problems easier
- Make the computer system convenient to use
- Use the computer hardware in an efficient manner, acts like a resource manager

These goals may sometimes conflict. For instance, improving performance might reduce convenience. So operating system designers have to make careful choices to balance these needs.

Operating systems

- Examples:

Operating systems

- Examples: Android, macOS, Windows, Linux/GNU



Operating systems

Advantages:

- Operating System manages external and internal devices for example, printers, scanners, and other.
- Operating System provides interfaces and drivers for proper communication between system and hardware devices without needing to understand every technical detail.
- Allows multiple applications to run simultaneously
- Organizes and manages files on storage devices
- Operating system allocates resources to various applications and ensures their efficient utilization.

Operating systems

Disadvantages:

- If an error occurred in your operating system, then there may be a chance that your data may not be recovered therefore always have a backup of your data.
- Threats and viruses can attack our operating system at any time, making it challenging for the OS to keep the system protected from these dangers.
- learning about new operating system can be a time-consuming
- Operating systems consume system resources, including CPU, memory, and storage, which can affect the performance of other applications.

Operating systems

What was the first operating system?

Operating systems

- **1940s-1950s: Early Beginnings:**

- Computers operated without operating systems
- Programs were manually loaded and run, one at a time
- Operators had no direct interface with the system, and job implementation was done in a batch system, meaning that users have to prepare a job separately to perform it.
- The first operating system was introduced in **1956**. It was a batch processing system GM-NAA I/O (1956) that automated job handling

Operating systems

- **1960s: Multiprogramming and Timesharing**

- Introduction of multiprogramming to utilize CPU efficiently.
- Timesharing systems, it allowed multiple users to access the same machine simultaneously. Two major systems were CTSS in 1961 and Multics in 1969.

These systems showed the power of shared computing, and they heavily influenced what came after.

Operating systems

- **1970s: Unix and Personal Computers**

- Unix was born in 1971, and it brought something new: simplicity, portability, and multitasking.
- Personal computers emerged, leading to simpler OSs like CP/M (1974) and PC-DOS (1981).

Systems like CP/M and later DOS allowed people to run computers at home or in small offices. Unix was special, it was built for power users and developers, and its influence would be enormous

Operating systems

- **1980s: GUI and Networking**

In the 1980s, computing moved closer to the general public

- Graphical User Interfaces (GUIs) gained popularity with systems like Apple Macintosh (1984) and Microsoft Windows (1985). Now you didn't need to memorize commands, you could point and click
- Unix systems adopted TCP/IP, the protocol that powers the internet today. So now computers could not only be personal, they could also be connected.

Operating systems

- **1990s: Linux and Advanced GUIs**

- Linux (1991). Linus Torvalds released the Linux kernel as a free alternative to proprietary Unix systems
- Graphical interfaces became more advanced. Windows 95, Mac OS, and desktop environments like KDE and GNOME on Linux made computers more attractive and intuitive.

It was a period of rapid growth, especially as the internet took off.

- **2000s-present: Mobility & Clouds**

- Phones became smartphones. Apple released iOS in 2007, and Android (based on Linux) arrived in 2008.
- Cloud computing exploded, with Amazon, Google, and Microsoft offering powerful Linux-based servers on demand.
- AI integration

Why UNIX was created?

- **The Computing Context of the 1960s**

- Computers were large, expensive, and shared
- Operating systems were **complex, proprietary**, and specific to each machine. Each vendor had its own OS, and they weren't compatible
- Developers wanted **more portable, efficient**, and **multi-user** environments.

→ Existing systems were **not flexible or user-friendly** for developers.

Why UNIX was created?

- **Multics – The Ambitious Predecessor:**
 - **(Multiplexed Information and Computing Service)** was started in the **mid-1960s** by **MIT, Bell Labs, and General Electric**
 - Aimed to create a **next-generation, time-sharing** operating system that would:
 - Serve **many users simultaneously**.
 - Be **modular** and easily expandable.
- Ambitious goals:
 - Real-time performance and reliability
 - Hierarchical file system (tree-like structure).
 - Dynamic linking and segmented memory management
 - Written in a high-level language: very advanced for its time

Why UNIX was created?

- **Multics – The Ambitious Predecessor:**

But it failed

- Multics became **overly complex and hard to manage**
- **Development delays** and **high costs** led Bell Labs to **pull out in 1969**
- Although **innovative**, it was too ambitious for the hardware and software capabilities of the era.
- Ken Thompson, who had worked on Multics, wanted to build something simpler, something that captured the good ideas, but left the bloat behind. That became Unix

Why UNIX was created?



MULTICS

UNIX:

- The **idea of a time-sharing OS.**
- A **hierarchical file system.**
- **User access control and shell-based interaction.**

Multics inspired UNIX but also showed the importance of **simplicity and elegance** in system design

History of UNIX

- In **1969**, **Ken Thompson** at AT&T Bell Labs developed the first prototype of Unix on the PDP-7 which was a small machine in the early days of computing, but perfect for experimentation.

The original name was 'Unics', a pun on Multics. It eventually became Unix

What's amazing is that this project, born from a failed one, ended up shaping the future of computing.

History of UNIX

- In **1969**, **Ken Thompson** at AT&T Bell Labs developed the first prototype of Unix on the PDP-7 which was a small machine in the early days of computing.
- In **1972**, **Dennis Ritchie** at AT&T Bell Labs created the **C programming language**. C was designed to be a high-level, portable language that was still close enough to the hardware for system-level programming

History of UNIX

- In **1969**, **Ken Thompson** at AT&T Bell Labs developed the first prototype of Unix on the PDP-7 which was a small machine in the early days of computing.
- In **1972**, **Dennis Ritchie** at AT&T Bell Labs created the **C programming language**. C was designed to be a high-level, portable language that was still close enough to the hardware for system-level programming
- The process of rewriting Unix in C began around **1973**, after Ritchie and **Ken Thompson** realized that C's portability would allow Unix to run on multiple types of machines. This was a turning point: portability made Unix viral. It could spread. And that's exactly what it did.

History of UNIX

- In **1969**, **Ken Thompson** at AT&T Bell Labs developed the first prototype of Unix on the PDP-7 which was a small machine in the early days of computing.
- In **1972**, **Dennis Ritchie** at AT&T Bell Labs created the **C programming language**. C was designed to be a high-level, portable language that was still close enough to the hardware for system-level programming.
- The process of rewriting Unix in C began around **1973**, after Ritchie and **Ken Thompson** realized that C's portability would allow Unix to run on multiple types of machines. This was a turning point: portability made Unix viral. It could spread. And that's exactly what it did.

→ Rewrite Unix in C was revolutionary because it made Unix **portable** for the first time (unlike the PDP-7 or PDP-11). Unix could now be adapted to run on various machines, from mainframes to minicomputers.

History of UNIX

- In **1969**, **Ken Thompson** at AT&T Bell Labs developed the first prototype of Unix on the PDP-7 which was a small machine in the early days of computing.
- In **1972**, **Dennis Ritchie** at AT&T Bell Labs created the **C programming language**. C was designed to be a high-level, portable language that was still close enough to the hardware for system-level programming.
- The process of rewriting Unix in C began around **1973**, after Ritchie and **Ken Thompson** realized that C's portability would allow Unix to run on multiple types of machines.
- 1979: After years of development and academic distribution, **AT&T (Bell Labs)** officially **commercialized UNIX in 1979** with the release of **Version 7 Unix (V7)**.
The last true UNIX version

Open-source

Open-source

- **Open Source** means the source code of a software is made freely available, and users are allowed to **view**, **modify**, **use**, and **redistribute** it.

This idea is the foundation for Linux and many other modern tools. It's not just about cost, it's about freedom, collaboration, and transparency. Open source enables innovation by letting people around the world work together on the same software.

Open-source

- **Open Source** means the source code of a software is made freely available, and users are allowed to **view**, **modify**, **use**, and **redistribute** it.

- **BSD's Open Source Legacy:**

- In **1977**, researchers at the **University of California, Berkeley** released the **first version of BSD**, an enhanced Unix-like system based on **Unix Version 6 (V6)** from Bell Labs

- BSD began as a **set of add-ons and improvements** to AT&T's Unix, including tools like the **C shell (csh)**, improved performance, and networking utilities.

- It quickly evolved into a **full Unix-like operating system** used widely in academia and eventually in commercial products.

Open-source

• **Open Source** means the source code of a software is made freely available, and

- Because BSD was originally based on AT&T Unix code, **AT&T sued the University of California in the early 1990s**, claiming that BSD still contained proprietary Unix source code.
- This lawsuit created uncertainty in the Unix world and **delayed the adoption of free Unix-like systems**.
- Eventually, the **lawsuit was settled in 1994**, and the BSD codebase was cleaned of AT&T code, leading to the **fully open and free 4.4BSD-Lite** release.

academia and eventually in commercial products.

Open-source

- GNU project :**

- In 1983, another major player emerged: Richard Stallman launched the GNU Project

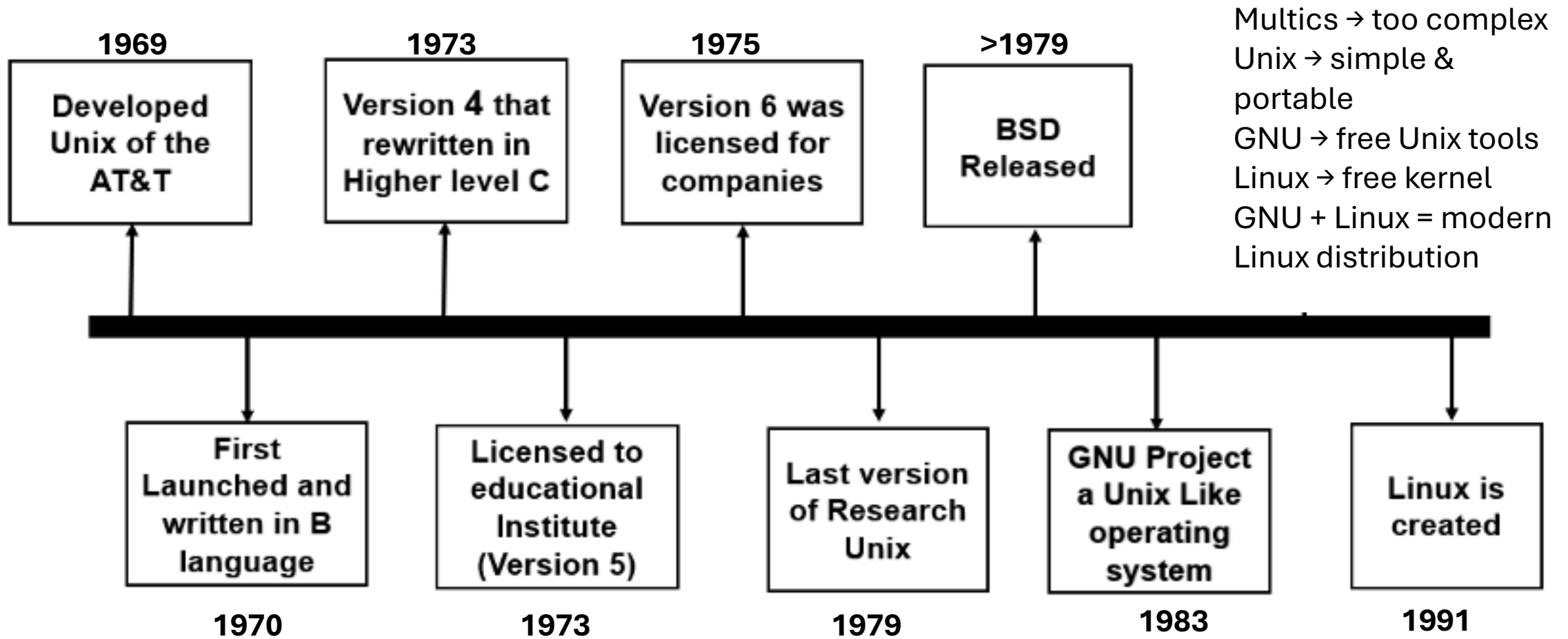
- Philosophy: Build a **completely free** Unix-like operating system.

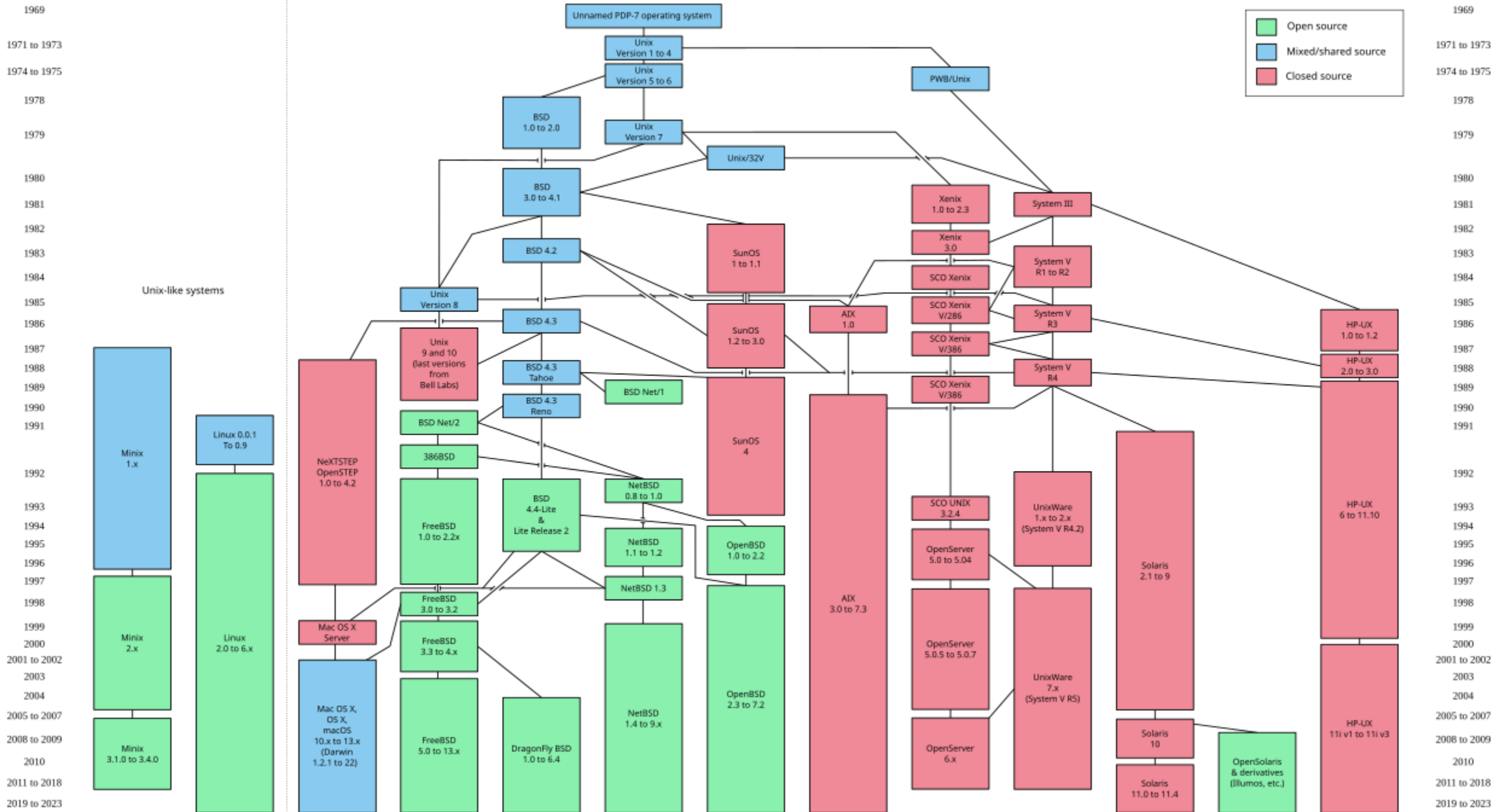
- "GNU" = "**GNU's Not Unix**".

- GNU is also a software license : allows for code modifications as long as they are shared. This ensures freedom and collaboration.

GNU wasn't a full OS at first, but it provided all components except the kernel. Its philosophy is about user freedom: to use, modify, and share software

History of UNIX



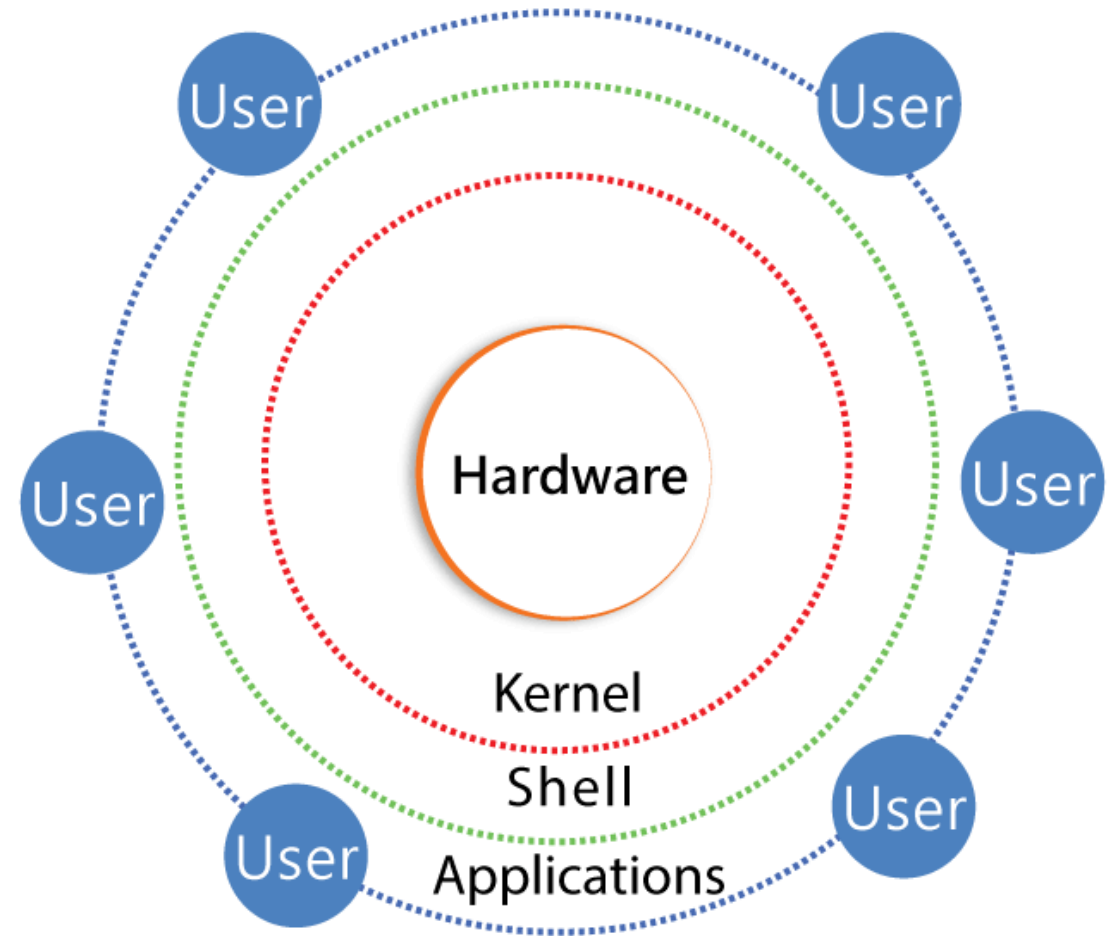


Uniqueness

- **Multitasking:** UNIX is capable of running multiple processes at the same time.
- **Multiuser support:** UNIX allows multiple users to simultaneously access the same system and share resources
- **Portability:** UNIX can run on a wide variety of hardware platforms, from small embedded systems to large mainframe computers.
- **Shell scripting:** UNIX provides a powerful scripting language that allows users to automate tasks.
- **Security:** UNIX has a robust security model that includes file permissions, user accounts, and network security features.
- **Process Tracking:** UNIX maintains a record of the jobs that the user creates (improves system performance by monitoring CPU usage and use that information to regulate disk space)

System structure

- **Hardware:** Physical machine (CPU, memory, devices) : *not part of the OS.*
- **Kernel:** Core of the OS that manages and controls hardware resources.
- **Shell:** Interface that allows users to interact with the kernel.
- **User Applications:** Programs run by users, relying on the shell and kernel to function.



Shell, terminal, prompt, command line?

- **Shell:** A program that provides a user interface to interact with the operating system. It interprets and executes commands typed by the user (e.g. bash, ksh, csh, zsh, fish) .

Shell, terminal, prompt, command line?

- **Shell:** A program that provides a user interface to interact with the operating system. It interprets and executes commands typed by the user (e.g. bash, ksh, csh, zsh, fish) .
- **Terminal:** A program that provides the environment where the user can interact with the shell (e.g. GNOME Terminal, Konsole, or xterm).

The terminal is **the interface**, and the shell is **what processes your command**

Shell, terminal, prompt, command line?

- **Shell:** A program that provides a user interface to interact with the operating system. It interprets and executes commands typed by the user (e.g. bash, ksh, csh, zsh, fish) .
- **Terminal:** A program that provides the environment where the user can interact with the shell (e.g. GNOME Terminal, Konsole, or xterm).
- **Prompt:** The text that appears in the terminal, indicating that the shell is ready to accept a command. It usually ends with a symbol such as \$ or # (for user or root, respectively) .

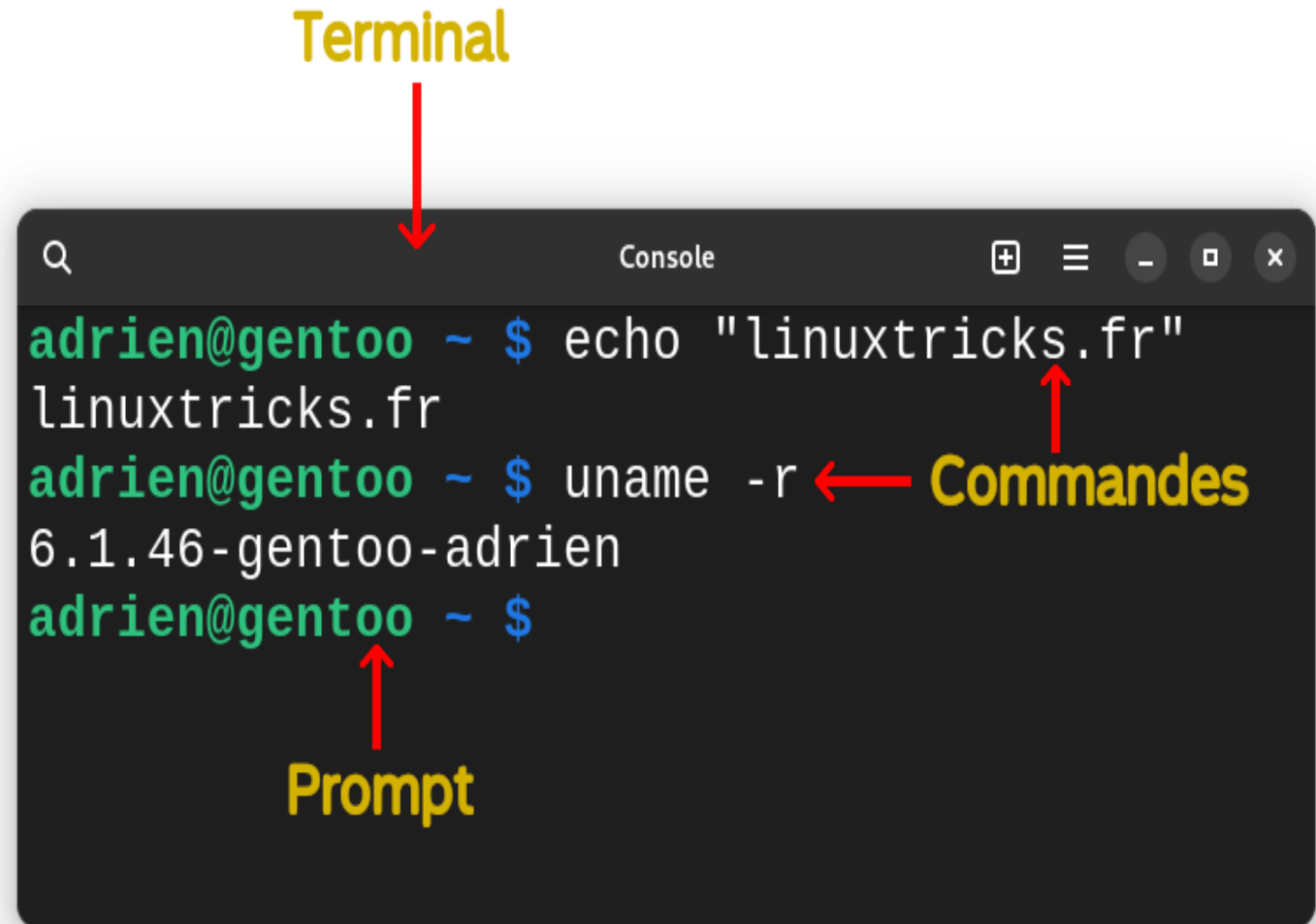
Shell, terminal, prompt, command line?

- **Shell:** A program that provides a user interface to interact with the operating system. It interprets and executes commands typed by the user (e.g. bash, ksh, csh, zsh, fish) .
- **Terminal:** A program that provides the environment where the user can interact with the shell (e.g. GNOME Terminal, Konsole, or xterm).
- **Prompt:** The text that appears in the terminal, indicating that the shell is ready to accept a command. It usually ends with a symbol such as \$ or # (for user or root, respectively) .
- **Command line:** The actual input field where you type commands in the terminal. It's the space where you write and execute commands.

Shell, terminal, prompt, command line?

So, in short:

- **Shell:** The program interpreting commands.
- **Terminal:** The program displaying the shell's prompt and results.
- **Prompt:** The text shown, waiting for commands.
- **Command line:** The area where you type your commands.



The image shows a terminal window titled "Console" with a dark background. The terminal displays the following text:

```
adrien@gentoo ~ $ echo "linuxtricks.fr"
linuxtricks.fr
adrien@gentoo ~ $ uname -r
6.1.46-gentoo-adrien
adrien@gentoo ~ $
```

Annotations with red arrows and yellow text labels are present:

- A red arrow points from the word "Terminal" (in yellow) above the terminal window to the terminal window itself.
- A red arrow points from the word "Prompt" (in yellow) below the terminal window to the prompt text "adrien@gentoo ~ \$".
- A red arrow points from the word "Commandes" (in yellow) to the command "uname -r".
- A red arrow points from the word "Commandes" (in yellow) to the command "echo \"linuxtricks.fr\"".

What is LINUX?

Linux is NOT an open source operating system on its own.

- Linux is a **kernel**, first released by **Linus Torvalds** in 1991. It's combined with GNU software to form a complete operating systems like Ubuntu or Fedora (GNU tools: like compilers, libraries, and shells)
- The code used to create Linux **is free and available to the public to view, edit**, and, for users with the appropriate skills, to contribute to.
- Unix and Linux have comparable components, including the kernel, shell, and programs.

Linux versus UNIX

- Unix and Linux are similar in many ways, and in fact, Linux was originally created to be similar to Unix. Both have similar tools for interfacing with the systems, programming tools, filesystem layouts, and other key components.
- However, Unix is not free.
- Over the years, a number of different operating systems have been created that attempted to be “unix-like” or “unix-compatible,” but Linux has been the most successful, far surpassing its predecessors in popularity.

Linux versus UNIX

Feature	Linux	UNIX
Source Code	Open Source	Mostly Proprietary
Cost	Free	Commercial License (varies by version)
Usage	Servers, Desktops, Mobile (e.g., Android)	Enterprise Servers, Workstations
Development	Community-driven (collaborative)	Vendor-specific (AT&T, IBM, Oracle, etc.)
Customization	Highly customizable	Limited customization
Examples	Ubuntu, Fedora, Debian, Arch, CentOS	AIX (IBM), HP-UX, Solaris (Oracle)
User Interface	CLI and GUI options	Mostly CLI (some with GUI)
Compatibility	Compatible with many hardware platforms	Limited to specific hardware

History of LINUX

- Linux was created in 1991 by Linus Torvalds, a then-student at the University of Helsinki. Torvalds built Linux as a free and open source alternative to Minix, another Unix clone that was predominantly used in academic settings.
- At first, he wanted to call it 'Freax', a combination of 'free' and 'freak', but the server admin who hosted it renamed the folder to 'Linux' after a combination of Torvalds' first name and the word Unix, and the name stuck.



Why LINUX?

Why LINUX?

- **Secure and Stable**

Long uptimes, strong permissions model, fewer viruses.

- **Free and Open Source**

No license fees. You can use, modify, and distribute it.

- **Community-Supported**

Thousands of contributors and online resources.

- **Widely Used**

Powers servers, supercomputers, IoT, mobile (Android), cloud infrastructure.

- **Ideal for Developers**

Built-in tools, scripting, and full control over the system.

Where is LINUX?

- **Widespread Usage :**

- **Servers** : Dominates web servers, data centers, and cloud infrastructures (AWS, Google Cloud, Azure).
- **Supercomputers** : Over 90% of the world's top supercomputers run on Linux.
- **Embedded Systems & IoT** : Linux powers devices like smartphones, routers, smart TVs, and IoT devices.

- **Linux in Consumer Devices :**

- **Android OS** : Based on Linux, used in nearly all smartphones.
- **Desktop** : Used by enthusiasts and in some enterprise environments (e.g., Ubuntu, Fedora).

- **Enterprise & Security :**

- **Enterprise Servers** : Linux is widely used for web hosting, databases, and enterprise applications.
- **Security** : Known for its robustness and security, preferred for sensitive environments.

Linux

What are the different
Linux/GNU distributions?

Linux

Over 600 Linux distributions, 300 actively developed. This diversity is a strength—it means people can tailor Linux for servers, desktops, security, privacy, education, gaming, and more. Some distros are designed for specific hardware; others for specific use cases



Linux

Lab:

Each group picks a distribution and answers (15-20min):

- Origin (date, place)
- Target audience
- Technical features
- 2min presentations with 1-2 slides (all the presentations will be added to the class)

Installation of Linux

Session 2

Installation of Linux

Session 2

Goals:

- WSL for Windows
- VirtualBox for Windows/macOS/Linux
- Post-installation configuration

Linux



Linux

Linux Distributions Focus

- **Ubuntu:** User-friendly, modern, based on Debian (Kubuntu light)
- **Debian:** Stability, minimalism, used for servers
- Why we use Kubuntu/Ubuntu for this course:
 - Easier to install
 - Light

Installing LINUX

Installation Methods

- **WSL**: Windows Subsystem for Linux (Windows only)
- **VirtualBox**: Install Linux inside a virtual machine
- **Dual Boot (Mac)**: Native Linux installation beside macOS
- Focus today: WSL and VirtualBox

Installing LINUX

Choosing the best method

Method	Best For	Comments
WSL	Beginners on Windows	Easy, quick, non-destructive
VirtualBox	Anyone with free resources	Safe, isolated environment
Dual Boot	Advanced users with backups	High risk, full hardware use

What is WSL?

WSL: Windows Subsystem for Linux

- WSL was designed by Microsoft in partnership with Canonical, the creators of Ubuntu.
- Run Linux directly inside Windows
- No need for dual boot or VM
- WSL is not intended to run GUI (Graphical User Interface) applications
- **WSL requires a 64-bit version of Windows 10**

WSL

You can install WSL from the command line. Open a PowerShell prompt as an Administrator and run:

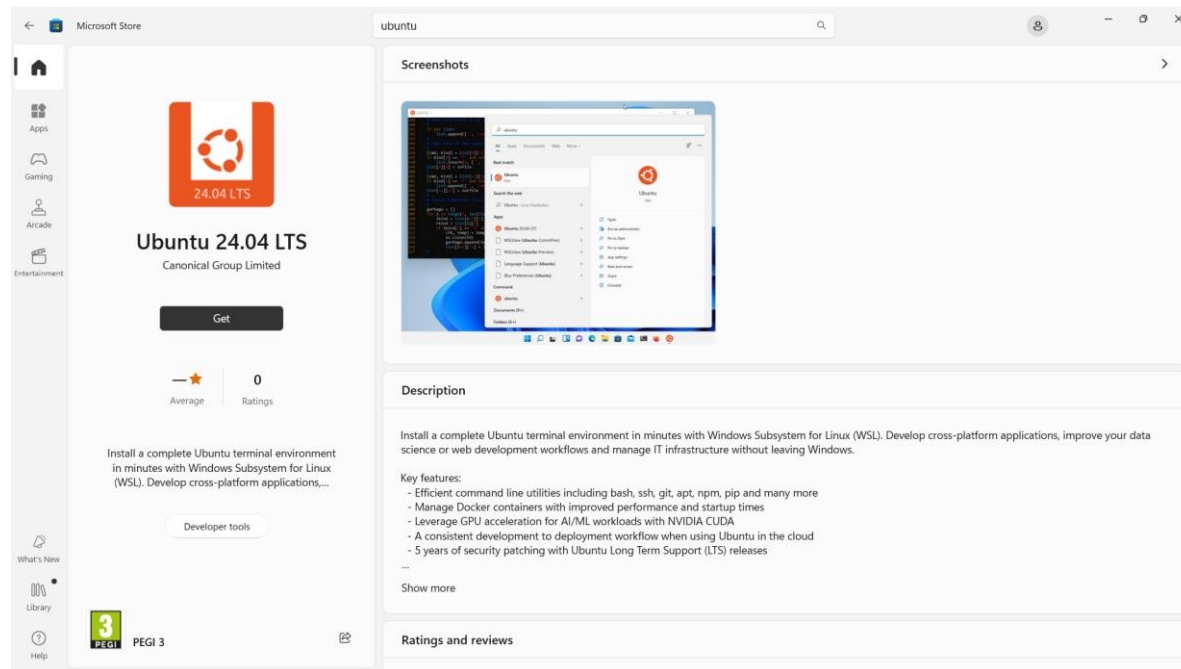
- `> wsl --install`

It is recommended to reboot your machine after this initial installation to complete the setup.

`wsl -l -v` check the version of the installed WSL

Install Ubuntu WSL

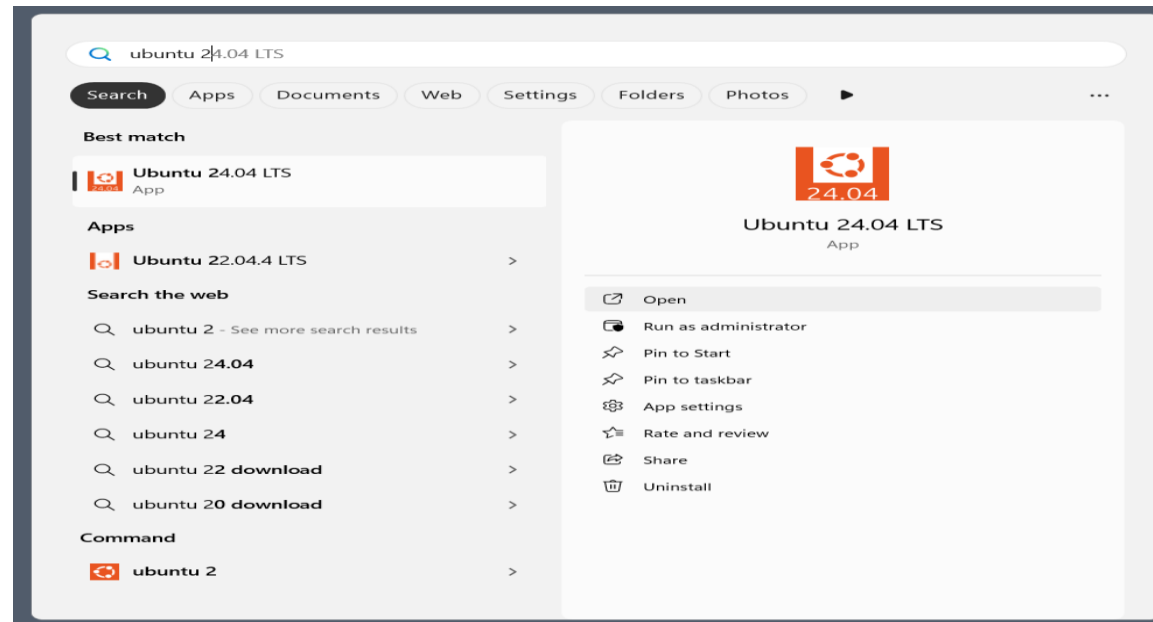
Find the distribution you prefer on the Microsoft Store and then click **Get**.



Or directly from the terminal: `> wsl --install -d Ubuntu-24.04`

Install Ubuntu WSL

Ubuntu will then be installed on your machine. Once installed, you can either launch the application directly from the Microsoft Store or search for Ubuntu in your Windows search bar



Or from the terminal: `> ubuntu2404.exe`

Uninstall Ubuntu WSL

Check the version:

```
wsl --list
```

Then

```
wsl --unregister NAME
```

What is a VirtualBox?

- VirtualBox is a cross-platform virtualization application.
- It runs on your existing Intel or AMD-based computers. It works on Windows, Mac, Linux or Solaris operating systems.
- **Enhanced Security and Stability:** Since each VM operates in its own environment, issues in one VM won't affect the others.
- It extends the capabilities of your existing computer so that it can run multiple operating systems (inside multiple virtual machines) at the same time.
- You can run Windows and Linux on your Mac via versa.
- You can install and run as many virtual machines as you like -- the only practical limits are disk space, memory and CPU.

Requierements

- **Before Installing**
- Download VirtualBox: [virtualbox.org](https://www.virtualbox.org)
- Download Ubuntu ISO: ubuntu.com/download
- Download Xubuntu ISO: [Xubuntu 22.04.5 LTS \(Jammy Jellyfish\)](#)

Installing VirtualBox?

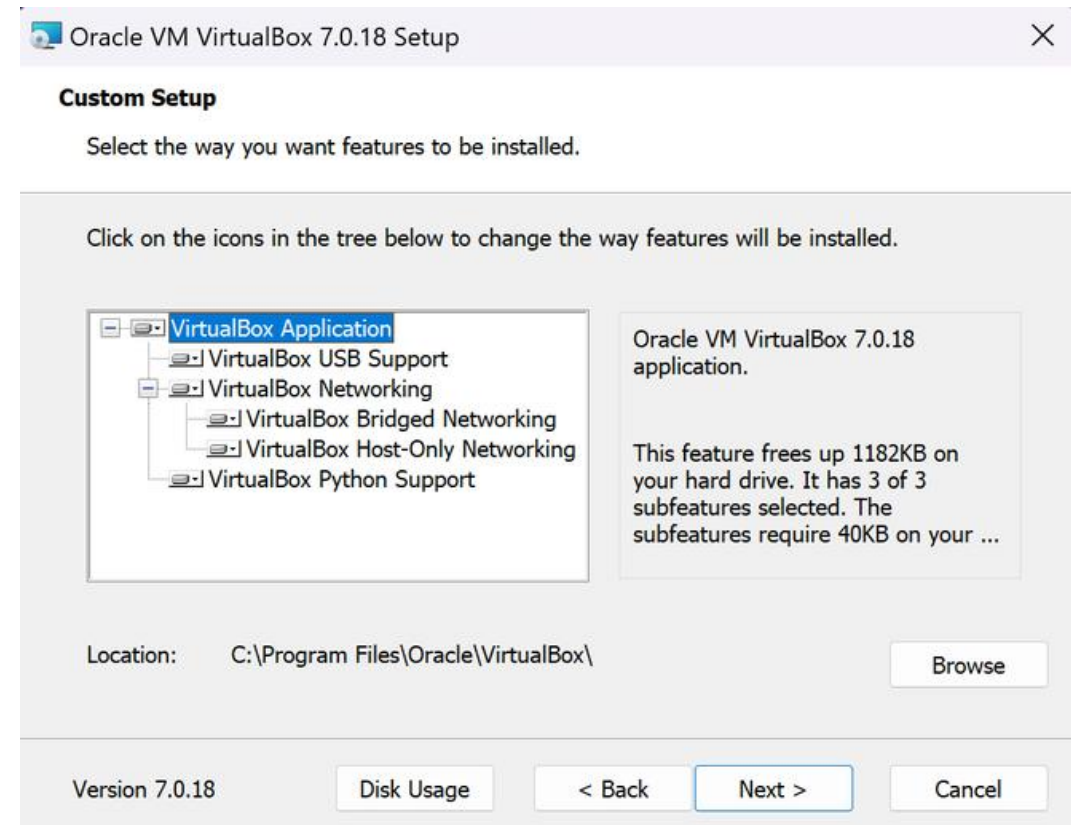
- **Step 1** : To download VirtualBox, visit the Virtualbox website and choose the right version for your OS.



Installing VirtualBox?

- **Step 2 :** Run the downloaded setup or exe file. If it asks for making changes on your device, click **Yes**.
- **Step 3:** Click **Next** on setup wizard. It will redirect you to "custom setup" dialog box where you can choose or change the features that will be going to installed on you system.

Click on **Next**, it will ask "**Proceed with the installation ?**", Click **yes** to go to next menu. Then, click on install. It will take some time to install on your system, basically depends on system to system. After that, click on finish.



If errors

- **Python core package and win32api :**
- Download Python (python.org/downloads/)
- Run the Python app and check the box to add the exe file to the PATH
- Install
- Open PowerShell/CMD and type “`py -m pip install pywin32`”
- Then `python.exe -m pip install --upgrade pip`

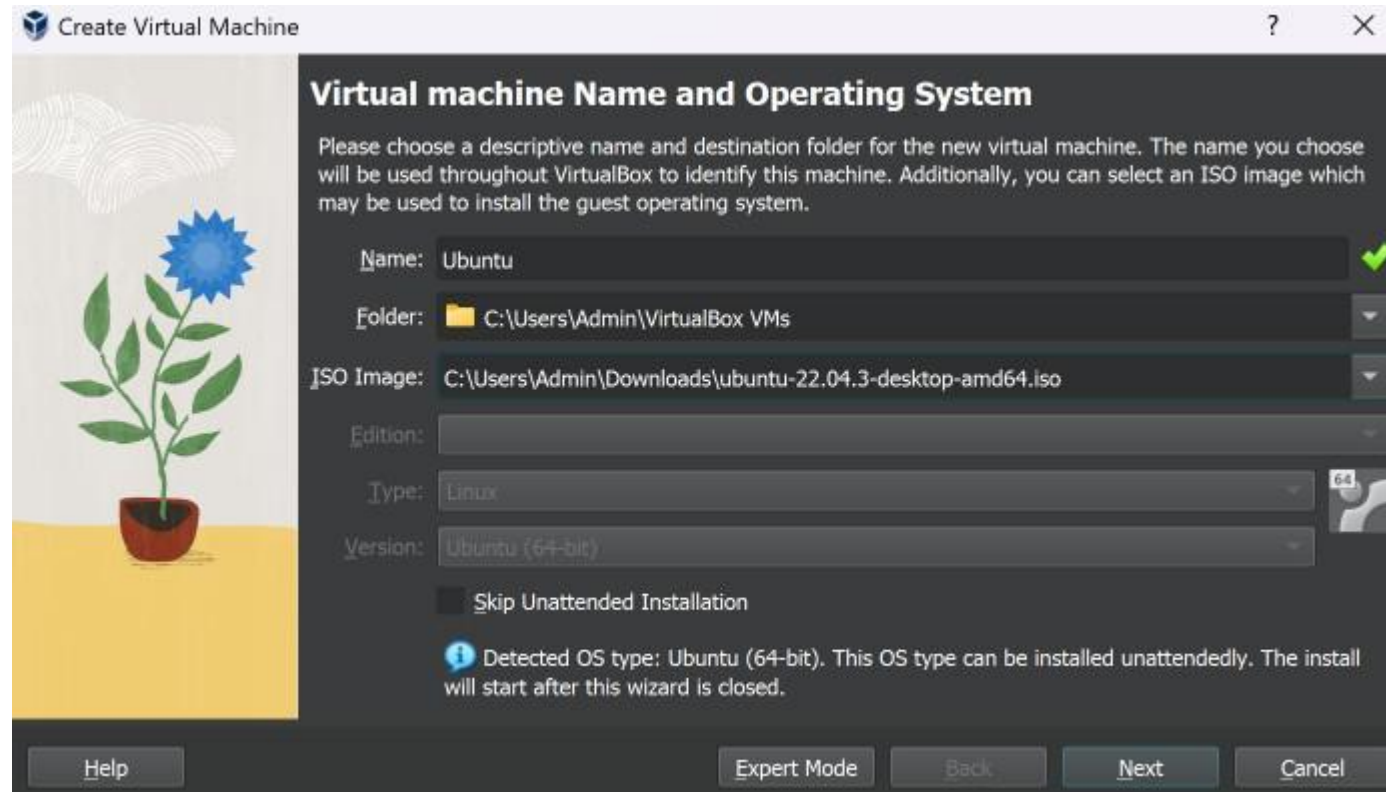
Installing Linux Using a Virtual Machine

- **Step 4:** Upon installation of VirtualBox onto your device, launching the application will result in the appearance of the Virtualbox Manager window.



Installing Linux Using a Virtual Machine

- **Step 5:** Click on the “**New**” button to generate a new VM. You will be asked for a VM name, path to where VM files will be saved and an **ISO image**. The operating system (OS) you intend to install may be like, Windows, macOS or Linux. Also, the OS version you will install, e.g., Ubuntu or Debian, will be required.



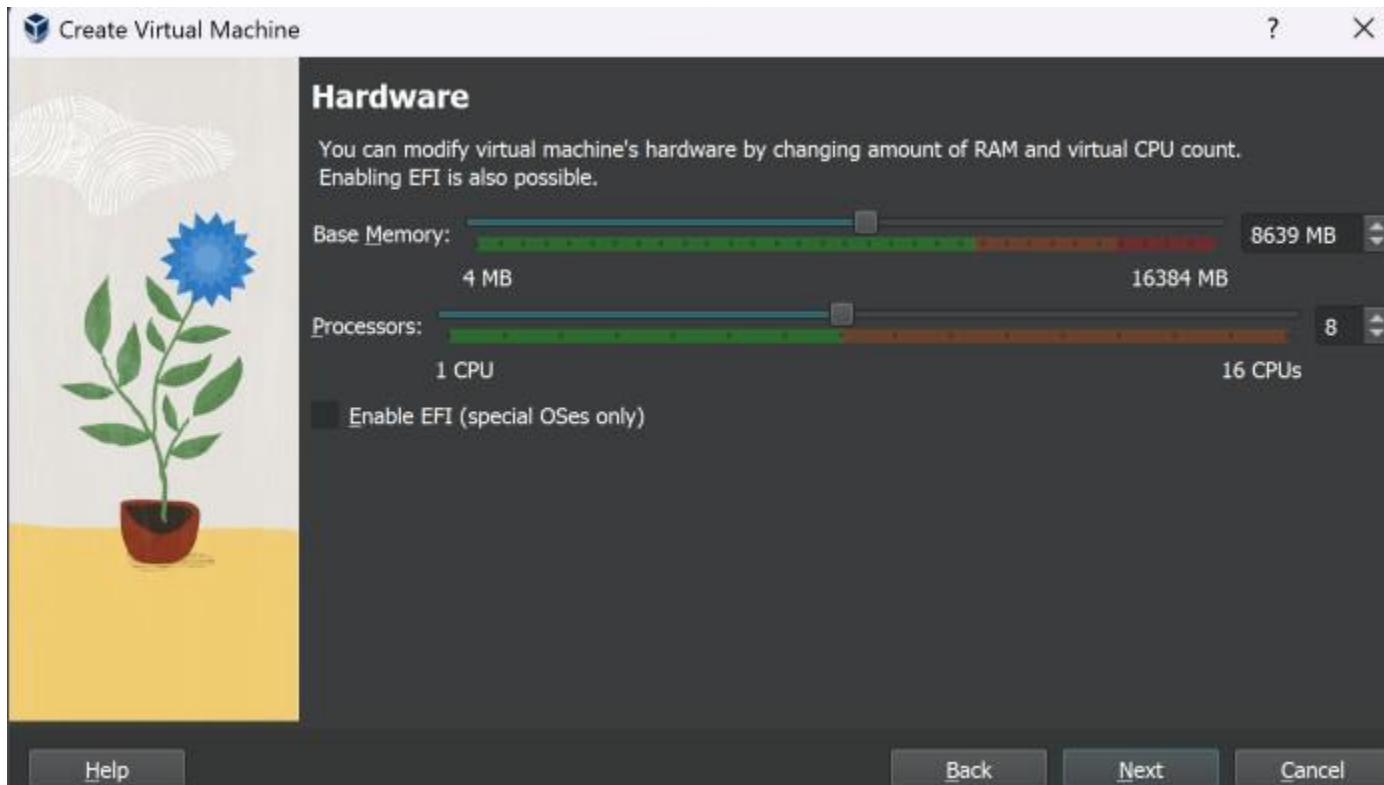
Installing Linux Using a Virtual Machine

- **Step 6:** For Ubuntu, VirtualBox supports creating a username, password, and specifying the hostname. You can decide to customize it or go with the defaults. Click on **Next**.



Installing Linux Using a Virtual Machine

- **Step 7:** Later on, in running your virtual machine, what you need is to allocate it some memory & CPU. Please plan to assign half of what system resources are there for utilization during functioning of your VM. For example, if there are 16GB of memory (RAM), it would be wise enough to allocate just 8GB of RAM.



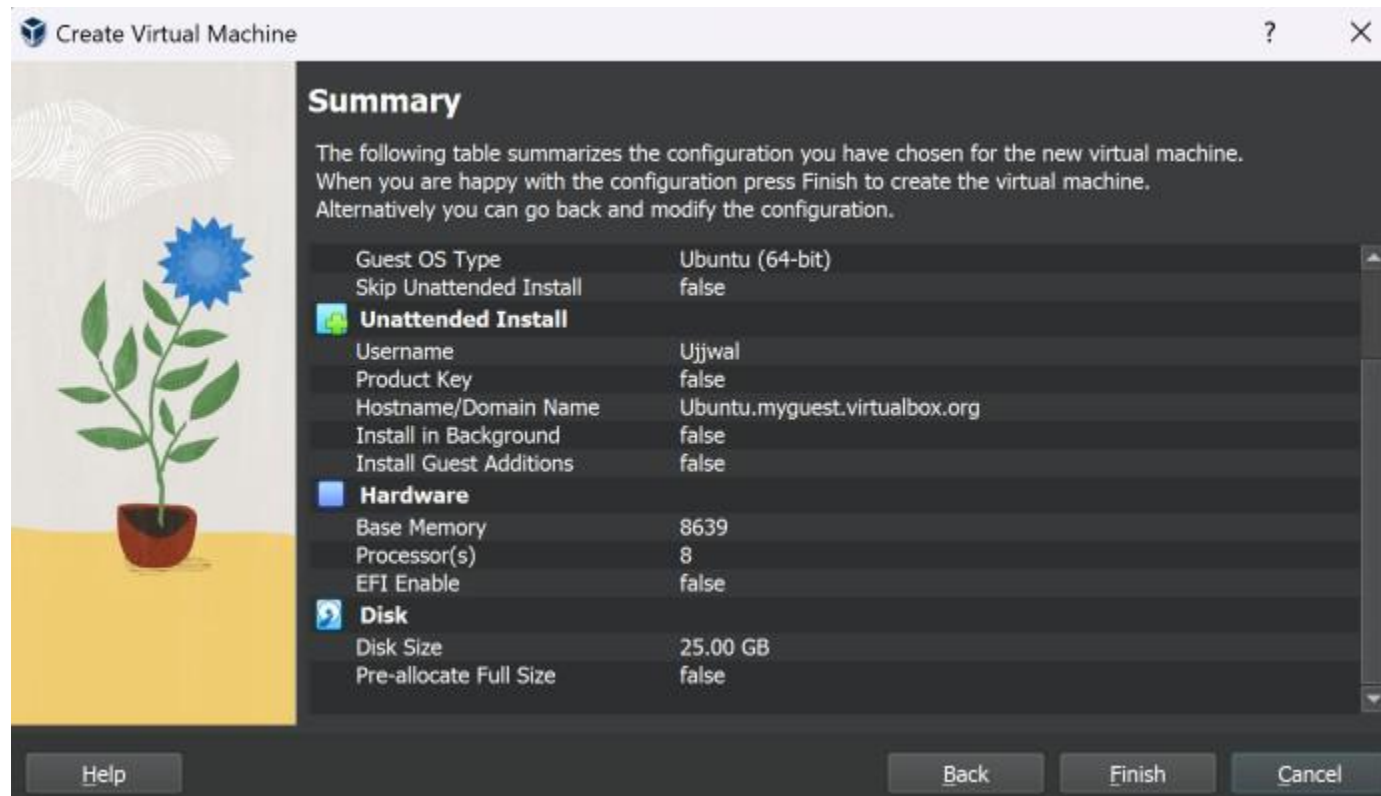
Installing Linux Using a Virtual Machine

- **Step 8:** Allocate the Hard disk size for the VM. Allocate at least 20 GB of storage. **Click** on Next.



Installing Linux Using a Virtual Machine

- **Step 9:** A summary of all of your choices is shown to you now, and it is necessary to launch and initiate the installation of the Linux OS as a virtual machine. Then Click on **Finish**.



How to remove a VM

- **Removing a Virtual Machine (VM) in VirtualBox**

1. Open VirtualBox Manager

2. Right-click the VM → **Remove**

3. Choose **Delete all files** to completely erase the virtual machine

How to access your windows

- Open **VirtualBox Manager**

Select your Linux VM → **Settings**

Go to **Shared Folders**

Add a new folder:

Folder Path: Choose your Windows directory

- Check

Auto-mount

Make Permanent

- Install Guest Additions in the Linux VM

`sudo apt update`

`sudo apt install virtualbox-guest-utils`

`reboot`

`sudo usermod -aG vboxsf $USER`

if permission denied: `sudo umount /media/sf_Thomas`

`sudo mount -t vboxsf NAMEFOLDER_SHARED /media/sf_Thomas`

Mac dual boot

Mac Users

- Dual Boot is possible
- More complex and risky
- Always backup first

full backup before attempting dual boot

Mac dual boot

Why dual boot?

- A **virtual machine** uses **only part** of the real computer's power. If you have a powerful graphics card (GPU), the VM usually **cannot access it properly**.
- A **dual boot** uses **100%** of the computer's resources.
- Full RAM.
- Full CPU performance.
- Full disk speed.
- Full GPU access (important for 3D, AI, games, heavy software)

Mac dual boot

Requirements?

- RAM: 4 GB
- Processor: 2 GHz dual-core
- Free Storage: 30 GB
- USB Drive: 4 GB
- VGA capable of 1024×768 screen resolution
- Internet access is helpful

Playing with the terminal

- **Objective:** Get comfortable using the terminal interface (CLI) for the first time.

- **Instructions:**

1. Open your terminal

1. On **Linux** or **macOS**: use Terminal
2. On **Windows**: open Ubuntu via WSL, or launch your VM

2. Type and run the following commands:

-whoami

-date

-uname -a

-ls

-echo "Hello Linux"

Playing with the terminal

- **Objective:** Get comfortable using the terminal interface (CLI) for the first time.

- **Instructions:**

1. Open your terminal

1. On **Linux** or **macOS**: use Terminal
2. On **Windows**: open Ubuntu via WSL, or launch your VM

2. Type and run the following commands:

- whoami # Displays your username
- date # Shows the current date and time
- uname -a # Shows system information
- -ls # Lists files in the current directory
- echo "Hello Linux" # Prints a message

Playing with the terminal

- **Objective:** Get comfortable using the terminal interface (CLI) for the first time.

- **Instructions:**

1. Open your terminal

1. On **Linux** or **macOS**: use Terminal
2. On **Windows**: open Ubuntu via WSL, or launch your VM

2. Type and run the following commands

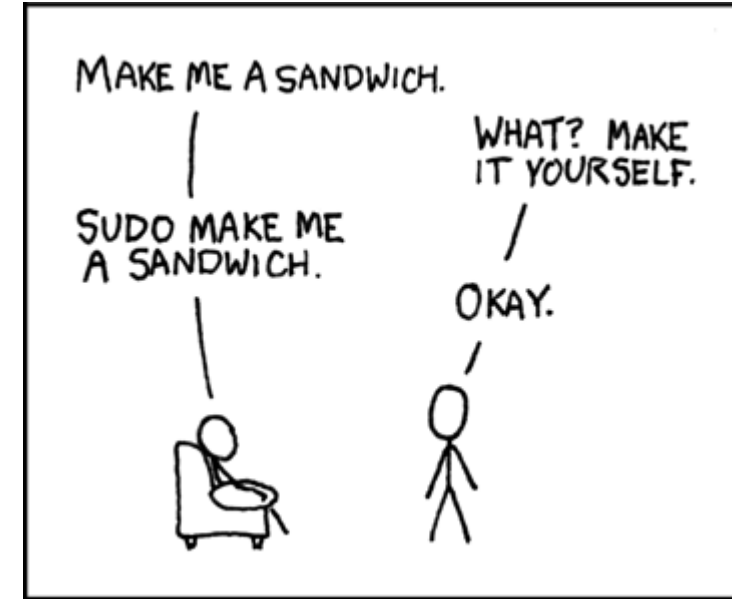
3. Questions:

- What is the name of your system?
- Are you working inside WSL, a VM, or a native system?
- What was the most surprising output?

Updating your system

- **Need to be administrator:** When you install something, use **sudo**
- `sudo apt update`
- `sudo apt upgrade`

Keep your system safe, secure, and up-to date



How to find help?

Finding Help in Linux

- Use man command → manuals inside terminal
- Use --help with any command
- Search online: forums, StackOverflow, AskUbuntu



How to find help?

Finding Help in Linux

- E.g: type “man cp”



Customize Your Linux System and Document Your Setup

For those who have already Linux.

- Help others students
 - Small project
-
- Learn to personalize a Linux system without using advanced CLI commands
 - Install non-essential applications
 - Document the system environment, installed apps, and user experience

Customize Your Linux System and Document Your Setup

Ideas:

Desktop Personalization

- Change the desktop background (wallpaper)
- Customize basic settings: theme, icons, mouse, window behavior
- Install a new theme or icons from software center (graphical app)

Install 3 Useful Applications

- From Software Center (graphical method, **NOT CLI**), install 3 new applications they would personally use:
 - Examples: GIMP, VLC Media Player, Thunderbird, Kdenlive, Geary
- Launch and test the installed applications.

Customize Your Linux System and Document Your Setup

Ideas:

Explore directory

- `ls -lh`
- `du -sh *`

Linux Architecture & File Systems

Session 3

Linux Architecture & File Systems

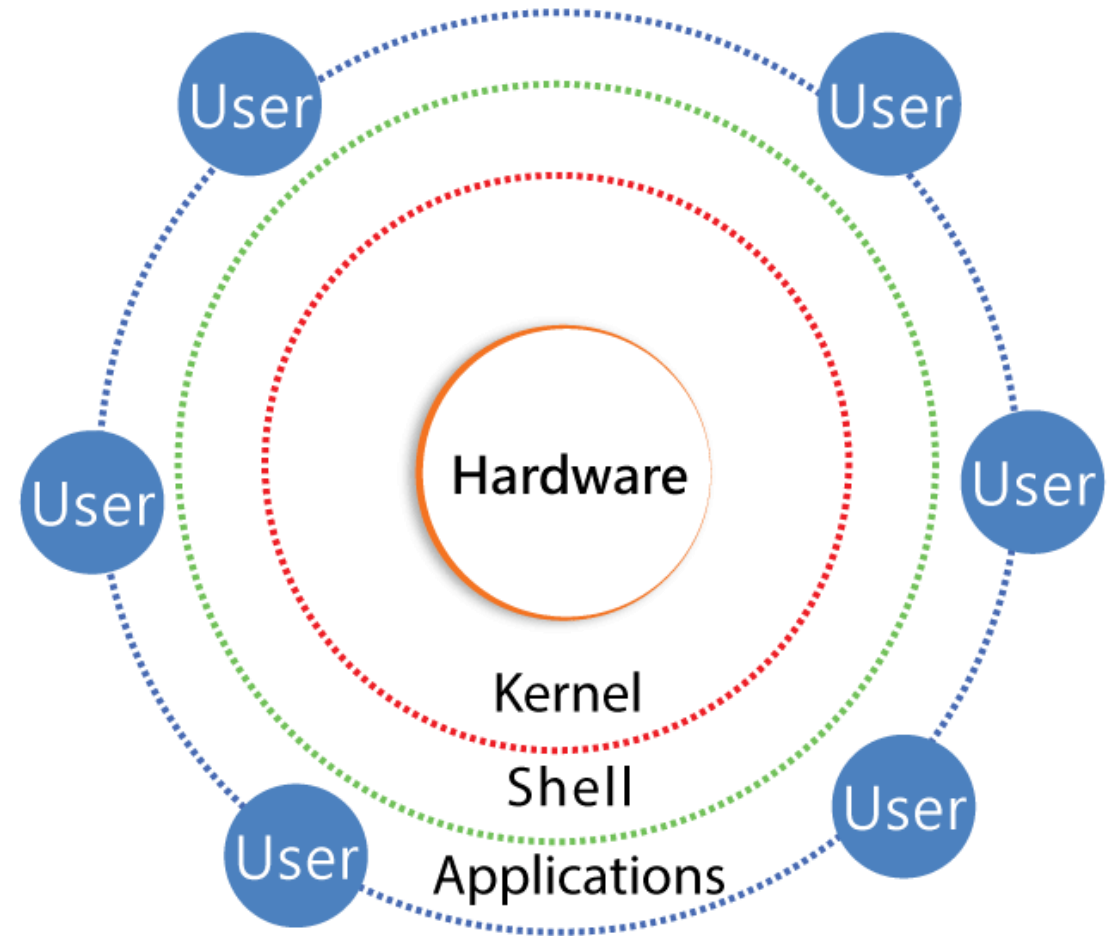
Session 3

Objectives:

- Identify Linux system components (kernel, shell, userland)
- Understand basic Linux boot process
- Explore key directories and their purposes
- Practice navigation and observation in Linux filesystem

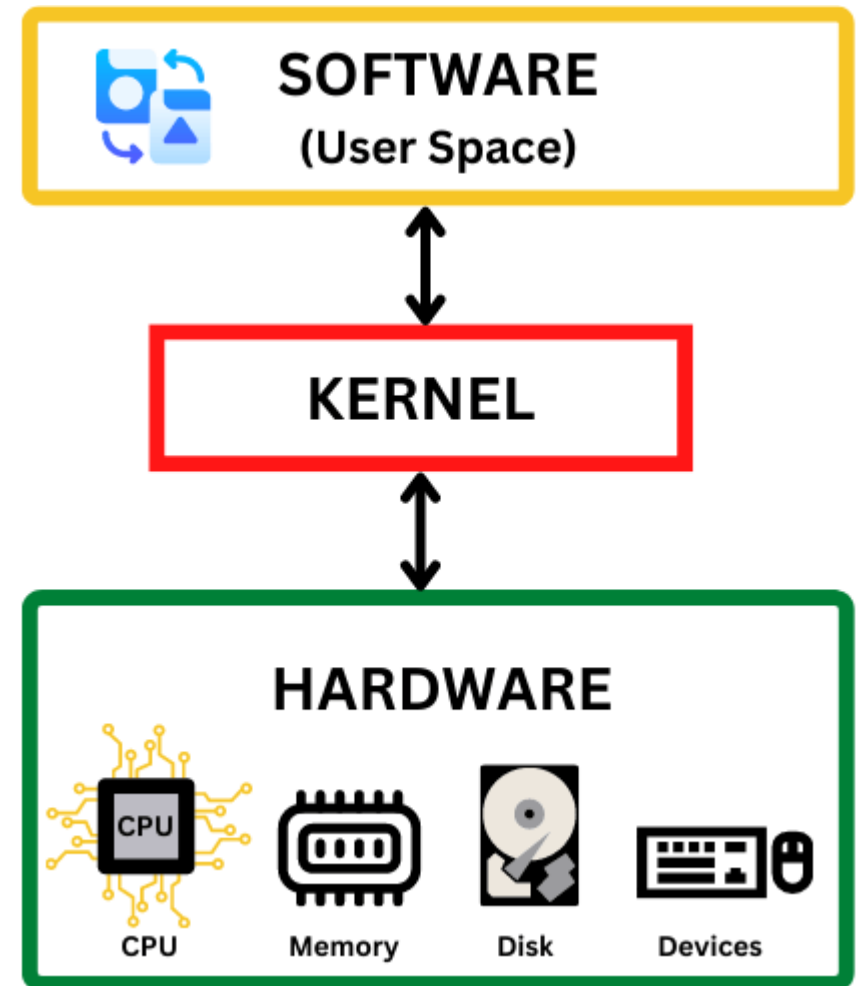
System structure

- **Hardware:**
- **Kernel:**
- **Shell:**
- **User Applications:**



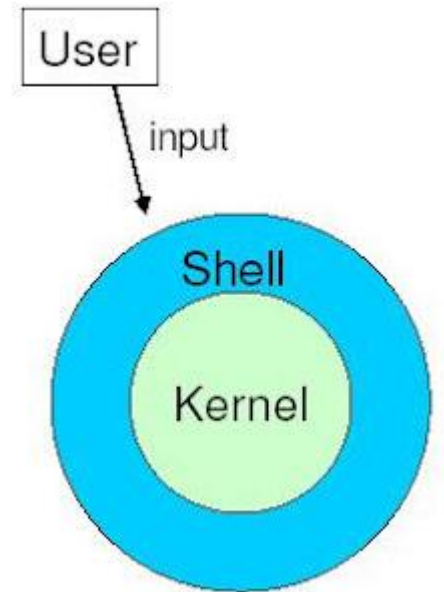
Kernel: core of Linux

- Operating system “kernel” is the core software used to “talk” to computer hardware
- It’s a core and modular system of drivers used to create a standardized environment for interfacing with hardware
- Resource manager for allocating memory and time to system and user processes as well as interacting with files (I/O)
- Kernel operates in its own memory or “kernel-space”



Shell: user interface

- On user log-in, the system runs a shell
- A shell is the environment within which you will interface with the kernel via commands
- It determines the syntax for complex command-line operations and shell scripting.
- The shell you're using is called “bash,” the successor to the venerable “Bourne Shell” called “sh”
- BASH: “Bourne Again SHell”



More Shell

- What shell am I in?

Typing “echo \$SHELL” will show you!

- You should see ‘/bin/bash’
- Typing “echo \$0” will also show your shell
- \$SHELL and \$0 are shell variables...more about variables later
- List of available shells on the system can be displayed by typing
cat /etc/shells

Shell Preference

- When you login, startup scripts are run to setup your environment
- For bash, you can customize your environment by adding or modifying environment variables and aliases in the **.bashrc** file in your home directory.
- Examples:
 - alias ls='ls -rtl' alias
 - bwulf='ssh \$USER@epita.fr'
 - PATH=\$PATH:/data/myusername
 - EDITOR=/usr/bin/vim

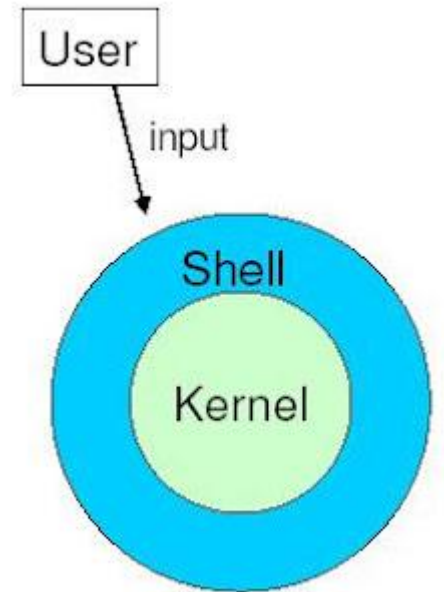
Different Shells

- sh – the original UNIX shell (Bourne shell)
- bash – written as a replacement/extension of sh
- csh – C shell based on the C programming language developed in the late 1970s
- tcsh – enhanced version of C shell
- ksh – Korn shell developed in the early 1980's, backward compatible with sh, but with some features of csh
- zsh – extended version of sh, developed in 1990
- dash – developed as replacement for ash in Debian



Applications

- Applications: browsers, text editors, games
- Services: web servers, mail servers
- Everything running on top of shell and kernel



Boot process

What really happens behind the scenes
When you press that power button to on!!

```
Configuring ISA PNP
Setting system time from the hardware clock (localtime).
Using /ect/random-seed to initialize /dev/random.
Initializing basic system settings ...
Updating shared libraries
Settings hostname: server23.kernel.org
INIT: Entering runlevel: 4
rc.M ==> Going multiuser...
Starting system logger ... [OK]
Initialising advanced hardware
Setting up modules ... [OK]
Initialising network
Setting up localhost ... [OK]
Setting up inet1 ... [OK]
Setting up route... [OK]
Setting up loca fancy console and GUI
Loading fc-cache ... [OK]
rc.vlinit ==> Going to runlevel 4
```

Viewing the Boot Process

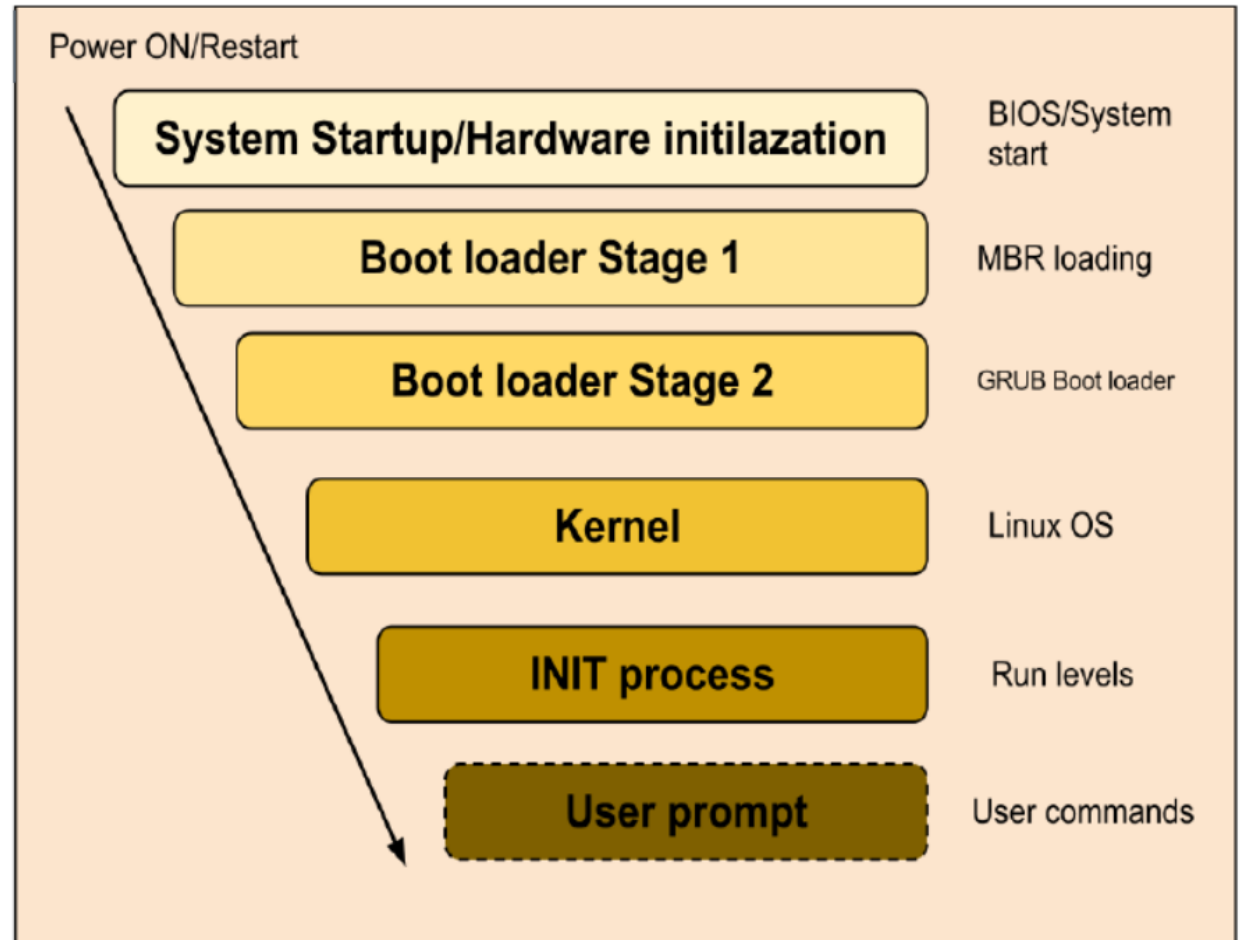
Boot kernel messages are stored into

- a buffer in memory, called the kernel ring buffer
- could be reviewed using the dmesg command; `sudo dmesg`
- a log file, usually in the `/var/log` folder
- `Sudo more /var/log/boot.log`

How Linux boots?

A quick view of booting sequence:

- Power on
- CPU jumps to BIOS
- BIOS runs POST
- Finds first bootable device
- Load and execute MBR
- Load OS
- User prompt



BIOS/UEFI Initialization

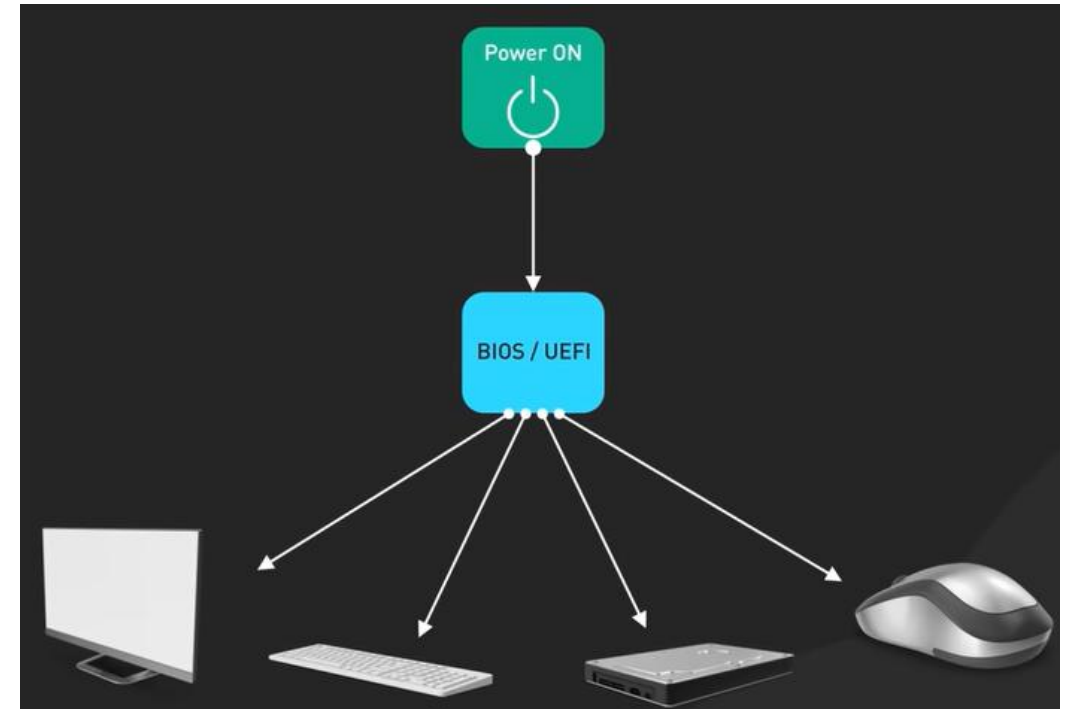
First a program called the BIOS or UEFI boots up.

These are basically pieces of software that get all the main parts of your computer ready for action: keyboard, screen, hard drive, and so on.

To access your BIOS setup screen, you can usually just press Esc, F2, F10

Basic Input/Output System vs Unified Extensible Firmware Interface:

- Boot larger drives: UEFI removes BIOS size limits and supports booting from drives up to 9.4 zettabytes.
- Faster startup: UEFI provides a more efficient, quicker boot process than traditional BIOS.
- Improved security: UEFI

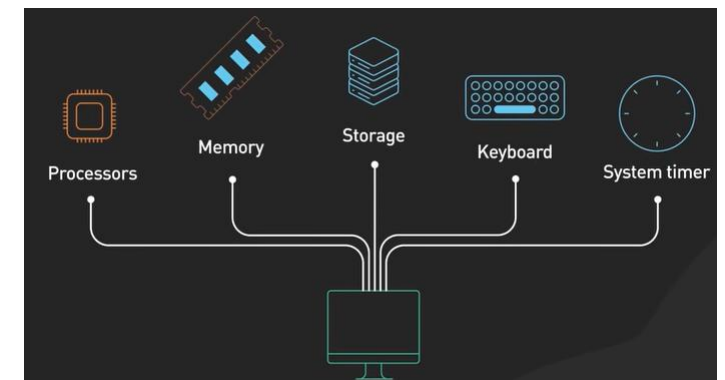
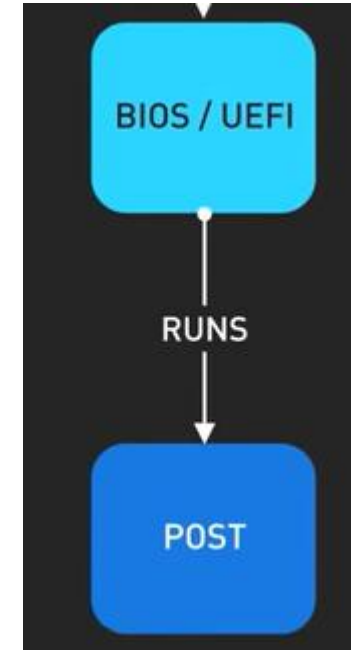


BIOS/UEFI Initialization

Next, the BIOS/UEFI runs a check called the power-on self-test (POST)

- Power-On Self-Test (POST):** basic device check on the most important hardware components which includes the RAM, CPU, and storage devices (HDD, SSD, and USB) before turning on
- To check if hardware is available for the present booting or not it will send an electric pulse to each and every device in the list that it already have. If an electrical pulse is returned from that device it will come to a conclusion the hardware is working fine and ready for use.

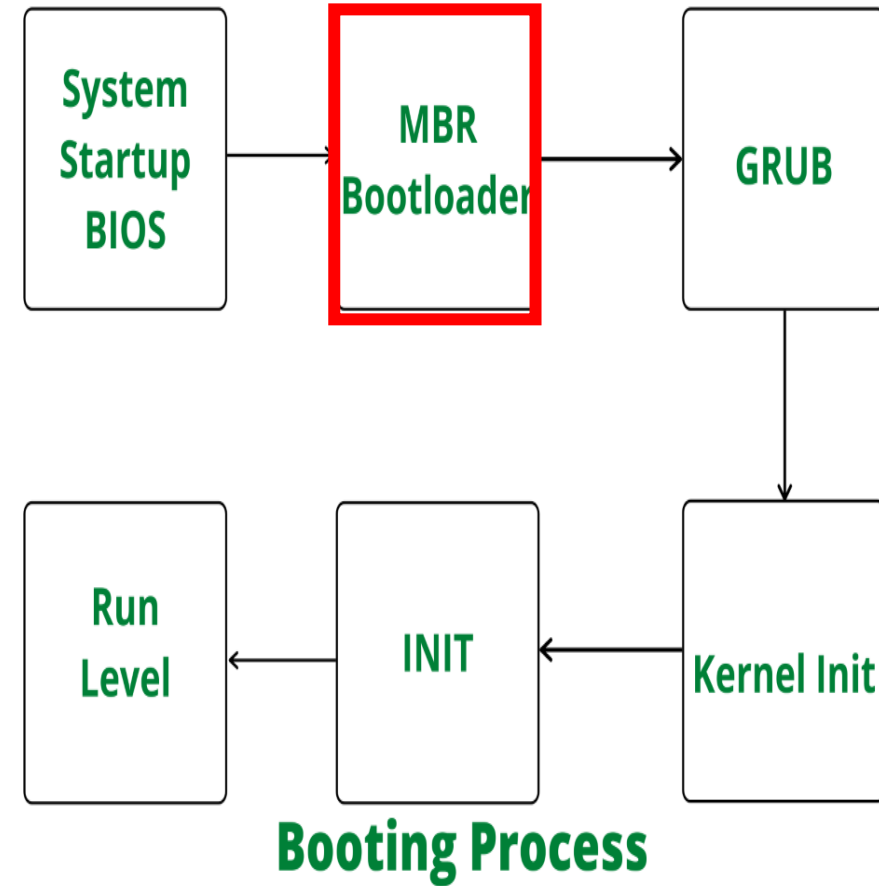
The BIOS then hands control back to the CPU to load the bootloader from that device. If the first device isn't available or doesn't contain a valid bootloader, it tries the next one in the list, then the next. If none of the devices can boot, the BIOS displays an error such as **“No boot device found.” (look hardware, cdroom..)**



MBR Bootloader

- MBR stands for **Master Boot Record** contains a **small program** that loads the full bootloader (like GRUB).
- Only 512 bytes!!
- located in the 1st sector of the bootable disk, which is typically **/dev/hda, or /dev/sda**.
- The MBR also contains information about GRUB

Its job: **find and load the next-stage bootloader** (e.g., GRUB).



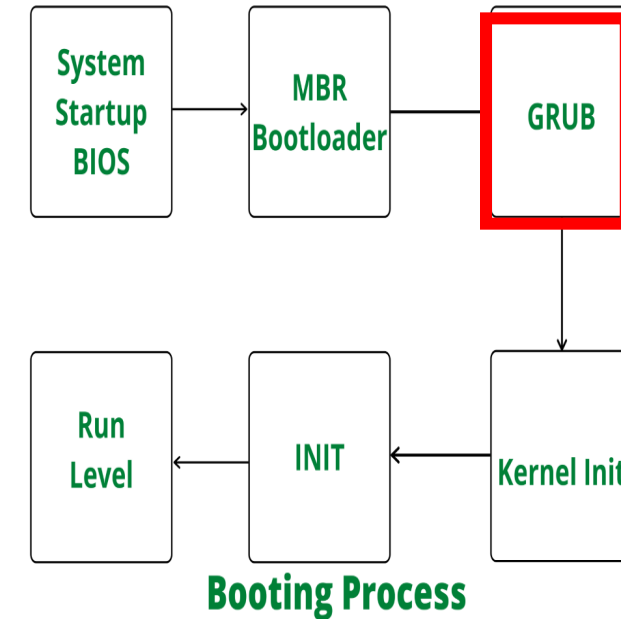
GRUB: GRand Unified Bootloader

Once the Bootloader stage 1 is completed and able to find the actual bootloader location, Stage 1 bootloader start second stage by loading Bootloader into memory

- GRUB is the most common (LILO, syslinux, etc..) and it is the first thing you see when you boot your computer. It has a simple menu where you can select some options (e.g. kernel).

```
Ubuntu 8.04, kernel 2.6.24-16-generic
Ubuntu 8.04, kernel 2.6.24-16-generic (recovery mode)
Ubuntu 8.04, memtest86+
Other operating systems:
Windows Vista/Longhorn (loader)
```

GRUB loads the user-selected (or default) kernel into memory and passes control on to the kernel. If user do not select the OS, after a defined timeout GRUB will load the default kernel in the memory for starting it

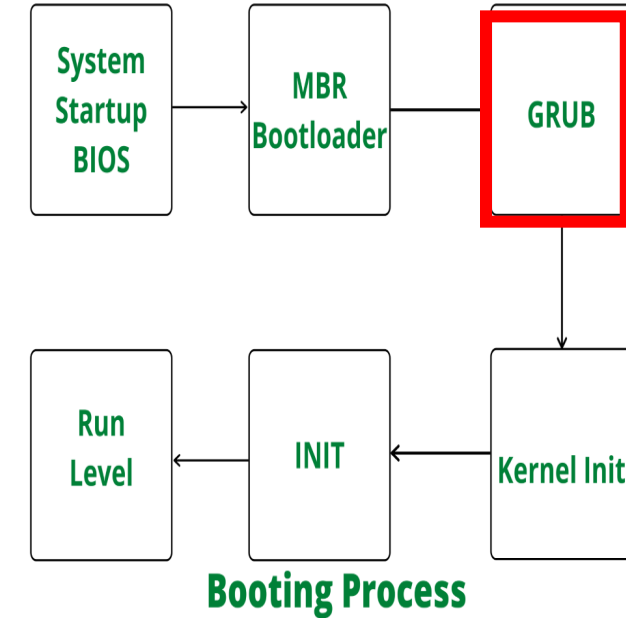


GRUB: GRand Unified Bootloader

GRUB configuration file at /boot/grub/grub.cnf or /etc/grub.conf

```
#boot=/dev/sda
default=0
timeout=5
splashimage=(hd0,0)/boot/grub/splash.xpm.gz
hiddenmenu
title CentOS (2.6.18-194.el5PAE)
    root (hd0,0)
    kernel /boot/vmlinuz-2.6.18-194.el5PAE ro root=LABEL=/
    initrd /boot/initrd-2.6.18-194.el5PAE.img
```

default=0 # Which menu entry to boot first (0 = first entry)
timeout=5 # Seconds to wait before auto-booting default entry
splashimage=(hd0,0)/boot/grub/splash.xpm.gz # GRUB background image
hiddenmenu # Hides the menu unless a key is pressed

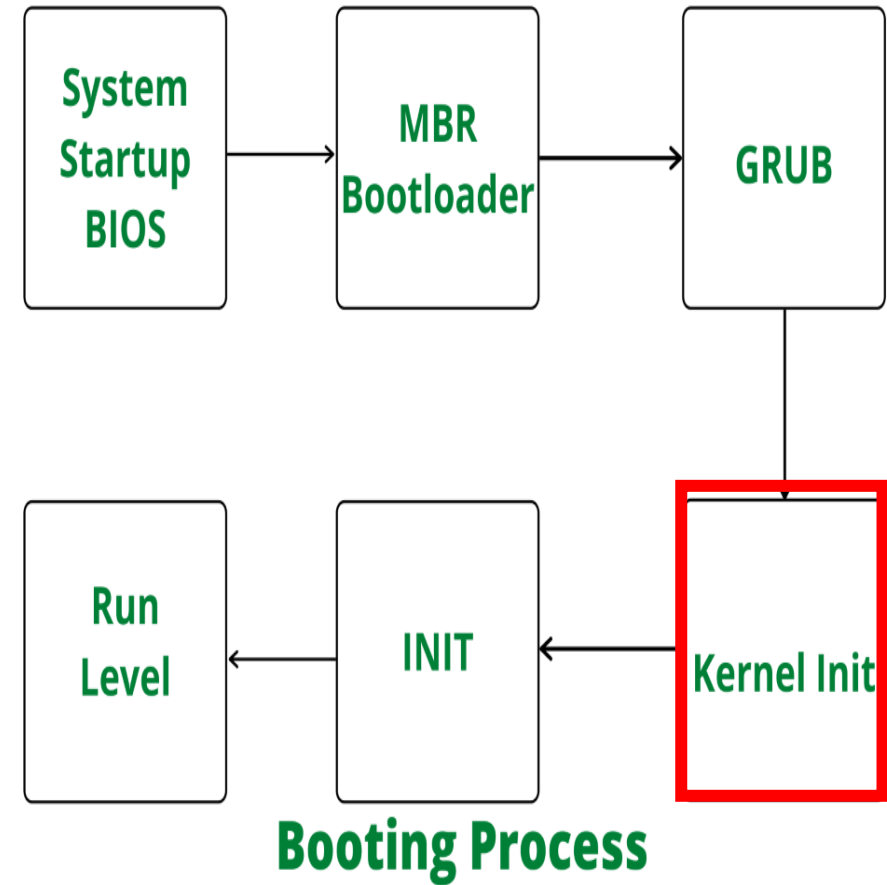


Kernel init

Once grub loads itself up it inserts the linux kernel into memory and hands control over to the kernel to finish the startup process

The kernel takes over computer's resources and starts initiating all the background processes and services

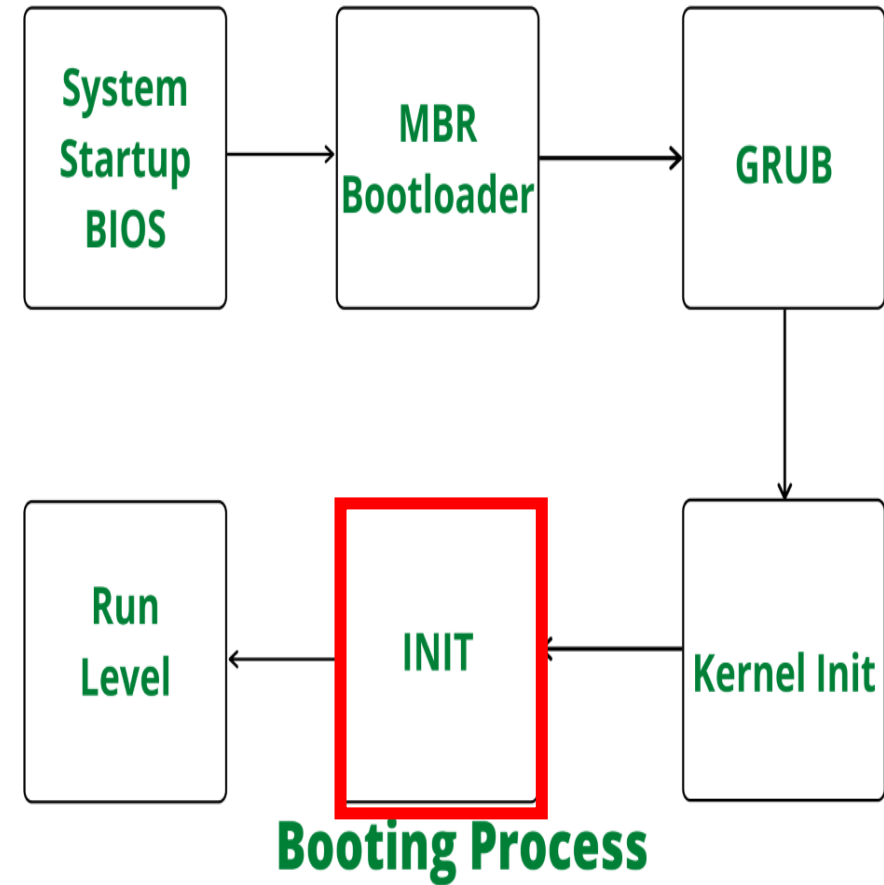
- it executes the /sbin/init program, which is always the first program to be executed. You can confirm this with its process id (PID), which should always be 1 (ps -ef | grep init)



INIT

init system:

- It is the first user-space process started by the kernel after system initialization.
 - It always has PID 1, making it the ancestor of all other user-space processes.
 - Its job is to bring the system from boot to a fully usable state.
 - On modern Linux systems, init is usually replaced by systemd
-
- Starts essential services and daemons like sshd (remote login), cron (scheduled tasks), and network services (DHCP, DNS).
 - Mounts file systems such as /home and /var, ensuring all partitions are accessible before services start.
 - Initializes system logging with tools like rsyslog or journald to capture events and errors.
 - Configures networking, bringing up interfaces and assigning IP/DNS settings.
 - Enters the default system state (runlevel or systemd target) and starts login managers (getty, GDM), enabling user access.



Run Level

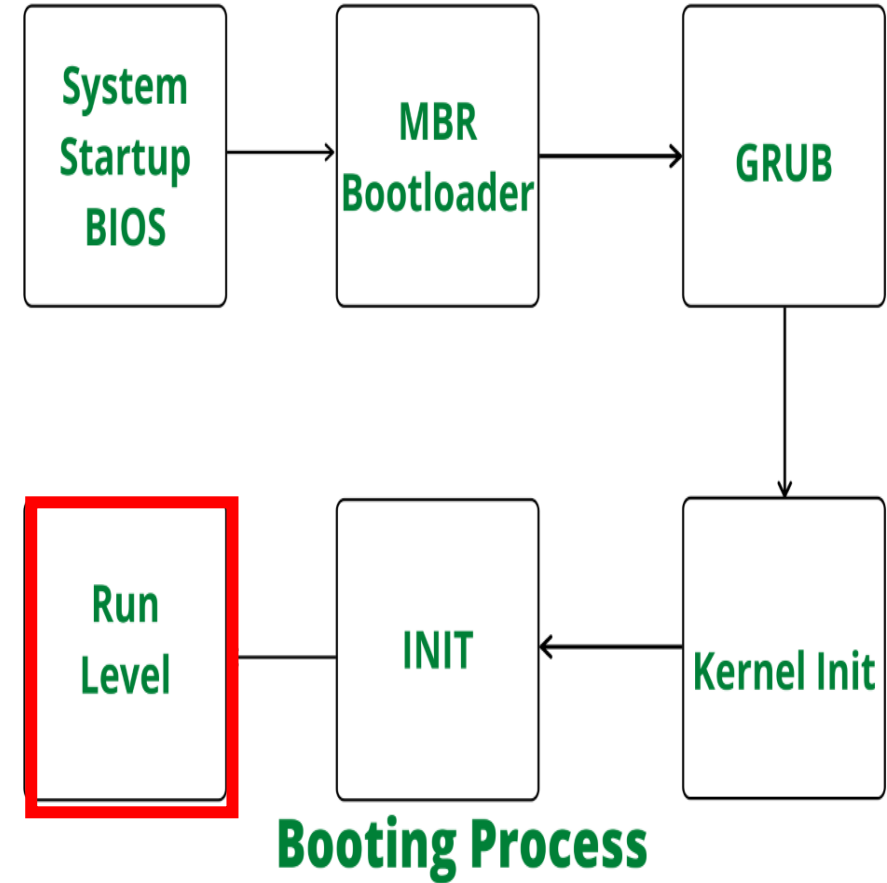
A runlevel defines the state of the system and which services are running.

Examples:

- 0: Halt
- 1: Single-user (rescue)
- 3: Multi-user (no GUI)
- 5: Multi-user with GUI
- 6: Reboot

With systemd, these are replaced by targets (e.g., multi-user.target, graphical.target).

- 0 → poweroff.target
- 1 → rescue.target (Minimal services, mostly for maintenance or recovery. No networking)
- 3 → multi-user.target (Full multi-user mode without GUI)
- 5 → graphical.target (Full multi-user with GUI)
- 6 → reboot.target
- Emergency → emergency.target



Linux file system structure

What is a file system?

Linux file system structure

What is a file system?

- Organizes data on disk
- Stores files and directories
- Defines access permissions

Unix File System is a logical method of organizing and storing large amounts of information in a way that makes it easy to manage

Linux file system structure

Popular File Systems

- **FAT** (File Allocation Table)
 - Old file system used by MS-DOS, early Windows
- **NTFS** (New Technology File System)
 - Modern Windows file system (permissions, encryption)
- **ext (ext2/ext3/ext4)** (Extended Filesystem)
 - Common file system for Linux
- **HFS+** (Hierarchical File System Plus)
 - Older macOS file system
- **APFS** (Apple File System)
 - New file system for modern macOS and iOS

Linux file system structure

What is a file?

Linux file system structure

What is a file?

- **Ordinary files (-):** text, data, or program information. (avoid space or special character in name)
- **Directories (d):** containers or folders that hold files, and other directories. They do not contain files, only entries that point to them.
- **Devices:** In a Windows-like operating system, devices like CD-ROM and hard drives are represented as drive letters like F: G: H, whereas in the Linux system, devices are represented as files. As for example, /dev/sda1, /dev/sda2, and so on.
- **Links (l):** Shortcuts or references to files (hard/soft). A **hard link** is a second name for a file. A **symbolic (soft) link** is more like a shortcut pointing to another file or folder.

EVERYTHING IS A FILE

(even directories, devices, and links)

Linux file system structure

File convention?

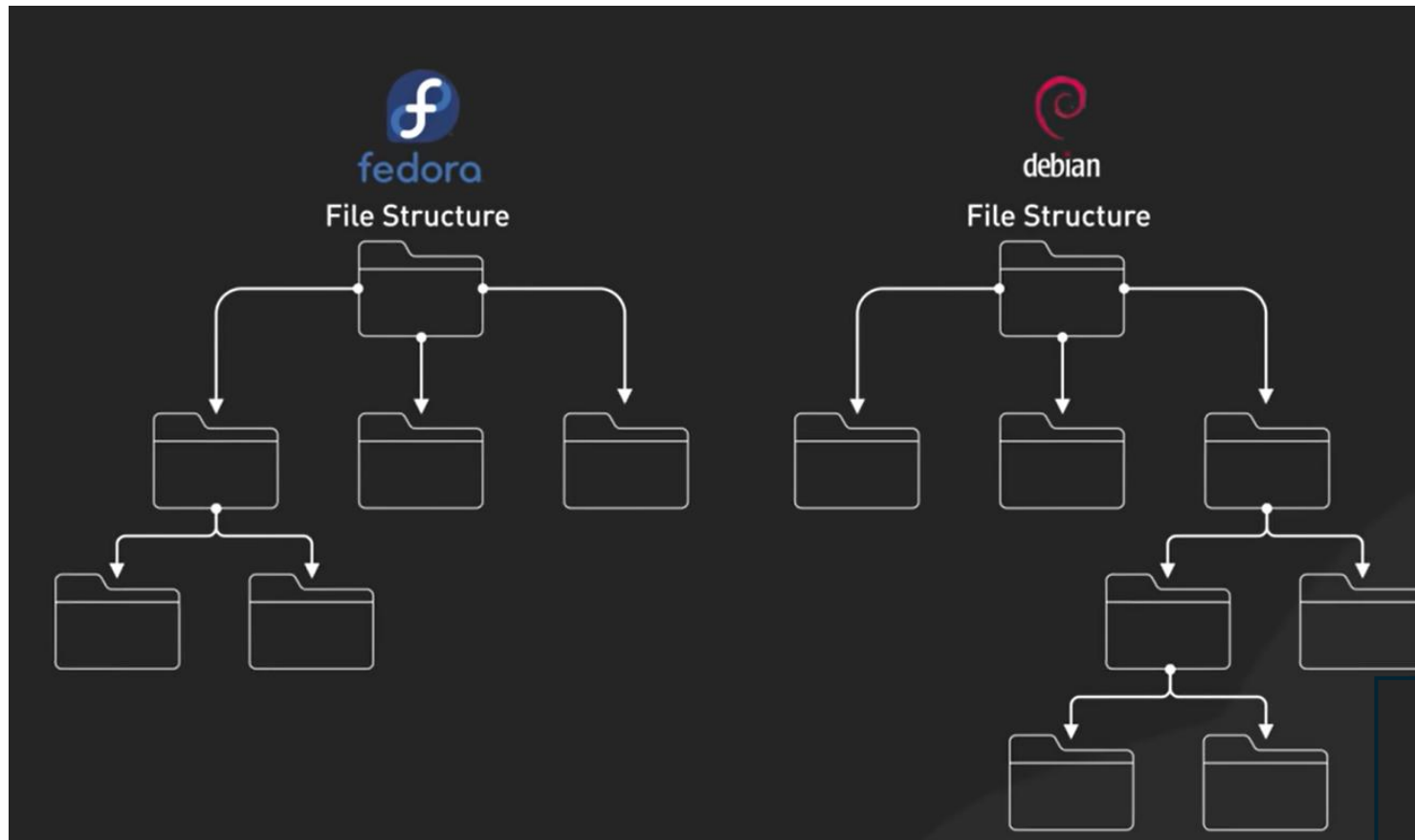
- Filenames are **case-sensitive**. For example, '**File.txt**' and '**file.txt**' are considered two distinct files
- Special characters like `/`, `*`, `?`, `:`, `$`, `%`, are not typically allowed in filenames.

Directory structure

How does linux organize the folders?

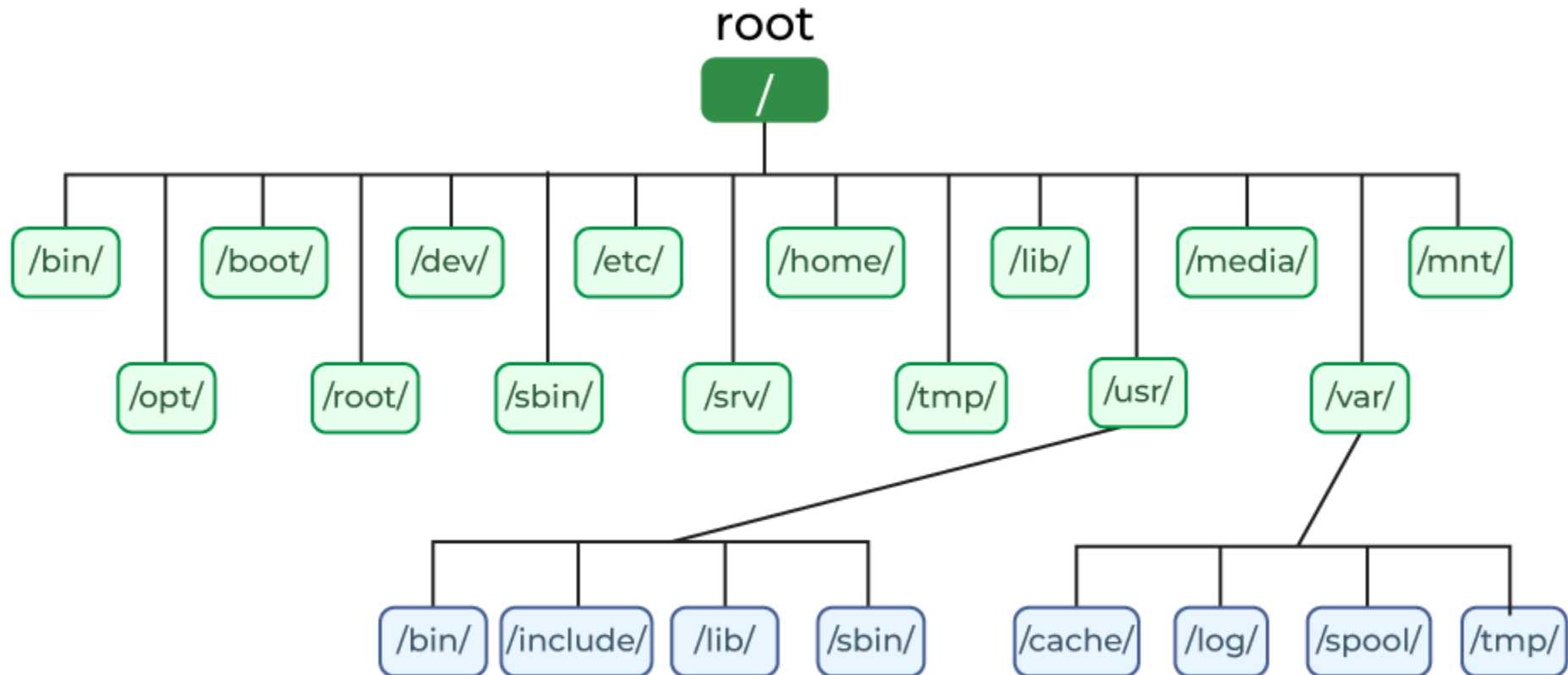
Directory structure

At the beginning, each distribution has its own organization!!



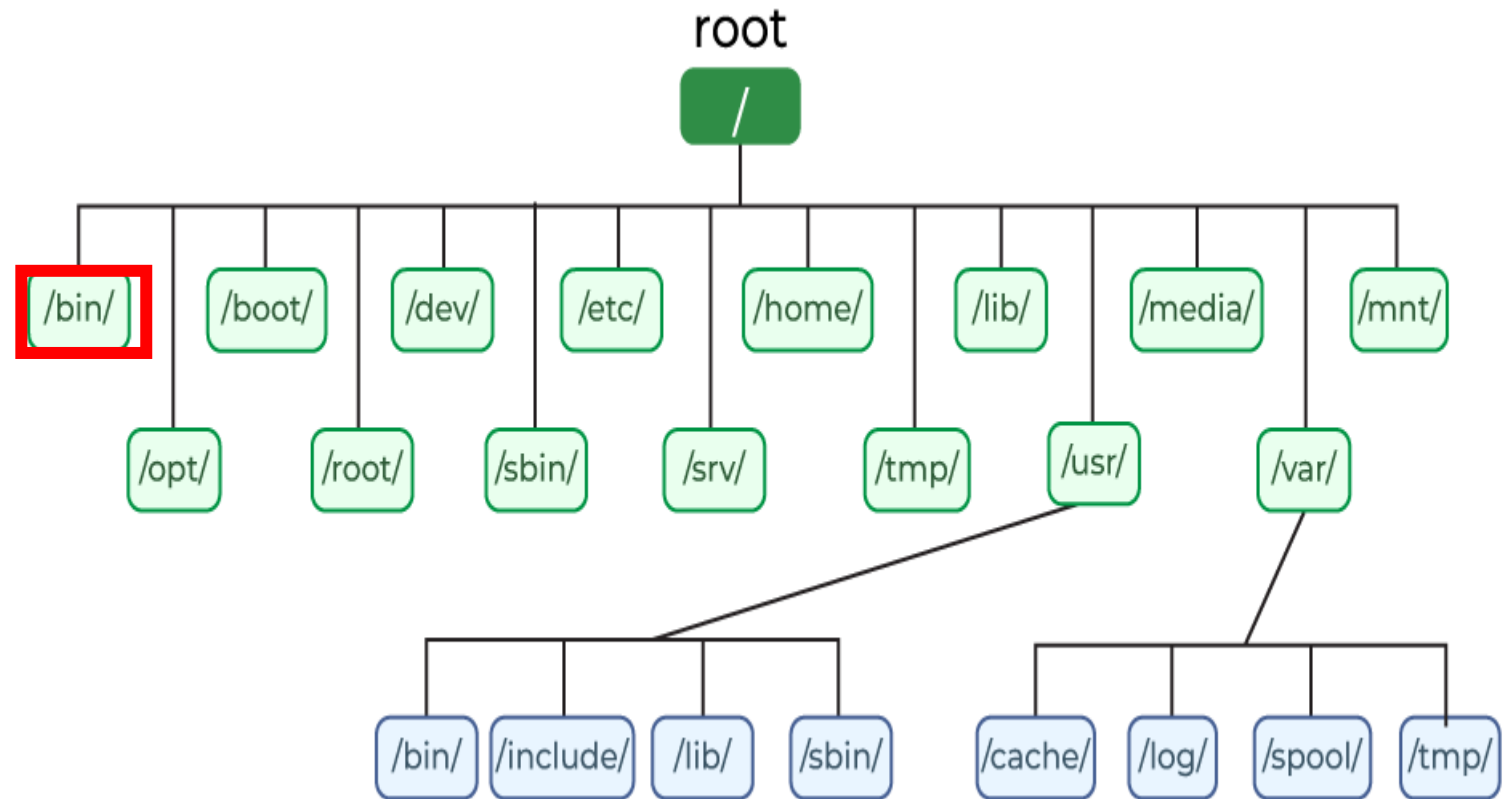
Directory structure

hierarchical tree structure which is anchored at a special top-level directory known as the root (designated by a slash '/').



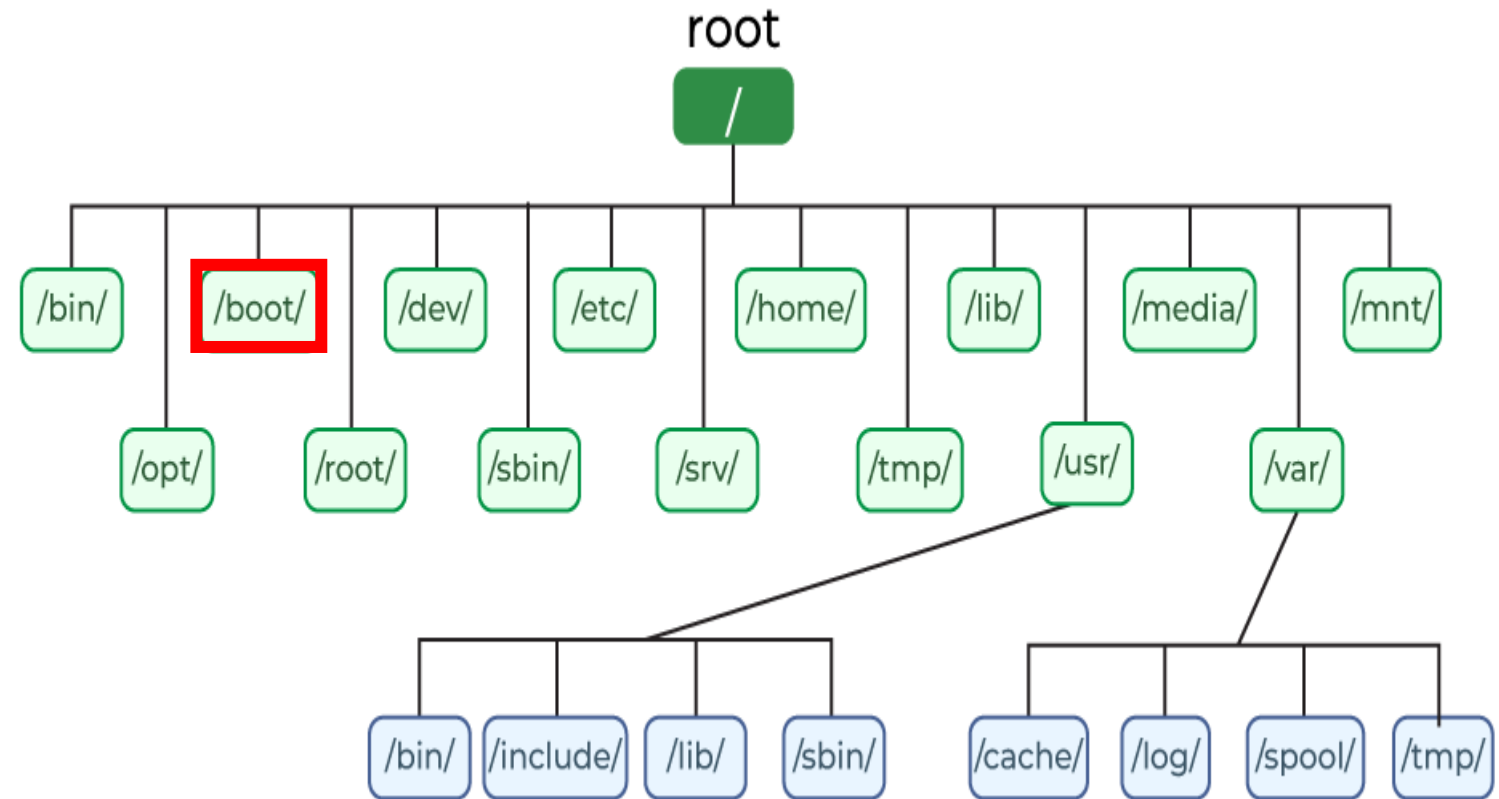
Directory structure

- /bin:
- Contains basic commands needed by all users
- Required for the system to boot and run in minimal mode
- Available even if other filesystems (like /usr) are not mounted
- Holds simple command-line tools you use every day cat, ls, cp.



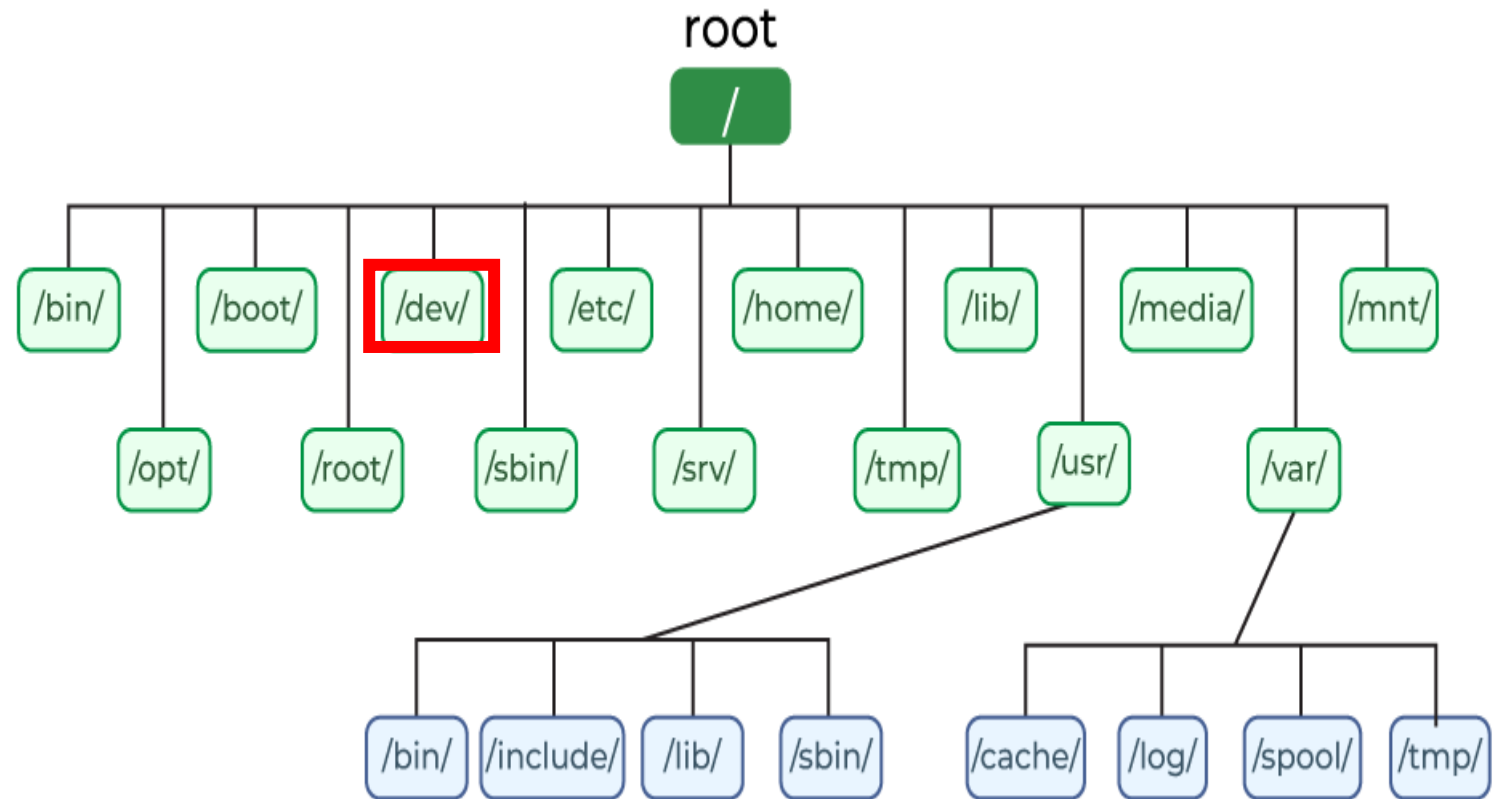
Directory structure

- /boot: Boot loader files, e.g., kernels, initrd.



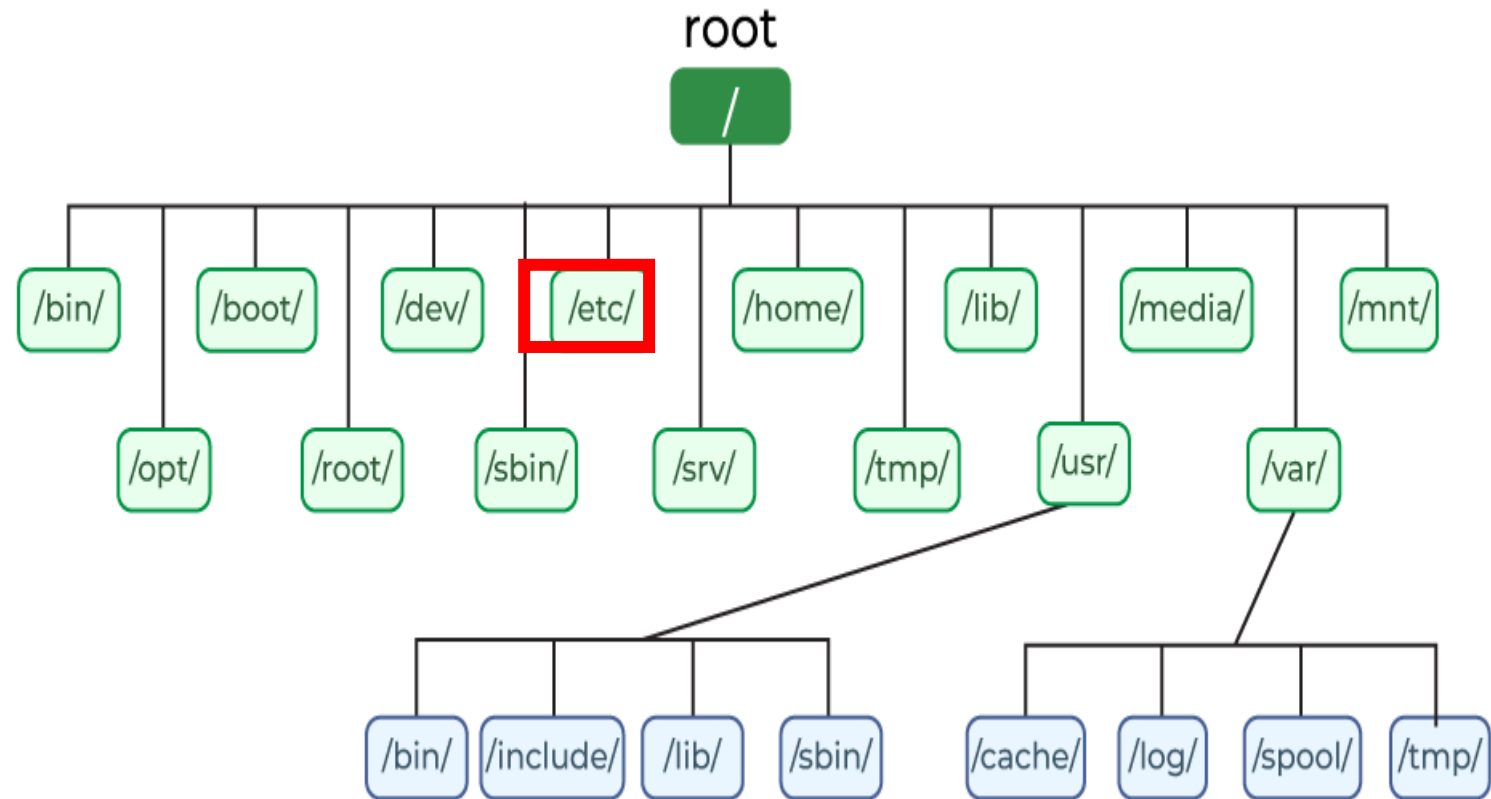
Directory structure

- /dev: system device files
- Represents hardware devices as files (e.g., /dev/sda for disks, /dev/null, /dev/tty).
- Used to interact with hardware through file operations



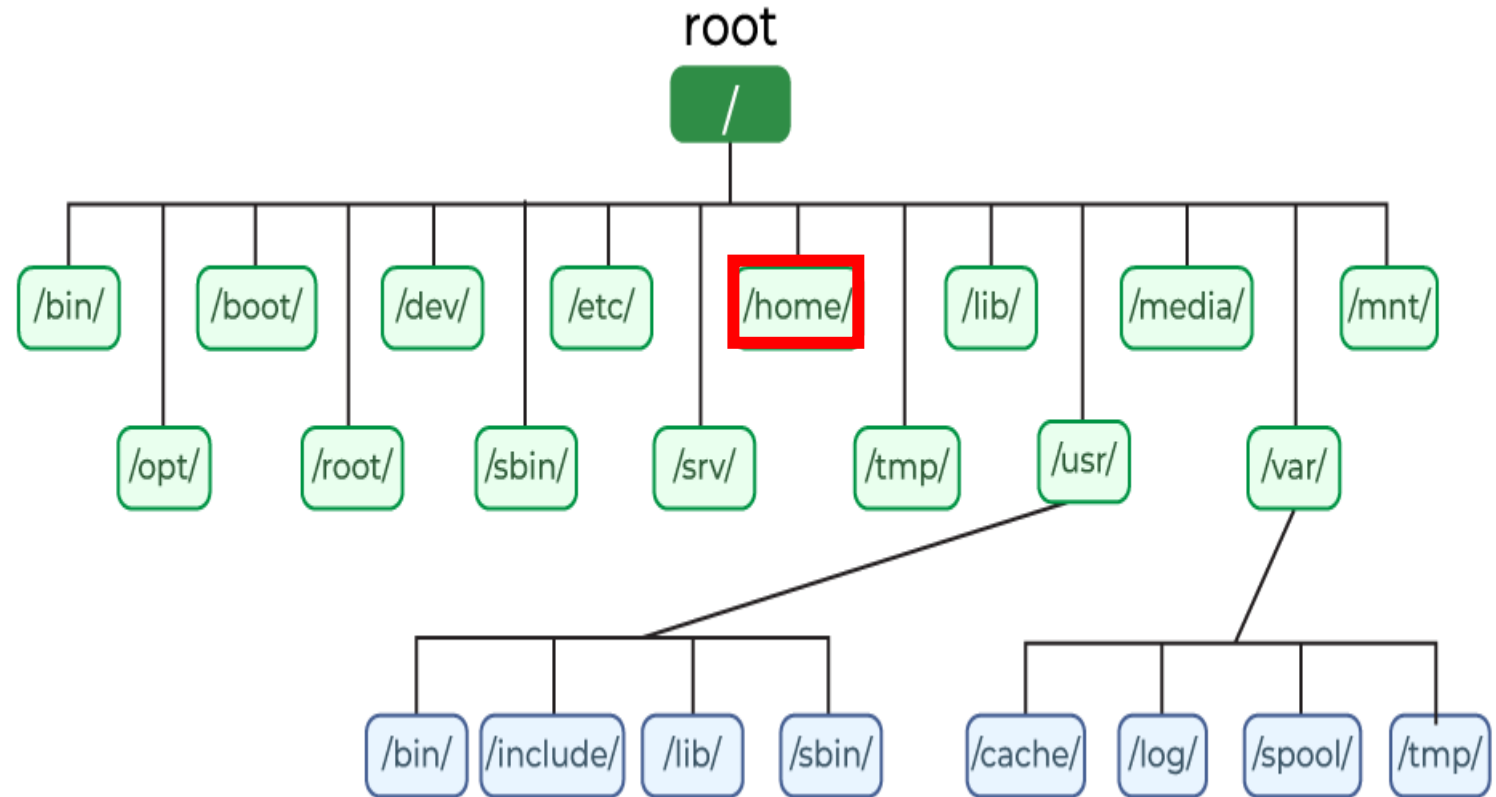
Directory structure

- /etc: Stores system-wide configuration files (e.g., /etc/passwd, /etc/hostname, /etc/hosts).
 - Editable by admins to configure the system and services.
-
- /etc/bashrc System-wide defaults and aliases for the Bash shell.
 - /etc/crontab Shell script that schedules commands to run at specific intervals.
 - /etc/exports Defines filesystems available to the network (NFS).
 - /etc/fstab Lists disk drives and their mount points.
 - /etc/passwd User account information (public part).



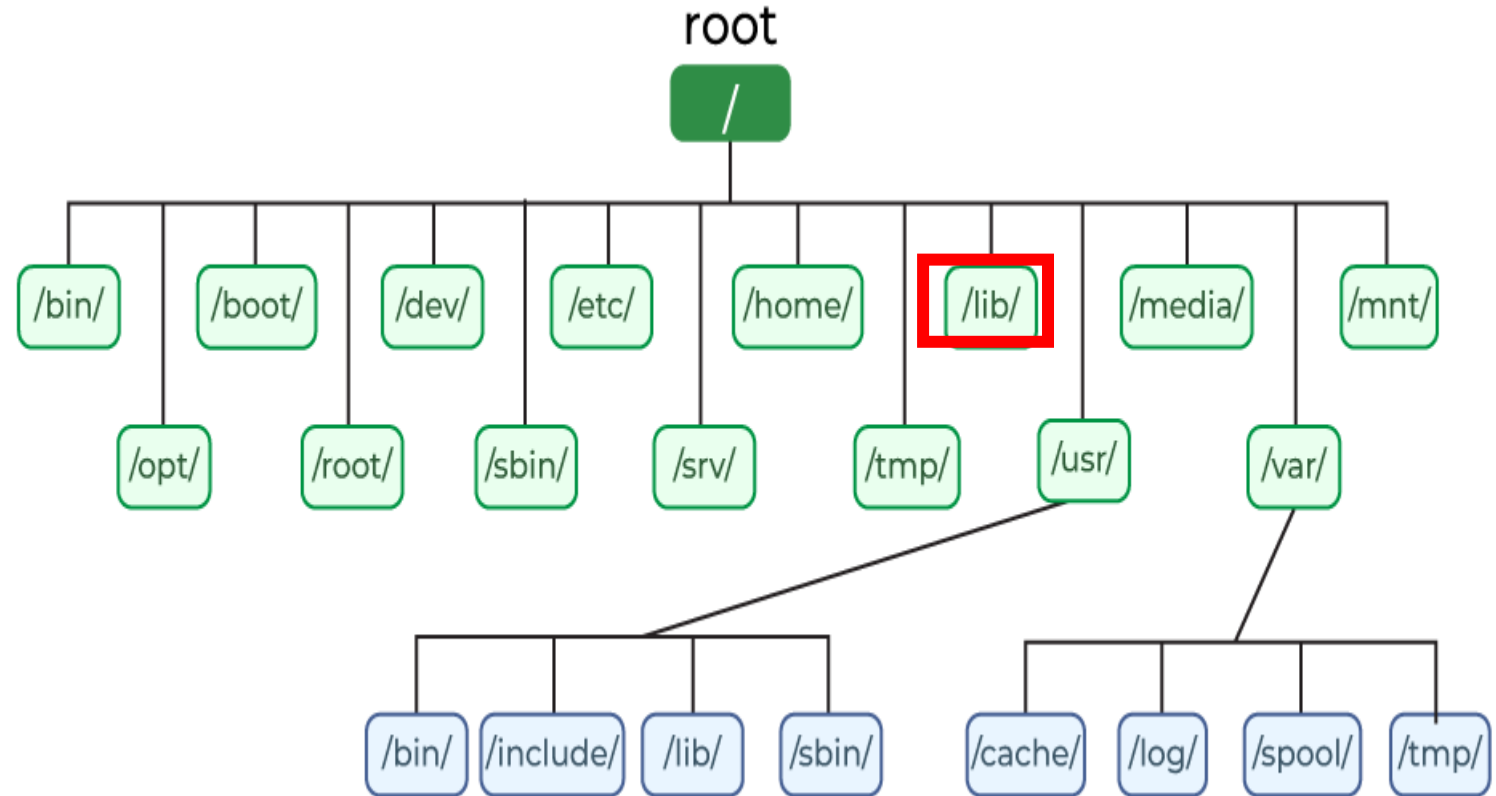
Directory structure

- /home: Contains the home directories for the users.
- Stores user data, configuration files, downloads, etc.



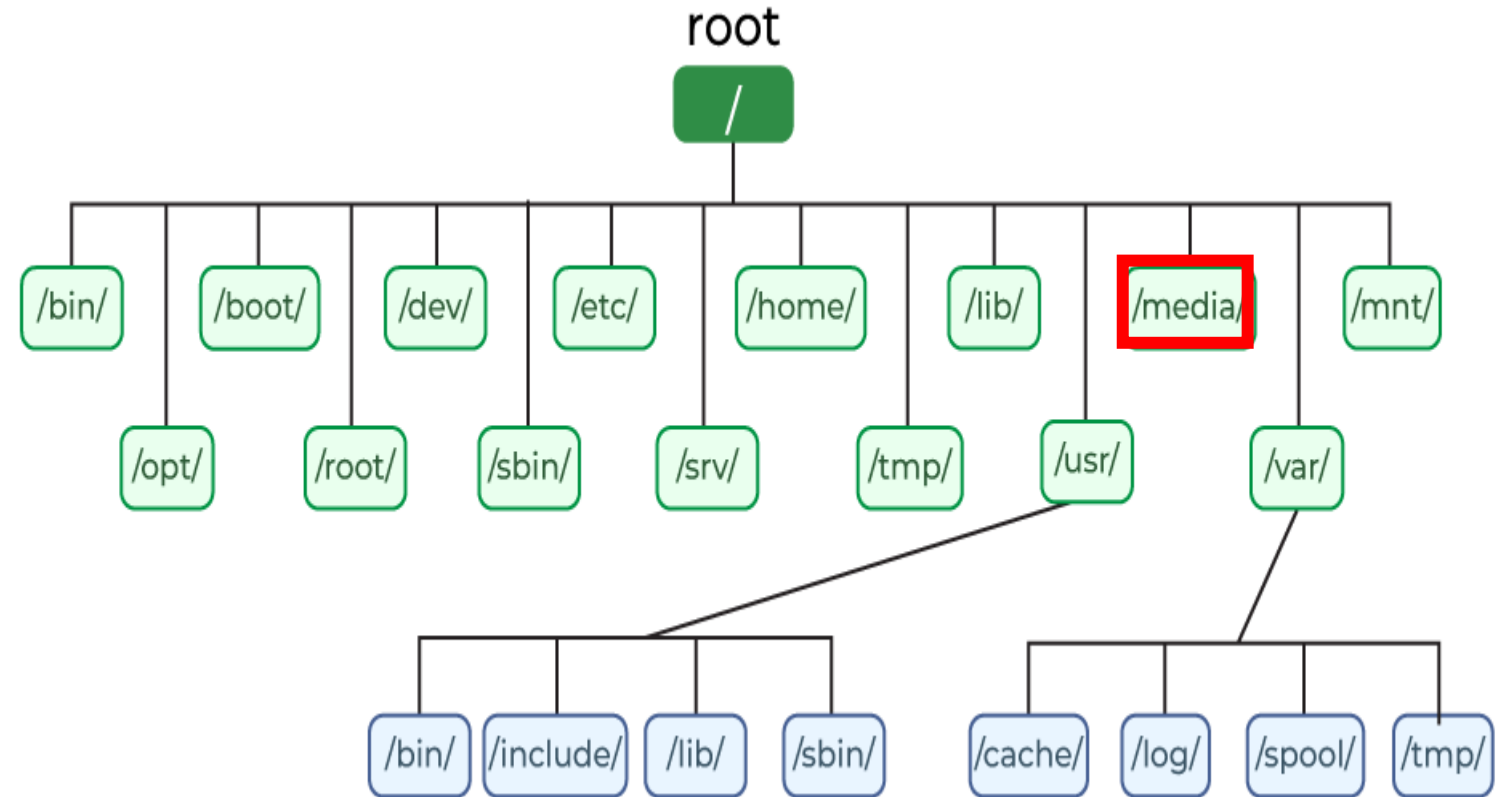
Directory structure

- /lib: Contains system libraries, and some critical files such as kernel modules or device drivers.
- Contains libraries needed by binaries in /bin and /sbin.



Directory structure

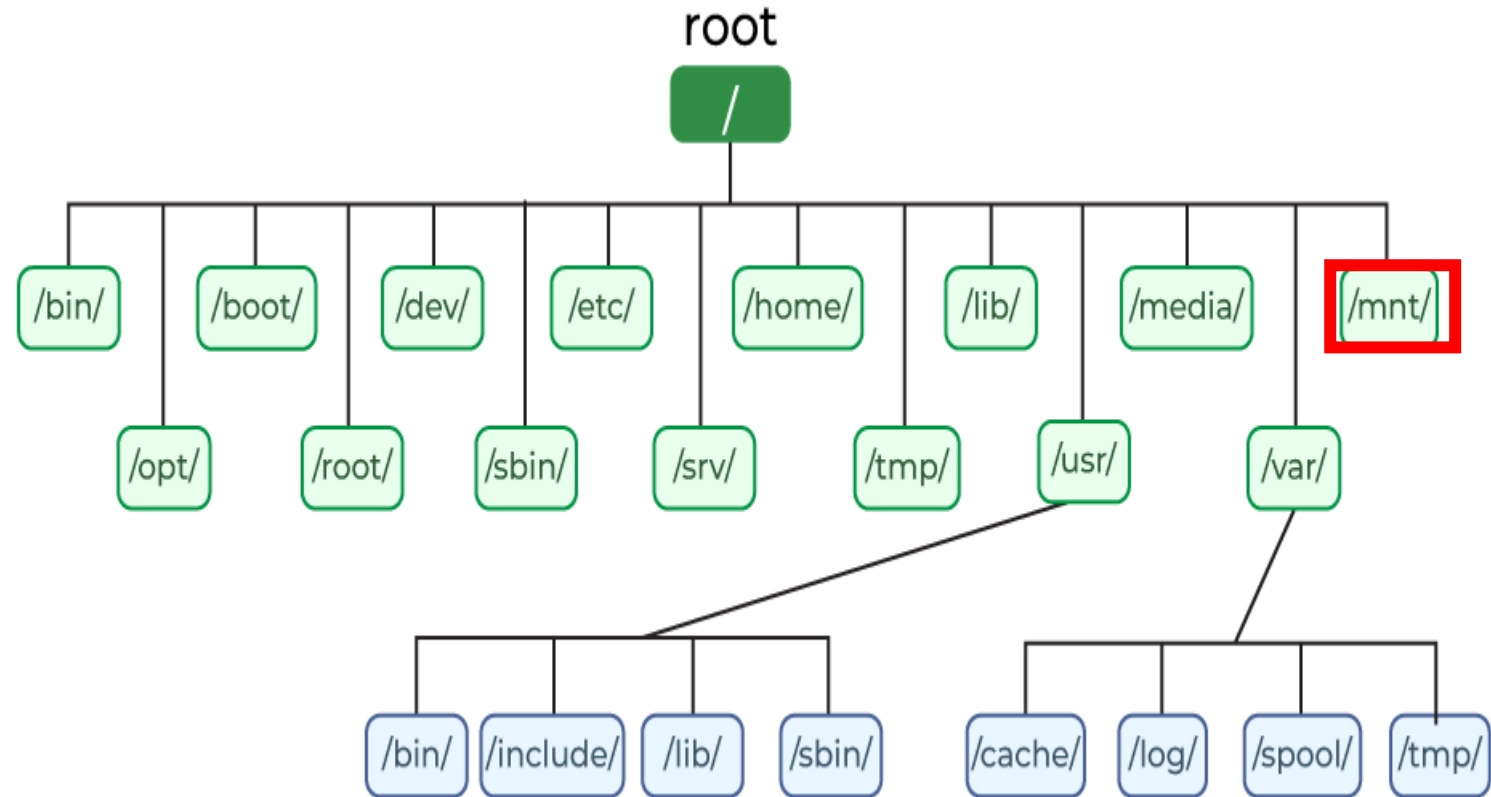
- /media: For automounted devices (e.g., USB sticks, CDs).



Directory structure

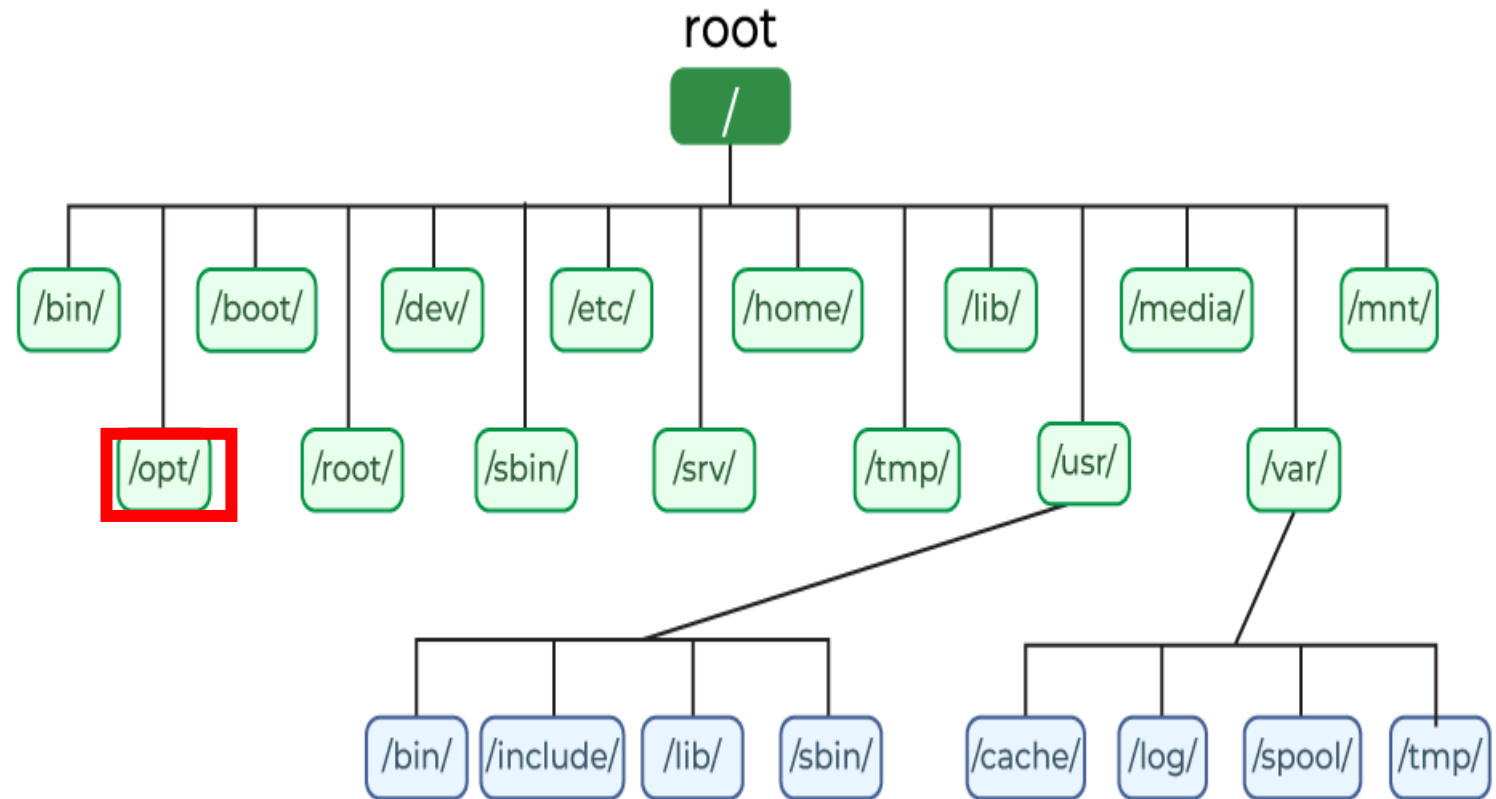
- /mnt: “mount” Contains filesystem mount points.
- For temporarily mounting filesystems manually

These are used, for example, if the system uses multiple hard disks or hard disk partitions. It is also often used for remote (network) filesystems, CD-ROM/DVD drives, and so on.



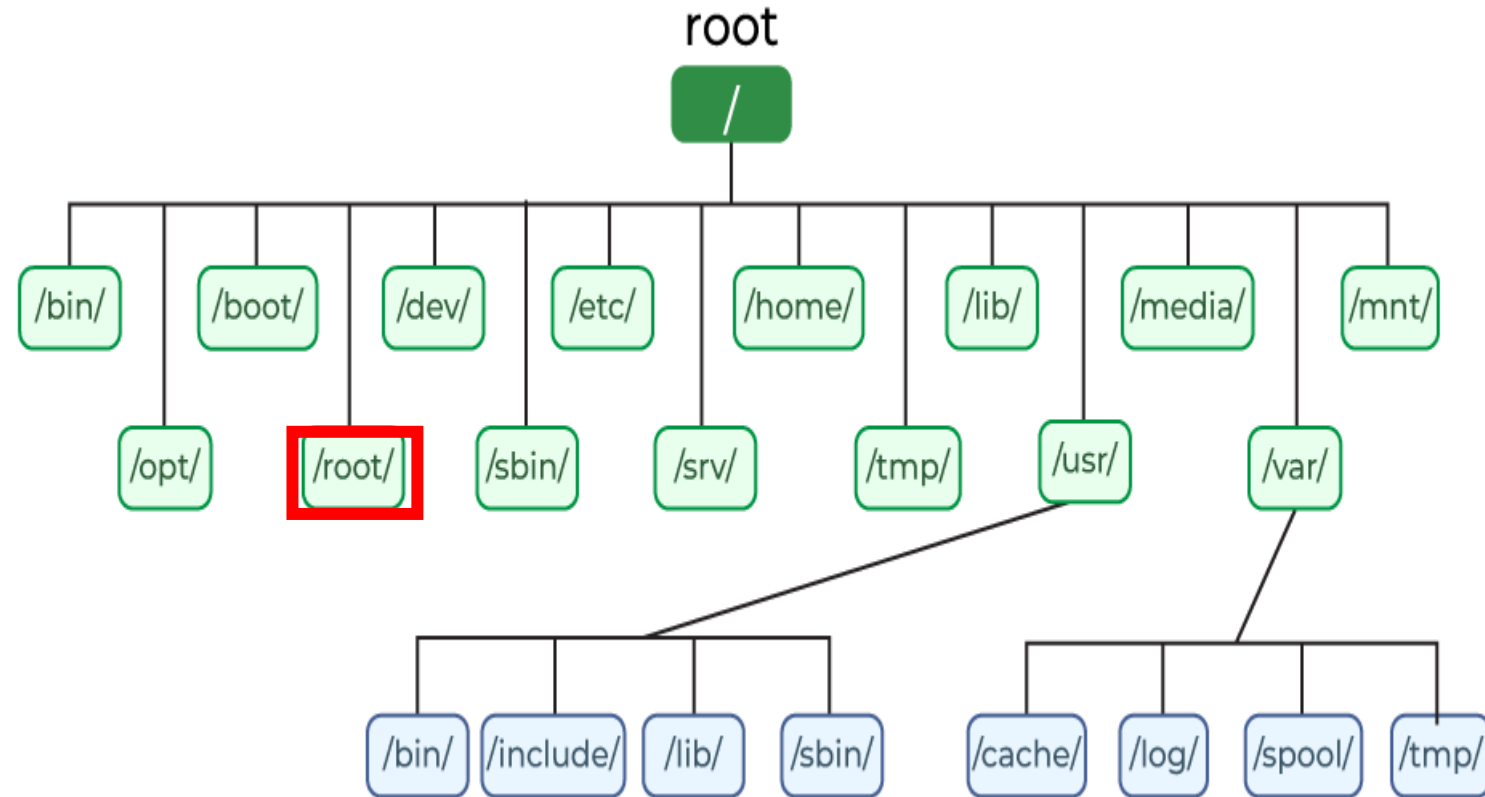
Directory structure

- /opt: optional application software packages that are not part of the standard Linux distribution.
- For large third-party apps that aren't part of the system.



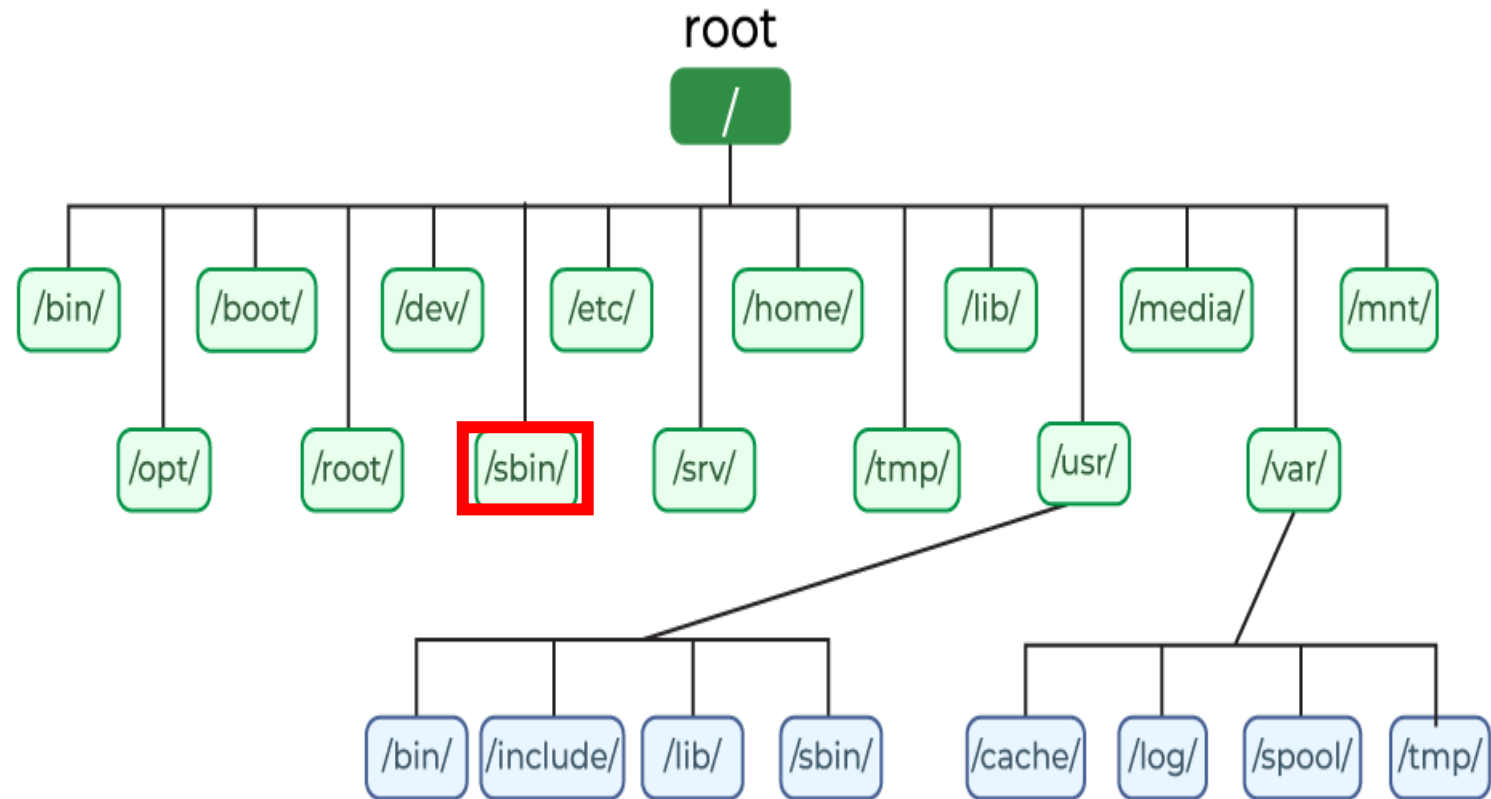
Directory structure

- /root: Home directory for the root user.
- Always available even if /home/ fails



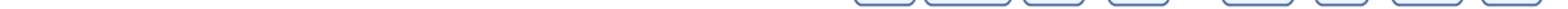
Directory structure

- /sbin: Superuser system utilities (for performing system administration tasks).
- Contains binaries for system administration (e.g., reboot, shutdown, fsck).
- Contains essential commands for system management
- Typically used by root (administrator)
- Also required during boot or repair mode



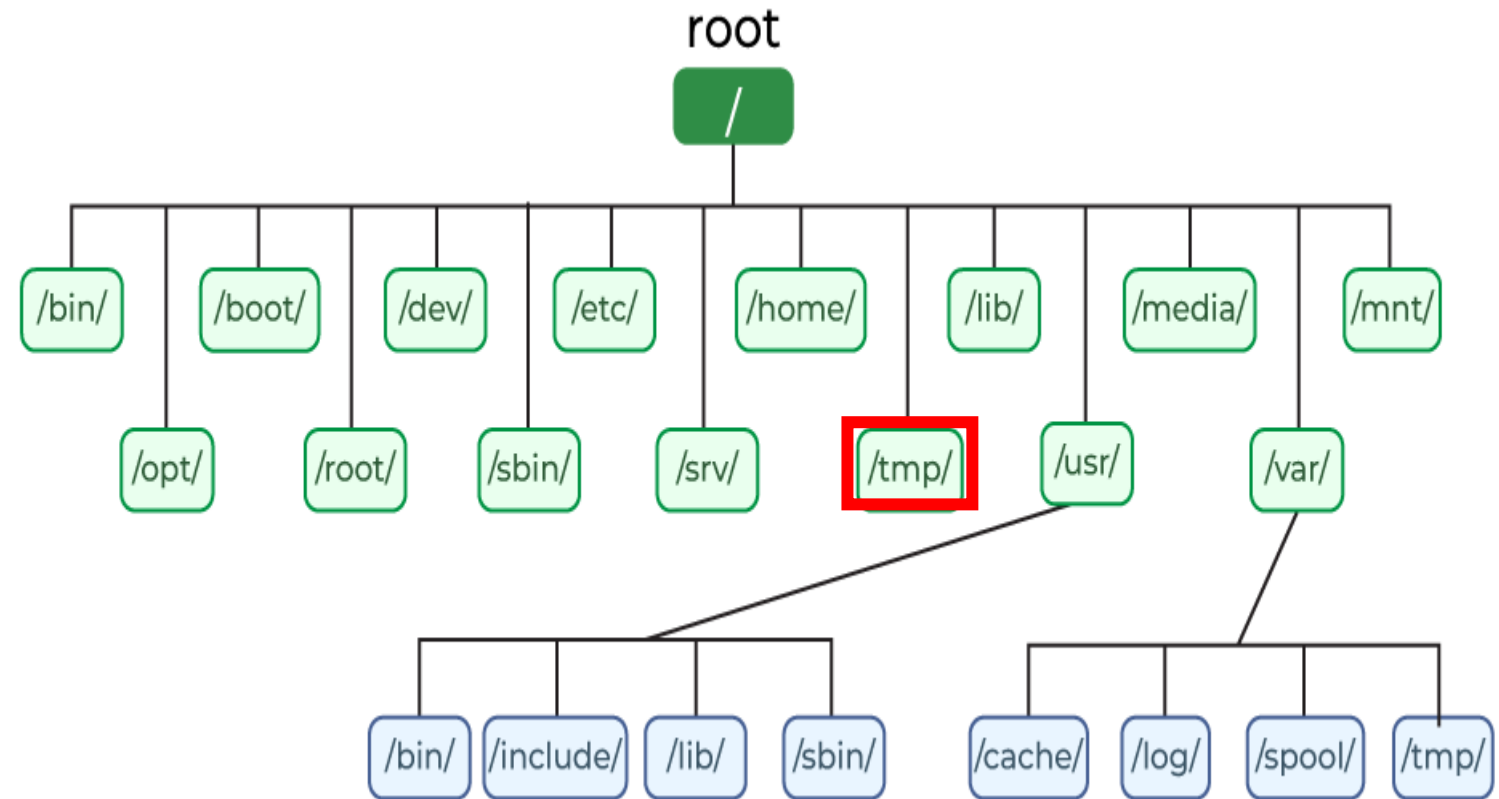
fsck : check filesystem
reboot : restart system
mount : mount filesystems
Iptables: firewall rules

- [/bin/](#) [/boot/](#) [/dev/](#) [/etc/](#) [/home/](#) [/lib/](#) [/media/](#) [/mnt/](#)

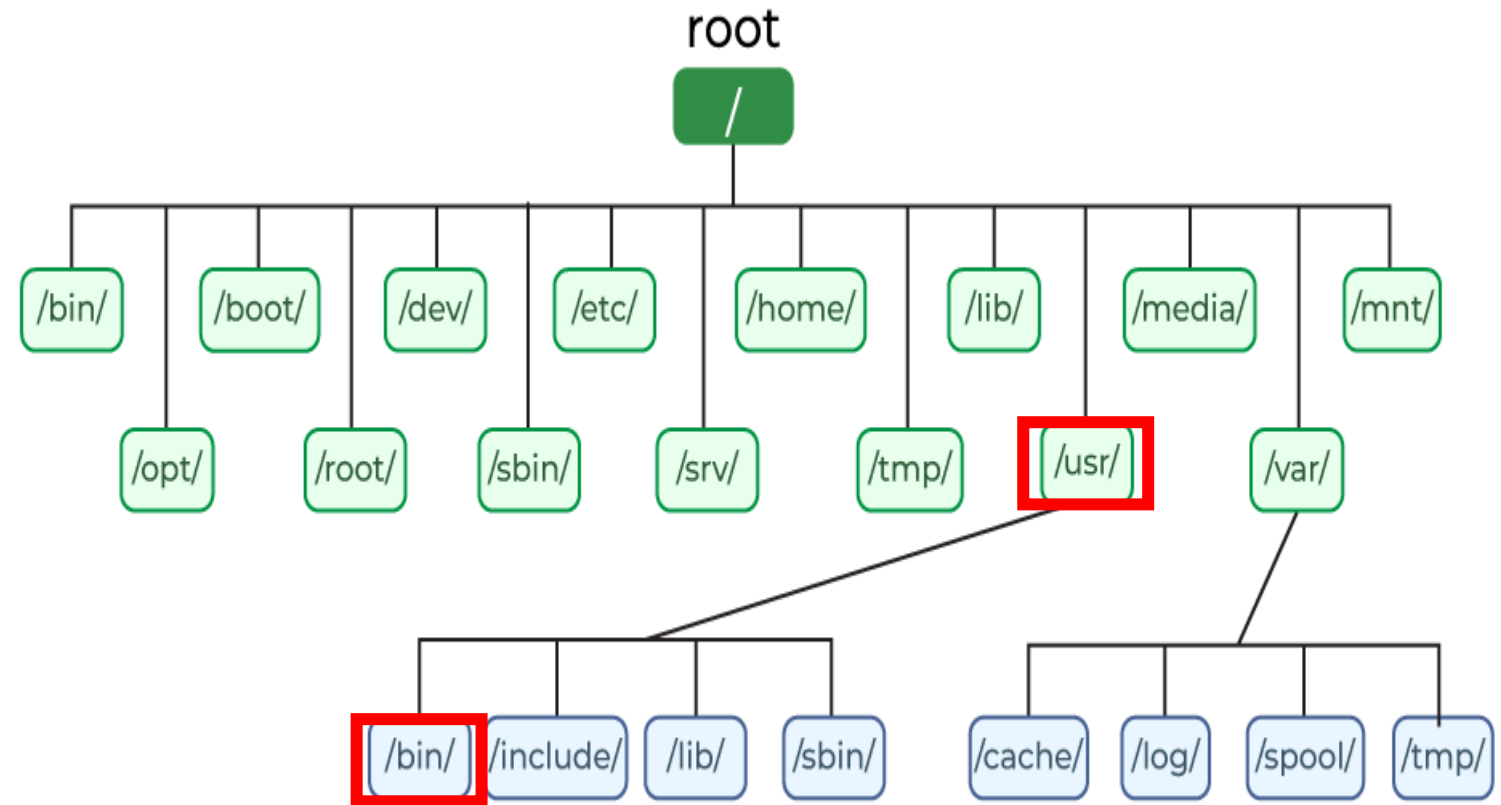


Directory structure

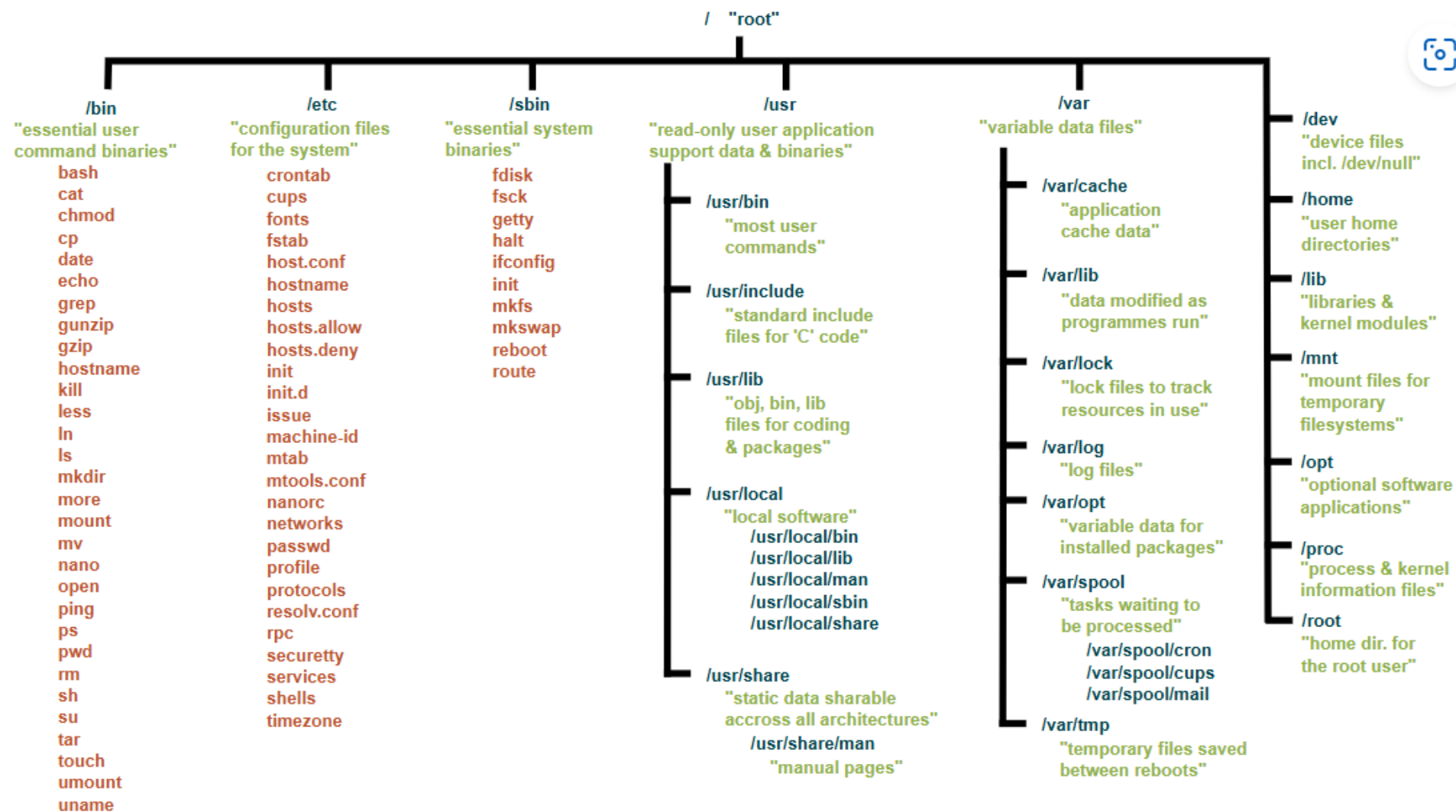
- /tmp: temporary files.



Directory structure



Directory structure



Paths

What is a path?

- A path is a unique location to a file or folder in the filesystem.
- Paths use / and alphanumeric characters to describe locations

Paths

What is a path?

- A path is a unique location to a file or folder in the filesystem.
- Paths use / and alphanumeric characters to describe locations

Two types of paths:

- **Absolute Path:** starts from the **root directory /** and specifies the full path to a file or folder, no matter what your current directory is
- **Relative Path:** Path relative to your current directory

Basic commands

Commands to use:

- `pwd` → Show where you are (Print Working Directory)
- `ls` → List files and directories
- `cd` → Change directory

Basic commands

Quick Tasks:

- Move to the root directory:

```
$ cd /
```

- List what's inside /home/:

```
$ cd /home
```

```
ls
```

- Find what's inside /etc/:

```
cd /etc
```

```
ls
```

- Come back to home directory:

```
cd ~
```

Basic commands

Exercises :

As you, I am an astronomer. Do you know what is a black hole? They're huge concentrations of matter packed into very tiny spaces. A black hole is so dense that is strong enough that nothing, not even light, can escape.

Do you know that there is a black hole in Linux? You will explore the directories and find it.

- Start looking in the folder that provides access to peripheral devices (like disks, keyboards, and other "hardware").
- Look for a device whose name suggests nothingness.
- Check what happens when you redirect output to it. (echo "Hello" >.....)
- What is the purpose of this black hole?

Basic commands

Exercices :

As you, I am an astronomer. Do you know what is a black hole? They're huge concentrations of matter packed into very tiny spaces. A black hole is so dense that is strong enough that nothing – not even light – can escape.

Do you know that there is a black hole in Linux? You will explore the directories and find it.

- Start looking in the folder that provides access to peripheral devices (like disks, keyboards, and other “hardware”).
- Look for a device whose name suggests nothingness.
- What is the purpose of this black hole?
- Answer: /dev/null, discarding output, Testing Command (dummy input or output file for commands that require a file but where you don't want to provide actual data).

Basic commands

Exercises :

- Continue to explore the filesystem tree using `cd`, `ls`, `pwd`. Look in `/bin`, `/usr/bin`, `/sbin`, `/tmp` and `/boot`. What do you see? What are the permissions on `/etc/passwd`
- Run `echo $SHELL`, `echo $0`, and `ps -p $$`.
- Explore `/dev/`: Identify your terminal device. (look for `/dev/tty`)
- Use `whoami`
- Can you find your USB or disk partitions?
- Inspect `/boot/` : what kernel version is installed? Cross-check your user looking into `proc/`
- Open `/etc/fstab` : what filesystems are auto-mounted?
- Why `cd ../` does not give an error message
- Give 2 different ways to go to your home directory

Basic commands

More Tasks (sol):

- `/bin` Essential system programs like `ls`, `cp`, `mkdir`, `bash`
- `/usr/bin` Most user applications like `python3`, `gcc`, `vim`, `nano`
- `/sbin` System administration programs like `ifconfig`, `reboot`, `fdisk`
- `/tmp` Temporary files (might be empty if recently rebooted)
- `/boot` Boot files, including `vmlinuz` (kernel image) and `grub` configuration
- `echo $SHELL` → e.g., `/bin/bash`; `echo $0` → current shell.
- `ps -p $$` → shows the shell process info.
- `tty` or `who am i` → shows current terminal device (e.g., `/dev/pts/0`).
- Use `lsblk` or `df -h` to list block devices and mounted partitions.
- Character devices `/dev/tty0`, `/dev/tty1`, `/dev/console`, `/dev/random`
- Block devices `/dev/sda`, `/dev/sdb`, `/dev/loop0` (disks and partitions)
- `ls /boot/` → look for files like `vmlinuz-5.15.0-XX` for kernel version.
- `cat /etc/fstab` → shows device mount rules, e.g., `/dev/sda1 / ext4 defaults 0 1`.
- `systemctl list-units --type=target` → shows active systemd targets.
- Because `../` exists and it points back to `/` itself.
- `cd` , `cd ~`, `cd $home`

`df -h`

Shows mounted filesystems

- Displays used / free space
- Focuses on what the OS is actually using

`lsblk`

Shows physical and virtual devices

Displays disk structure

Does NOT show usage

`uname -r`, see kernel used

CLI Basics: Commands & Navigation

Session 4

CLI Basics: Commands & Navigation

Session 4

Objectives:

- Understand the CLI structure.
- Navigate the filesystem using basic commands.
- List files and directories.
- Create, move, rename, and delete files and folders.
- Develop basic command-line reflexes.

Introduction to CLI?

What is a CLI?

- CLI = Command Line Interface.
- Used to communicate with a computer program, you can input text into a by typing command lines.
- Offers more control, speed, and flexibility than graphical interfaces.

Introduction to CLI?

Advantages?

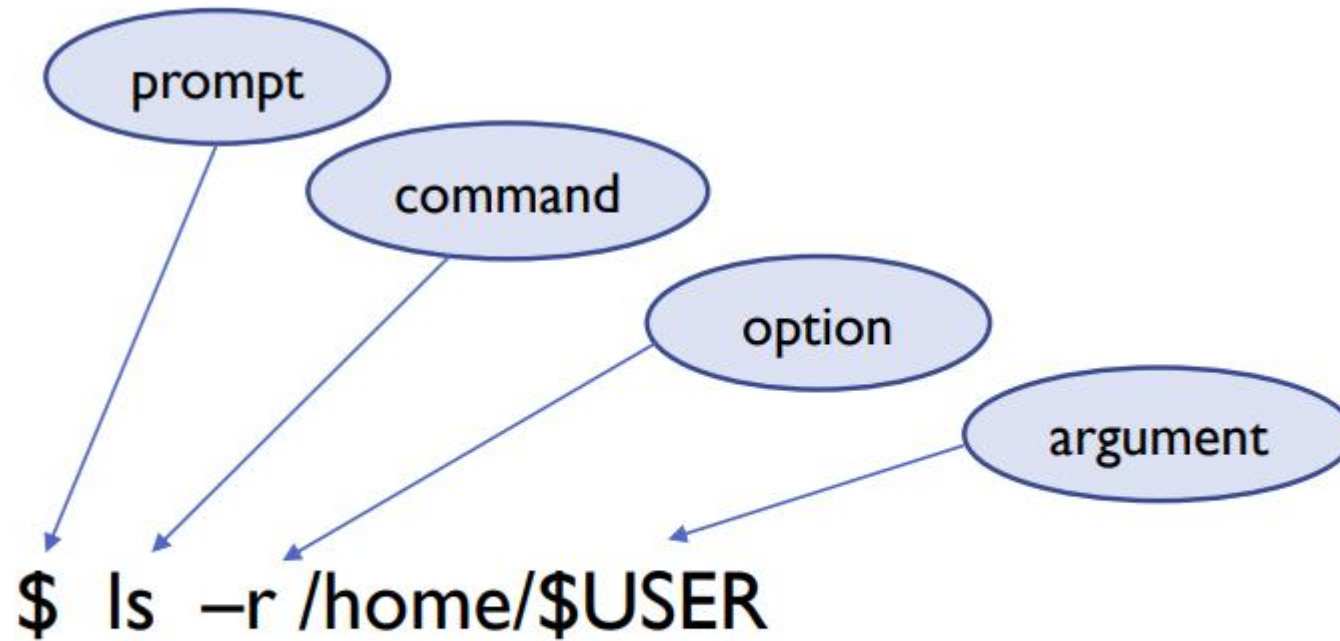
- **Remote access:** CLI applications typically require fewer network resources than GUI ones, appropriate for devices with modest hardware resources or remote server setups.
- **Efficiency:** The CLI lets you perform actions on several files with a single text command, saving you time from having to look for and click on each file individually. You can write scripts that carry out a variety of CLI commands to automate tedious or repetitive tasks.
- **Repetitive operations:** The CLI is great at automating repetitive operations, and you may use a batch file to automate jobs at any time.
- **Power user:** A CLI is suitable for power users since it allows them to execute commands that are not available in the GUI.

Introduction to CLI?

Disadvantages?

- **Error management:** Errors in command writing might cause files to be deleted or moved to the incorrect location. You run the risk of encountering issues if you unintentionally remove any files, especially if they hold crucial information.
- **Learning curve:** Acquiring command knowledge takes time. The majority of commands are lost if they are not used on a regular basis.
- **Replicate orders:** The majority of commands on the command line interface cannot be undone or reversed. As a result, using commands requires extreme caution. The majority of the time, a command's execution is irreversible and continues.
- **Difficult to remember:** Recalling every command that is used in the CLI might be challenging. Over 100 CLI commands are available in UNIX

Linux commands



Linux command basics

- Linux commands are case-sensitive

ls is not the same as LS

- Linux commands may have options that come after the command that start with a “-” and followed by a letter or “- -” and a word:

```
$ ls -r
```

```
$ ls --reverse
```

- Linux commands may allow for arguments:

```
$ ls /tmp
```

- You can run more than one command on the same line by separating the commands with a semicolon (;)

```
$ ls;date
```

- man give information about a command

UNIX commands that give information

Exercise 1: Type

- **hostname**
- **date** , can you print only the date? the time?
- **who**
- **whoami**
- **env**, Scroll up and down the screen and make a note of the values of the following Environment variables: HOSTNAME = SHELL= TERM= USER= PATH=
- **uptime**
- **uname** what does the option r, n ?
- **man uname.**

UNIX commands that give information

Exercise 1: Type

- **hostname** Displays the name of the machine. Each machine on the network is given a unique name by the system administrator.
- **Date** Used to display the current system date and time. `date +%D` Displays date only
`date +%T` Displays time
- **who** Login details of all users such as their IP, Terminal No, User name.
- **whoiam** Used to display the login details of the user.
- **env** Displays a list of environment variables and their values. We will meet these again later
- **uptime** Displays the time duration that the machine has been turned on.
- **uname** Displays information about the computer system.

Shows version number of the OS (kernel), Displays domain name of the server

- **man uname** The man command gives information on the command that follows it.

UNIX commands relating to directories

Exercise 2: Type

- **pwd** Print (on the screen) the pathname of the working directory (i.e. the directory you are presently in).
- **ls** List contents of directories.
- **cd** Change working directory.
- **mkdir** Make a directory.
- **rmdir** Remove empty directory.
- **mv** Move a directory (also used to rename a directory or file).

Example of useful options:

- **ls -l** : Long format (permissions, size, date).
- **ls -a** : Show hidden files (those starting with .).
- **ls -t** : Sort by modification time, newest first.
- **ls -r** : Reverse the order of the sort.
- **ls -lrt** : Combined options to show oldest first in detailed view.

UNI

Exercise 2: Type

2	ls	List the contents of your current directory (should be empty at this stage).
3	mkdir testdir	Make a subdirectory named 'testdir'.
4	cd testdir	Change directory to the new directory.
5	pwd	Write down the pathname of your current directory.
6	ls	List the contents of your current working directory (should be empty at this stage).
7	cd	The change directory command, cd , on its own will move you back to your 'home directory'. Another way to return to your home directory is to use the command cd ~
8	pwd	Write down the pathname of your current directory (it should be your home directory)
9	ls	List the contents of your current directory (this should now show the name of the subdirectory named testdir)
10	cd / pwd ls ls -p	Change to the top most level directory (i.e. the root directory) Confirm where you are using the pwd command. List the contents of the current directory List the contents of the current directory and put a forward slash at the end of any entry that is a directory.
12	cd pwd ls	Change to your home directory Confirm where you are using the pwd command. List the contents of the current directory
13	cd testdir	Change to the directory named testdir
14	cd .. pwd ls	Change to the directory one level up (as specified by the use of two dots). Confirm where you are using the pwd command. List the contents of the current directory
15	pwd ls mkdir testdir/play ls cd testdir/play pwd ls cd .. pwd ls mv play play2 ls	Check you are in your home directory. List the contents of the current directory. Create a directory named play underneath the one named testdir. List the contents of the current directory. Change directory into the one just created. Confirm where you are. List the contents of the current directory. Change up one level (ie. back up to the testdir directory) Confirm where you are. List the contents of the current directory. Rename the directory named play to play2 . List the contents of the current directory to check that the renaming has been done.
16	Note:	The command cd testdir/play uses the name of the directory relative to the current directory. To be more precise you could have specified the command as: cd ./testdir/play where the single dot means the current directory.
17	cd pwd ls	Move back up to your home directory. Confirm where you are. List the contents of the current working directory.
18	rmdir testdir	Remove directory. You will find that this command will not work since the directory contains subdirectories. Overcome this by first removing the lower level directories. Later you will be shown a potentially dangerous command that will remove any directory and anything underneath it.
19	pwd ls	Check where you are. List the contents of the current directory.
20	cd mkdir testdir mkdir testdir/play	Recreate the directories again.

irectories

UNIX commands relating to files

Commands

- **cp** Copy file
- **mv** Move a file (also used to rename a file).
- **rm** Remove a file. (Can also be used with the appropriate option to removed a directory and all its contents).
- **touch** create or update a file.

Example of useful options:

- **rm -i** : Prompt before every removal
- **rm -r** : Recursive delete (for folders)
- **rm -f** : Force deletion (no confirmation)

Important shortcuts

Shortcuts

- `cd ..` : Go up one directory
- `cd -` : Go to the previous directory
- `cd ~` : Go to home
- `!!` : repeat last command

Pattern Matching in the Shell

Pattern	Meaning
*	Any number of characters (even none)
?	Exactly one character
[wtz]	One character: w, t, or z
[a-p]	One character in the range a to p

Examples:

```
ls *.txt      # All .txt files
ls ?.sh       # Any 2-character .sh file (e.g., a.sh)
ls [ab]*      # Files starting with a or b
ls file[1-3].txt # file1.txt, file2.txt, file3.txt
```

Advanced CLI: I/O & Text Processing

Session 5

Advanced CLI: I/O & Text Processing

Session 5

Objectives:

- Understand standard input, output, and error (stdin, stdout, stderr)
- Redirect input and output using `>`, `>>`, and `<`
- Chain commands with pipes `|`
- View and filter text using `cat`, `less`, `more`, `head`, `tail`
- Use `grep` for searching
- Use `wc`, `sort`, `cut`, and `uniq` for text analysis
- Practice combining commands through hands-on labs

What is stdin, stdout, stderr

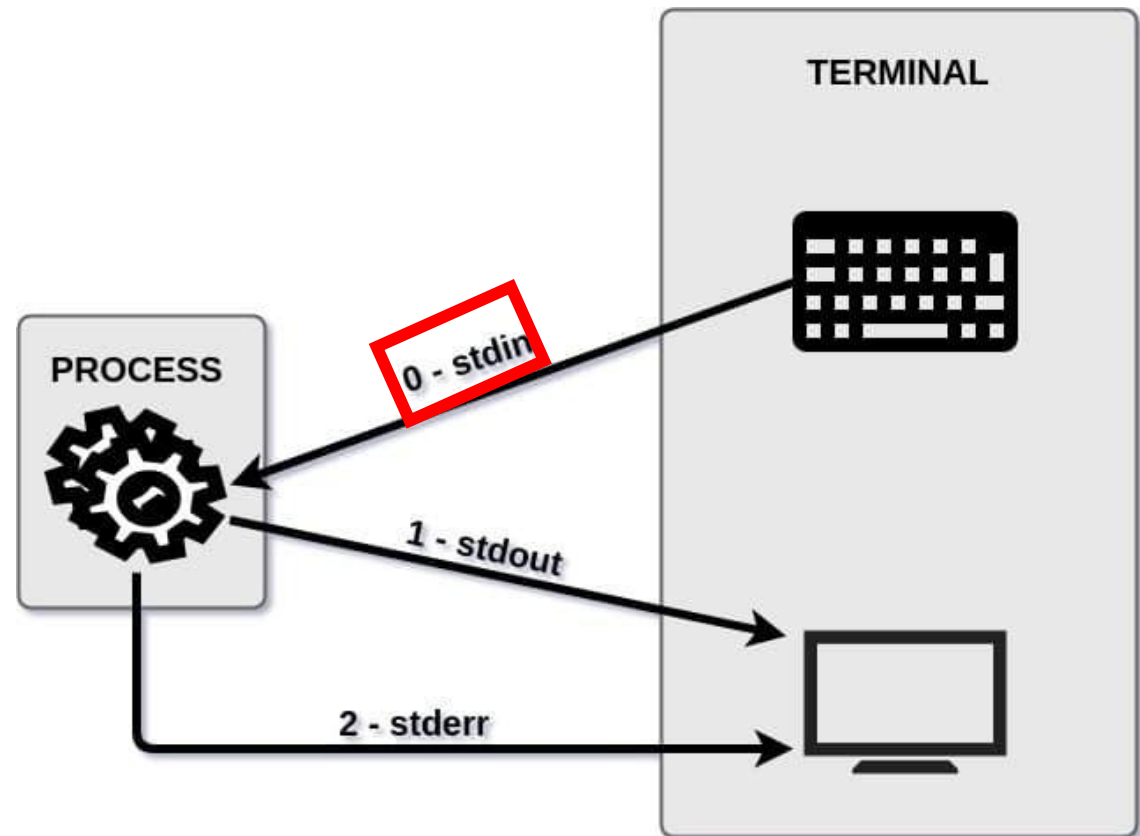
Linux has an interesting concept where basically all input and output (which are text) are actually streams of data/text. Like plumbing pipes where you can connect and disconnect sections to redirect water to different places, so you can you connect and disconnect streams of data.

Standard streams are input and output (I/O) communication channels between a program and it's environment.

What is stdin, stdout, stderr

Three standard streams:

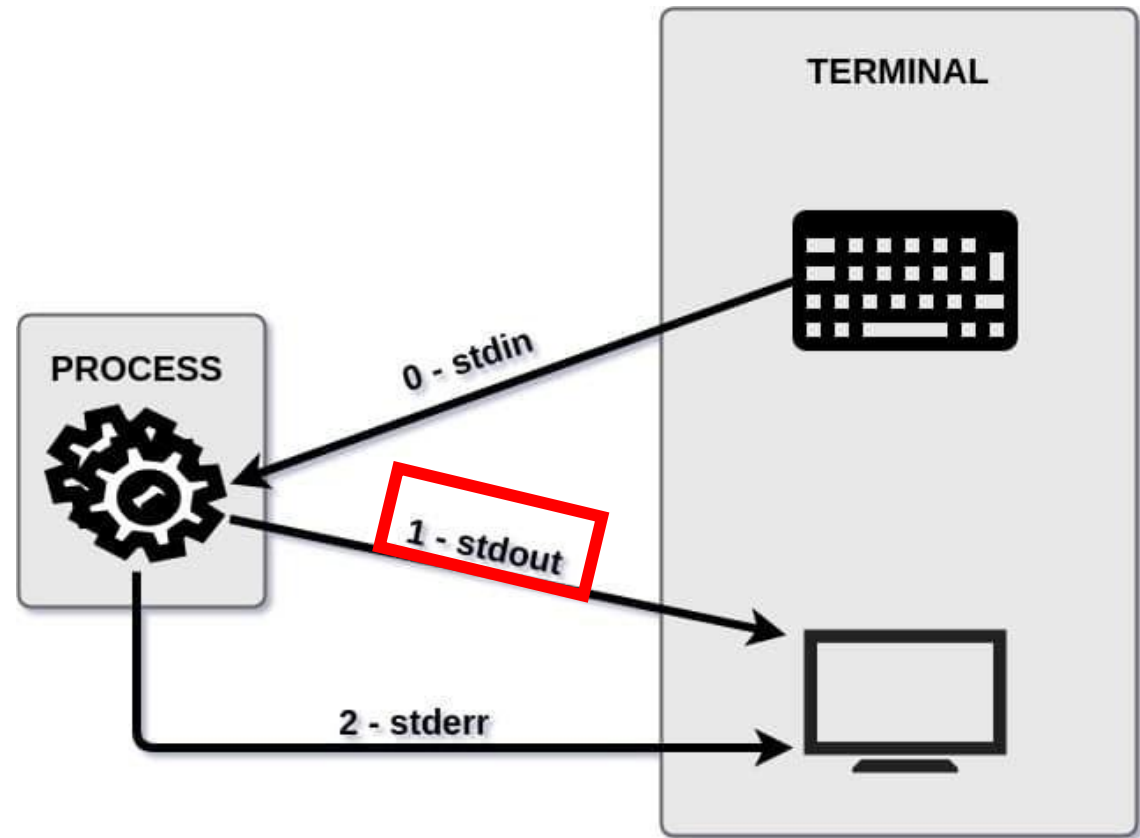
- **stdin** : standard input (keyboard) , the command line uses the stdin to receive inputs. If you type the command **ls** to list a directory content, you use the stdin.



What is stdin, stdout, stderr

Three standard streams:

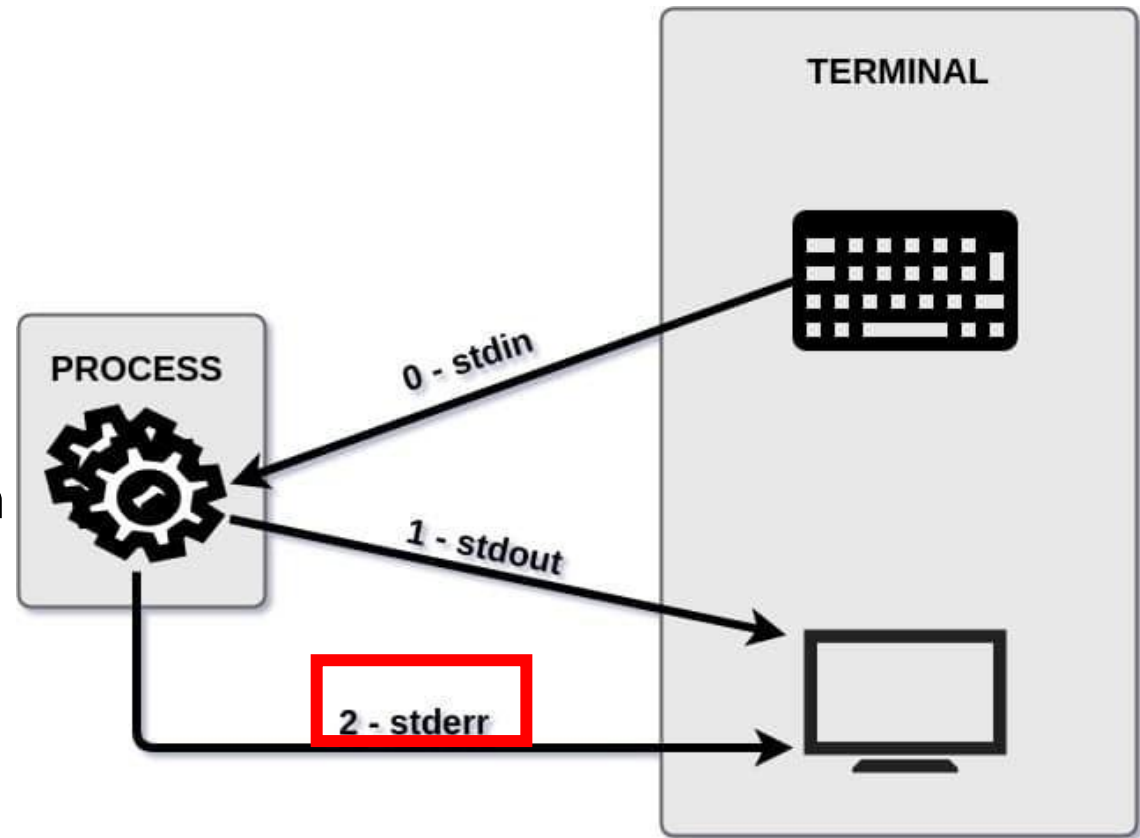
- **stdin** : standard input
- **stdout** : standard output (terminal screen) , this is the counterpart of the stdin. For « ls » the listing of the directories will be displayed by the stdout channel in your screen.



What is stdin, stdout, stderr

Three standard streams:

- **stdin** : standard input
- **stdout** : standard output
- **stderr** : standard error output (terminal screen) , this channel is also an output, but specific to any error messages or diagnostics the command may return. For example, if I run my ls command in a path that doesn't exist, the “No such file or directory” output will be returned through the stderr channel

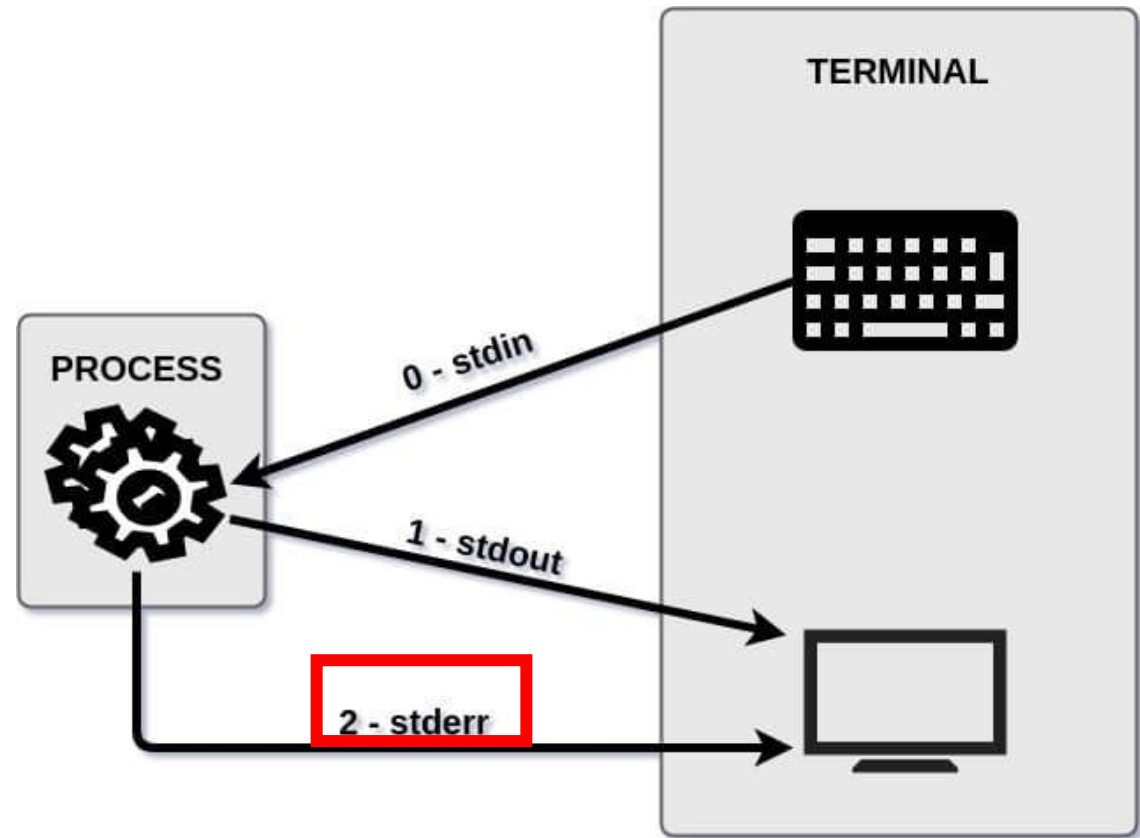


Output redirections

Three standard streams:

- **stdin** : standard input
- **stdout** : standard output
- **stderr** : standard error output (terminal screen) , this channel is also an output, but specific to any error messages. If I run « ls » in a path that doesn't exist, the “No such file or directory” output will be returned through the stderr channel

Despite being two different channels, stdout and stderr are displayed in the same way on the terminal.



Stream redirection

Stream output can be redirected to another stream, file, or command:

- `>` is frequently used in shell scripting to redirect a stream (typically `stdout`) into a file or another destination. If the target is a file, it will overwrite its contents.
- `>>` behaves similarly but appends the output to the file instead of replacing it.
- `<` is used in the opposite direction: it redirects input from a file to a command, substituting for keyboard input (`stdin`).

Stream redirection

Examples:

`ls > list.txt` # overwrite the file

`ls >> list.txt` # append to the file

`wc -l < list.txt` # count lines by reading from file as stdin

`command > output.txt 2>&1` # Combine stdout and stderr into one file

- Write "Welcome" into greeting.txt (overwrites if it exists) using echo
- Append another line ("to the course") to the same file
- Redirect standard error separately into error.log. Use ls and non existing file
- Read a greeting as input for wc command
- Try to use ls and combine stdout and stderr into 1 file

Output redirection

Examples:

`ls > list.txt` # overwrite the file

`ls >> list.txt` # append to the file

`wc -l < list.txt` # count lines by reading from file as stdin

- Write "Welcome" into greeting.txt (overwrites if it exists) using echo
`echo "Welcome" > greeting.txt`
- Append another line ("to the course") to the same file
`echo "to the course" >> greeting.txt`
- Redirect standard error separately into error.log. Use ls and non existing file
`ls non_existing_file 2> error.log`
- Read a greeting as input for wc command
`wc -w < greeting.txt`
- Try to use ls and combine stdout and stderr into 1 file
`ls > doesnotexist.log 2>&1`

Stream redirection

Examples:

- Create a file name.txt with three name:

Theo

Adele

Daniela

- Try sort names.txt
- Try sort < names.txt

Stream redirection

Examples:

- Create a file name.txt with three name: Adele, Daniela, Theo

```
cat > name.txt
```

```
Adele
```

```
Daniela
```

```
Theo
```

Or `echo -e "Adele\nDaniela\nTheo" > name.txt.`

Stream redirection

Examples:

- Create a file name.txt with three name: Adele, Daniela, Theo
- Try sort names.txt
- Try sort < names.txt

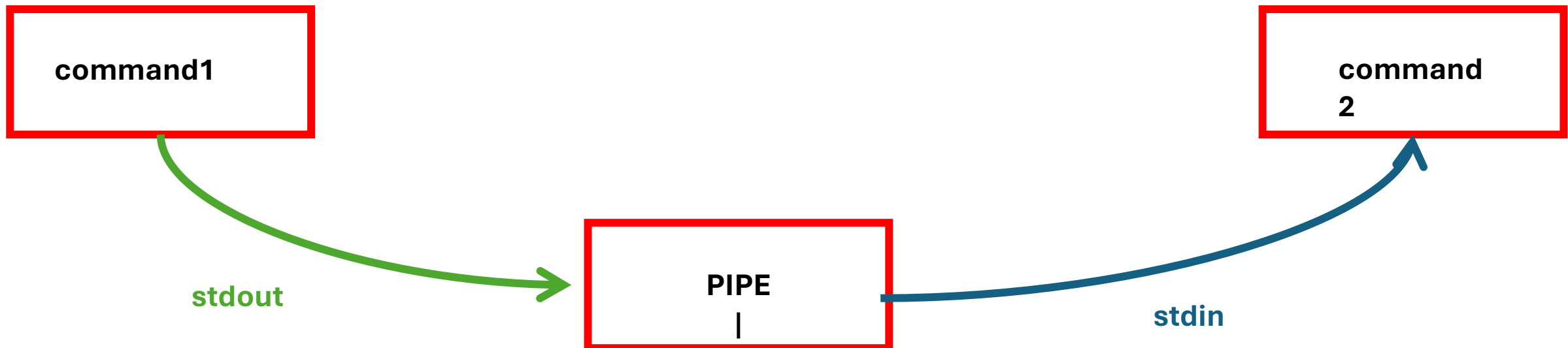
It is the same!!

Command	Reads from	Useful when...
sort names.txt	File directly	Simpler, preferred for file input
sort < names.txt	Standard input	For scripting, pipelines, stdin-based tools

Pipe |

Pipe |

Pipes allow you to connect the output of one command directly as input to another. This is a core concept in building powerful command-line workflows:



Pipe |

Examples:

- `ls /etc | less`
- `cat /etc/passwd | wc -l`
- `cat name.txt | sort`
- `ls /usr/bin | tail -n 5`

Pipe |

Examples:

- `ls /etc | less` # View long directory listings one page at a time
- `cat /etc/passwd | wc -l` # Count how many lines (users) in passwd file
- `cat name.txt | sort` # Sort the lines of file.txt
- `ls /usr/bin | tail -n 5` # Show the last 5 entries in /usr/bin

Text viewer: less, cat, more

- **cat** : display full content
- **less, more** : scrollable viewers

Examples:

- cat file.txt
- less file.txt
- more file.txt

Text viewer: head, tail, wc

- **head** : first lines
- **tail** : last lines
- **wc** : count lines, words, characters

Examples:

- `head -n 5 file.txt` (print the first 5 lines)
- `wc -l file.txt`, `wc -c file.txt`

Sort, cut, uniq

- **sort** : sort lines alphabetically or numerically
- **uniq** : remove duplicates (use with sort)
- **cut** : extract columns/fields:
 - -d Specifies the delimiter (e.g., : or ,)
 - -f Specifies the field number(s) to extract
 - -c Extracts characters by position

Examples:

- `cut -d":" -f1 /etc/passwd | sort | uniq`

Sort, cut, uniq

- **sort** : sort lines alphabetically or numerically
- **uniq** : remove duplicates (use with sort)
- **cut** : extract columns/fields:
 - -d Specifies the delimiter (e.g., : or ,)
 - -f Specifies the field number(s) to extract
 - -c Extracts characters by position

Examples:

- `cut -d":" -f1 /etc/passwd | sort | uniq`
 - Reads the file `/etc/passwd`
 - Uses `:` as the field delimiter
 - Extracts the **first field** from each line, which is the **username**
 - **| sort**
 - Sorts the usernames alphabetically
 - **| uniq**
 - Removes any duplicates (although `/etc/passwd` usually doesn't have username duplicates)

Search with grep

- **grep** : is for filtering/searching'

Examples:

- `grep "root" /etc/passwd` and `cat /etc/passwd | grep root`
- Create a logs.txt with

“Line 1: Reading configuartion... done.

Line 2: ERROR: Falied to parse 'config.ini' — missing '=' on line 23.

Line 3: error connecting to the databse. Retrying...

Line 4: ERROR: Timeout while waiting for responce.

Line 5: Disk check: 1 warning, 3 errors found.

Line 6: error: Invalid user input detected.

Line 7: ERROR 403: Forbiden acces to /admin/panel.

Line 8: error in script.sh line 45: unexpected `fi` .

Line 9: ERROR: Unable to locate dependency 'libsys.so'.

Line 10: error writing log file — permission denied.

Line 11: Retrying... error persists.

Line 12: All retries failed due to critical ERROR.

- `grep -i "error" logs.txt` (What does the -i option?)
- If you want to find a file do: `sudo find / -name “filename”`, sudo because you will look everywhere, instead of / use ~ to look only in your home

Search with grep

- **grep** : is for filtering/searching'

Examples:

- `grep "root" /etc/passwd` and `cat /etc/passwd | grep root`
- `grep -v "localhost" /etc/hosts` # Show lines not containing 'localhost'
- `grep -n "root" /etc/passwd` # Show matching lines with line numbers
- `grep -c "/bin/bash" /etc/passwd` # How many line match
- `grep -r "TODO" ~/projects` # Search recursively in a folder
- `grep -l "password" *.conf` # List only files containing 'password'

Linux commands

awk	allows manipulation of text	more	scroll through file a page at a time
bg	place suspended job into background	mv	change the name of a file (move)
cal	display calendar	nano/pico	text editors
cat	view contents of a file	printenv	display shell variables
cd	change directory	ps	show current process information
chmod	change permissions on a file/directory	pwd	print current working directory
cp	copy a file	rm	delete or remove a file
cut	extract a field of data from text output	rmdir	delete or remove a directory
diff	compare files line by line	sed	stream editor
echo	output text to the terminal or to a file	sleep	pause
emacs	text editor	sort	perform a sort of text
fg	bring suspended job to foreground	tail	view end of the file
file	display file type	touch	create an empty file or update timestamps
find	search for files	tr	character substitution tool
grep	search a file or command output for a pattern	uniq	remove identical, adjacent lines
head	view beginning of file	vi/vim	text editor
history	display list of most recent commands	wc	print number of lines, words or characters
less	scroll forward or back through a file	which	shows full path of a command
ln	create a link to a file	whoami	displays username
ls	list files in a directory		
man	view information about a command		
mkdir	make directory		

Lab

Objective: Build a complete command-line pipeline using redirection, piping, and text processing tools to extract structured information from real or synthetic system logs.

Scenario: Your team is helping a system administrator understand the content of a large text log. You will create a report that includes search, summary statistics, and file redirection.

Challenge-Based Instructions: Your working folder should be: ~/cli_project

Output files to use: errors.txt, warnings.txt, summary.txt, final_report.txt

1. Copy a log file to your working folder.
2. Extract all lines that contain the words error and warn, and save them to two different files.
3. Find how many lines are in each file. Then, try counting the number of words and characters too.
4. Identify the 5 earliest and 5 latest log messages for errors and warnings.
5. From each set, extract the timestamps (assume they are the first 2 words per line). Count how often each timestamp occurs.
6. Combine the results into a file called final_report.txt. Add a line showing the total number of warn messages.
7. Bonus: Try doing step 5 in one line using only pipes and no intermediate file

Lab

Questions to Answer:

How many lines contain error and warn?

What is the most frequent timestamp for error messages?

What is the longest line found in errors.txt?

What is the difference between using > and >> in your summary file?

Which options of grep and wc did you find most useful and why?

What did using a pipeline help you do more efficiently?

Lab

Objective: Build a complete command-line pipeline using redirection, piping, and text processing tools to extract structured information from real or synthetic system logs.

Scenario: Your team is helping a system administrator understand the content of a large text log. You will create a report that includes search, summary statistics, and file redirection.

Step-by-step Tasks (Explore Options!):

1. Prepare Environment:

```
mkdir ~/cli_project  
cd ~/cli_project  
cp /var/log/syslog .
```

2. Search for key terms:

```
grep -i "error" syslog > errors.txt  
grep -i "warn" syslog > warnings.txt
```

Lab

3. Play with wc and make stats:

```
wc -l errors.txt > summary.txt  
wc -w errors.txt >> summary.txt  
wc -c errors.txt >> summary.txt  
wc -L errors.txt >> summary.txt
```

Bonus .Play with grep:

```
grep -v "info" syslog > without_info.txt # Exclude matches  
grep -n "warn" syslog > numbered_warns.txt # Show line numbers  
grep -c "warn" syslog >> summary.txt    # Count matches
```

4. Inspect and slice data

```
head -n 5 errors.txt > first_errors.txt  
tail -n 5 warnings.txt > last_warnings.txt
```

5. Extract and sort log timestamps (assumes timestamp in fields 1 and 2)

```
cut -d' ' -f1,2 errors.txt | sort | uniq -c > error_times.txt  
cut -d' ' -f1,2 warnings.txt | sort | uniq -c > warning_times.txt
```


Lab

6. Make the final report:

```
cat summary.txt > final_report.txt
```

```
cat error_times.txt >> final_report.txt
```

```
echo "" >> final_report.txt
```

```
cat warning_times.txt >> final_report.txt
```

7. Optional (Bonus): Use pipeline instead of intermediate files:

```
grep -i error syslog | cut -d' ' -f1,2 | sort | uniq -c | sort -nr | head -n 5
```

Questions to Answer:

1. How many total lines mention "error" and "warn"?
2. What are the five most frequent timestamps associated with errors?
3. What is the longest line (in characters) found in errors.txt?
4. Which grep option did you find most useful? Why?
5. Which tools in this lab could be reused for analyzing other files?
6. What is the difference between using a pipeline and using intermediate files?

Shell Scripting I: Basics & Automation

Session 6

Objectives:

- Know what a shell script is and how to write one
- Understand the use of the shebang (!) line and permissions
- Be able to use variables, read, and echo
- Write conditional structures like if, else, and case
- Prepare for automation through scripting

What is a shell script?

- A script is a plain text file containing commands designed to be run by Linux
- Executed sequentially by the shell
- Like a to-do list for your terminal!

shell scripts include file manipulation, program execution, and printing text.

Example:

```
#!/bin/  
#print Hello, word  
echo "Hello, world!«  
# show the date  
date
```

Creating and running a shell script

Shell scripts have several required constructs that tell the shell environment what to do and when to do it.

shebang  `#!/bin/bash`
`#print Hello, word`
`echo "Hello, world!"`
`# show the date`
`date`

#! shebang

The seemingly insignificant #! characters at the beginning of a shell script has a major significance on how your script will be executed.

The shebang is the combination of **the # (pound key) and ! (exclamation mark)**. It is used to specify the interpreter with which the given script will be run by default

e.g.:

#!/bin/bash -> interpreter should be bash shell

#!/bin/zsh -> interpreter should be Z shell

So it is important to precise the shell interpreter in the script as different shells have different syntax

Creating and running a shell script

Shell scripts have several required constructs that tell the shell environment what to do and when to do it.

```
#          → #!/bin/  
           #print Hello, word  
           echo "Hello, world!"  
           # show the date  
           date
```

All the lines starting by **#** (except the shebang) are **comments**, meaning that they are ignored

However, they are very important to give explanations, notes for the collaborators

Creating and running a shell script

Shell scripts have several required constructs that tell the shell environment what to do and when to do it.

command  `#!/bin/bash`
`#print Hello, word`
`echo "Hello, world!"`
`# show the date`
`date`

Then you can write as many command lines as you want. Here, the script will print « Hello, world!»

Creating and running a shell script

- Create a file with .sh extension:

-> gedit hello.sh

- Add commands with a shebang:

```
#!/bin/bash
```

```
#print Hello, word
```

```
echo "Hello, world!"
```

```
# show the date
```

```
date
```

- Save and exit
- Change the permission (session 9)

-> chmod +x hello.sh

- Run it

-> ./hello.sh, you can also run bash hello.sh and it will ignore the shebang

Variables

- Syntax:

```
name="Thomas"  
echo "Welcome $name"
```

- NO SPACE BEFORE/AFTER =

- Refer with \$

```
#!/bin/sh  
MY_MESSAGE="Hello World"  
echo $MY_MESSAGE
```

- echo "The number of line is: \$(wc -l errors.txt)"

Reading input from user

- Syntax:

```
#!/bin/bash
echo "What is your name?"
read MY_NAME
echo "Hello $MY_NAME - hope you're well."
```

- **Read** stores input into a variable
- Use **export** if the variable needs to be available in sub-shells or other script
export MY_NAME

Reading input from user

- Example myvar.sh:

```
#!/bin/sh  
echo "MYVAR is: $MYVAR"  
MYVAR="hi there"  
echo "MYVAR is: $MYVAR"
```

What is the output, if you run ./myvar.sh ?

Reading input from user

- Example myvar.sh:

```
#!/bin/sh  
echo "MYVAR is: $MYVAR"  
MYVAR="hi there"  
echo "MYVAR is: $MYVAR"
```

What is the output, if you run ./myvar.sh ?

MYVAR is:

MYVAR is: hi there

Reading input from user

- Now try:

```
$ MYVAR="hello"
```

```
$ ./myvar.sh
```

Reading input from user

- Now try:

```
$ MYVAR="hello"
```

```
$ ./myvar.sh
```

Output:

```
MYVAR is:
```

```
MYVAR is: hi there
```

It is the same, because when you run `./myvar.sh`, a new shell is used

You can use `export`:

```
$ MYVAR="hello"
```

```
$ export MYVAR
```

```
$ ./myvar.sh
```

Reading input from user

You can use export:

```
$ MYVAR=hello
```

```
$ export myvar
```

```
$ ./myvar.sh
```

Output:

```
MYVAR is: hello
```

```
MYVAR is: hi there
```

If you do: `$ echo MYVAR` you will see hello

Once the shell script exits, its environment is destroyed. But MYVAR keeps its value of hello within your interactive shell.

Reading input from user

```
#!/bin/bash
echo "What is your name?"
read USER_NAME
echo "Hello $USER_NAME"
echo "I will create you a file called $USER_NAME_final.txt"
touch $USER_NAME_final.txt
```

What will happen?

Reading input from user

What can we do?

Curly brackets

What can we do? **CURLY brackets**

- Curly braces {} help clarify variable boundaries
- Prevent confusion when variables are next to other characters

```
#!/bin/bash
echo "What is your name?"
read USER_NAME
echo "Hello $USER_NAME"
echo "I will create you a file called ${USER_NAME}_final.txt"
touch "${USER_NAME}.txt"
```

Conditions if/else

Structure:

```
if [ ... ];then
  # if-code
else
  # else-code
fi
```

Common numeric test operators:

- eq (equal)
- ne (not equal)
- gt (greater than)
- lt (less than)
- ge (greater or equal)
- le (less or equal)

`$(( ... ))` is **arithmetic expansion**
Only for intergers

Common file test operators:

- f file → is a file
- d dir → is a directory
- x file → is executable
- r file → is readable
- w file → is writable

Examples

Check if a myfile.txt exists:

Examples

Check if a file exists:

```
if [ -f myfile.txt ]; then  
    echo "File exists."  
else  
    echo "File not found."  
fi
```

Examples

Check if a file exists:

```
#!/bin/bash
if [ -f myfile.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

Compare two numbers (a,b):

Examples

Check if a file exists:

```
#!/bin/bash
if [ -f myfile.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

Compare two numbers:

```
#!/bin/bash
echo "Enter two numbers:"
read a b
if [ $a -eq $b ]; then
    echo "Both are equal"
elif [ $a -gt $b ]; then
    echo "$a is greater than $b"
else
    echo "$b is greater than $a"
fi
```

Examples

Check if a file exists:

```
#!/bin/bash
if [ -f myfile.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

Compare two numbers:

```
#!/bin/bash
echo "Enter two numbers:"
read a b
if [ $a -eq $b ]; then
    echo "Both are equal"
elif [ $a -gt $b ]; then
    echo "$a is greater than $b"
else
    echo "$b is greater than $a"
fi
```

Ask for username and check if it is root:

Examples

Check if a file exists:

```
#!/bin/bash
if [ -f myfile.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

Compare two numbers:

```
#!/bin/bash

echo "Enter two numbers:"
read a b
if [ $a -eq $b ]; then
    echo "Both are equal"
elif [ $a -gt $b ]; then
    echo "$a is greater than $b"
else
    echo "$b is greater than $a"
fi
```

Ask for username and check if it is root:

```
#!/bin/bash
echo "Enter your username:"
read user
if [ "$user" == "root" ]; then
    echo "You are the administrator."
else
    echo "You are a regular user."
fi
```

Case structure

The case structure is perfect when you have many conditions to evaluate. Instead of chaining multiple if and elif, you define patterns in a clean and readable block.

- Start with case
- Patterns use | for alternates
- Every block ends with ;;
- esac closes the case lock

Case structure

Exemples.

- Give a number between 1 and 3
- Print “your number is one” if it is 1, etc....

```
echo "Enter a number (1-3):"
```

```
read choice
```

```
if [ "$choice" -eq 1 ]; then
```

```
    echo "You chose one."
```

```
elif [ "$choice" -eq 2 ]; then
```

```
    echo "You chose two."
```

```
elif [ "$choice" -eq 3 ]; then
```

```
    echo "You chose three."
```

```
else
```

```
    echo "Invalid choice."
```

```
fi
```

Case structure

Examples.

- Give a number between 1 and 3
- Print “your number is one” if it is 1, etc....

```
echo "Enter a number (1-3):"  
read choice  
if [ "$choice" -eq 1 ]; then  
    echo "You chose one."  
elif [ "$choice" -eq 2 ]; then  
    echo "You chose two."  
elif [ "$choice" -eq 3 ]; then  
    echo "You chose three."  
else  
    echo "Invalid choice."  
fi
```



`if` is used for conditional logic; `case` is cleaner for multiple value comparisons. Use `case` when comparing a single variable to many values

```
echo "Enter a number from 1 to 3:"  
read choice  
  
case $choice in  
    1) echo "You chose one.;;"  
    2) echo "You chose two.;;"  
    3) echo "You chose three.;;"  
    *) echo "Invalid choice.;;"  
esac
```

What you learn:

You now know:

- How to write and run scripts
- Use variables and input
- Create decisions using if and case
- Apply comments and use the `#!` line:

Let's practice!!

Labs

Lab 1: Swapping Two Variables (10 min)

- Create a script swap.sh:
- Ask for 2 values (A and B)
- Swap and display their new values

Labs

Lab 1: Swapping Two Variables (10 min)

```
#!/bin/bash
echo -n "Enter value for A: "
read a
echo -n "Enter value for B: "
read b
t=$a
a=$b
b=$t
echo "Values after Swapping:"
echo "A = $a, B = $b"
```

Labs

Lab 2: Fahrenheit to Celsius (10 min)

- Create degconv.sh:
- Ask user for Fahrenheit temperature
- Convert to Celsius using formula
- Show the result

Labs

Lab 2: Fahrenheit to Celsius (10 min)

```
#!/bin/bash  
echo -n "Enter Fahrenheit: "  
read f  
c=$(( (f - 32) * 5 / 9 ))  
echo "Centigrade is: $c"
```

Labs

Lab 3: Biggest of Three Numbers (15 min)

- Script big3.sh:
- Ask user for three numbers
- Use if and -gt to find the biggest
- Display which is biggest

Labs

Lab 3: Biggest of Three Numbers (15 min)

```
#!/bin/bash
echo -n "Enter values for A B and C: "
read a b c
if [ $a -gt $b ] && [ $a -gt $c ]; then
    echo "A is the biggest"
elif [ $b -gt $c ]; then
    echo "B is the biggest"
else
    echo "C is the biggest"
fi
```

Labs

Lab 4: Grade Determination (15 min)

- Script grade.sh:
- Ask for a mark (0–100)
- Output a grade using if/elif logic
- S: >90, A: >80, B: >70, C: >60, D: >50, U: <50

Labs

Lab 4: Grade Determination (15 min)

```
#!/bin/bash
echo -n "Enter your mark: "
read mark
if [ $mark -gt 90 ]; then
    echo "S Grade"
elif [ $mark -gt 80 ]; then
    echo "A Grade"
elif [ $mark -gt 70 ]; then
    echo "B Grade"
elif [ $mark -gt 60 ]; then
    echo "C Grade"
elif [ $mark -gt 55 ]; then
    echo "D Grade"
elif [ $mark -ge 50 ]; then
    echo "E Grade"
else
    echo "U Grade"
fi
```

Labs

Lab 5: Vowel or Consonant (10 min)

- Script vowel.sh:
- Read a single lowercase character
- Use case to determine if it's a vowel

Labs

Lab 5: Vowel or Consonant (10 min)

```
#!/bin/bash
echo -n "Enter a lowercase character: "
read choice
case $choice in
a|e|i|o|u)
    echo "It's a vowel";;
*)
    echo "It's a consonant";;
esac
```

Labs

Lab 6: Mini Calculator (20 min)

- Script calc.sh:
- Ask for two numbers
- Show menu: add, subtract, multiply, divide
- Use case and expr to display result

Labs

Lab 6: Mini Calculator (20 min)

```
#!/bin/bash
echo -n "Enter two numbers: "
read a b
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
echo -n "Choose an option: "
read op
case $op in
  1) echo "$a + $b = $((a + b))";;
  2) echo "$a - $b = $((a - b))";;
  3) echo "$a * $b = $((a * b))";;
  4) echo "$a / $b = $((a / b))";;
  *) echo "Invalid option";;
esac
```

Labs

Lab 7: Check lines and words (20 min)

Write a shell script which counts the number of lines and number of words present in a given file.

Use echo, read, wc to do:

- Ask the user for the file name
- Check if the file exist and count number of lines and words

Labs

Lab 7: Check lines and words (20 min)

```
#!/bin/bash
```

```
echo "Enter the file name:"
```

```
read filename
```

```
# Check if the file exists
```

```
if [ -f "$filename" ]; then
```

```
    echo "Counting lines and words in '$filename'..."
```

```
    echo "Number of lines: $(wc -l < "$filename")"
```

```
    echo "Number of words: $(wc -w < "$filename")"
```

```
else
```

```
    echo "File '$filename' not found."
```

```
fi
```

Shell Scripting II: Advanced Scripts

Session 7

Shell Scripting II: Advanced Scripts

Session 7

Objectives:

- Handle command-line arguments using \$0..\$9, \$#, \$@, and shift
- Use functions in shell scripts
- Implement control structures (case, if/elif/else, loops) in real examples
- Improve reusability and modularity with functions
- Use exit codes and simple debugging tools
- Write scripts that behave like command-line tools

Shell Positional Parameters

There are a set of variables which are set for you already, and most of these cannot have values assigned to them.

- \$0: Name of script
- \$1..\$9: Arguments
- \$#: Number of arguments
- \$@: All arguments

Shell Positional Parameters

myvar.sh

```
#!/bin/sh
```

```
echo "I was called with $# parameters"
```

```
echo "My name is $0"
```

```
echo "My first parameter is $1"
```

```
echo "My second parameter is $2"
```

```
echo "All parameters are $@"
```

Shell Positional Parameters

myvar.sh

`#!/bin/sh`

`echo "I was called with $# parameters"`

`echo "My name is $0"`

`echo "My first parameter is $1"`

`echo "My second parameter is $2"`

`echo "All parameters are $@"`

`./myvar.sh`

Shell Positional Parameters

myvar.sh

`#!/bin/sh`

`echo "I was called with $# parameters"`

`echo "My name is $0"`

`echo "My first parameter is $1"`

`echo "My second parameter is $2"`

`echo "All parameters are $@"`

./myvar.sh

I was called with 0 parameters

My name is ./myvar.sh

My first parameter is

My second parameter is

All parameters are

Shell Positional Parameters

myvar.sh

```
#!/bin/sh
```

```
echo "I was called with $# parameters"
```

```
echo "My name is $0"
```

```
echo "My first parameter is $1"
```

```
echo "My second parameter is $2"
```

```
echo "All parameters are $@"
```

./myvar.sh

```
I was called with 0 parameters
```

```
My name is ./myvar.sh
```

```
My first parameter is
```

```
My second parameter is
```

```
All parameters are
```

./myvar.sh hello world earth

Shell Positional Parameters

myvar.sh

```
#!/bin/sh
```

```
echo "I was called with $# parameters"
```

```
echo "My name is $0"
```

```
echo "My first parameter is $1"
```

```
echo "My second parameter is $2"
```

```
echo "All parameters are $@"
```

./myvar.sh

```
I was called with 0 parameters
```

```
My name is ./myvar.sh
```

```
My first parameter is
```

```
My second parameter is
```

```
All parameters are
```

./myvar.sh hello world earth

```
I was called with 3 parameters
```

```
My name is ./var3.sh
```

```
My first parameter is hello
```

```
My second parameter is world
```

```
All parameters are hello world earth
```

While

The **while** loop in shell is especially useful when you don't know in advance how many times you'll need to repeat something : for example, reading a file line by line or waiting for a process to finish

- while evaluates a test condition
- while [condition]: the loop continues while the condition is true
- do ... done: everything between do and done is the loop body
- Common use cases: reading lines, looping over arguments, timers

While

```
#!/bin/bash
count=1
while [ $count -le 5 ]; do
    echo "Count is $count"
    count=$((count + 1))
done
```

Output:

While

```
#!/bin/bash
count=1
while [ $count -le 5 ]; do
    echo "Count is $count"
    count=$((count + 1))
done
```

Output:

```
Count is 1
Count is 2
Count is 3
Count is 4
Count is 5
```

Shift command

The **shift** command in shell scripting discards the first positional parameter (\$1) and shifts all others one position to the left.

- shift : by default shifts by 1
- shift N : shifts by N positions
- Affects \$1, \$2, ..., \$n
- \$# decreases with each shift (the number of arguments)

When to use shift

- When you want to process all arguments one-by-one
- Inside loops to consume parameters cleanly
- Especially useful when argument count is variable

Shift command

```
#!/bin/bash
while [ "$#" -gt 0 ]; do
    echo "Argument: $1"
    shift
done
```

./shift_example.sh hello world 42 earth

Shift command

```
#!/bin/bash
while [ "$#" -gt 0 ]; do
    echo "Argument: $1"
    shift
done
```

./shift_example.sh hello world 42 earth

- Prints each argument
- Removes it from the list (\$1 becomes \$2, etc.)
- Stops when no arguments remain

Argument: hello

Argument: world

Argument: 42

Argument: earth

Functions

A **function** in a shell script is a reusable block of code that can be called by name, helping organize and avoid repetition.

Structure:

```
function_name() {  
    # commands  
}
```

Or

```
function function_name {  
    # commands  
}
```

Useful:

- When you need to repeat the same logic multiple times
- To make scripts easier to read and maintain
- To logically separate different parts of a script

Functions

Example:

```
hello() {  
    echo "Hello!"  
}  
hello
```

Output: Hello!

With argument:

```
hello_person() {  
    echo "Hello, $1!"  
}  
hello_person "Thomas"
```

Output: Hello, Thomas

You can source your function to use it . hello_func.sh (or source)

Using exit, return, and \$?

Use **exit**, **return**, and **\$?** to handle success or failure in scripts and functions, and to debug by checking what happened last.

- **exit <code>** : Terminates the script with a status code (usually 0 = success, non-zero = error)
- **return <code>** : return is like exit, but it's used inside functions : it tells us whether the function succeeded or failed.
- **\$?** : Stores the exit status of the last command run

Using exit, return, and \$?

Exit ./checkfile.sh

```
#!/bin/sh
if [ ! -f myfile.txt ]; then
    echo "File not found!"
    exit 1
fi
```

Output:

File not found

Then do echo \$?

After the output run

echo \$?

Output: 1

return

```
#!/bin/bash
```

```
check_number() {
    if [ "$1" -gt 10 ]; then
        return 0  # success
    else
        return 1  # failure
    fi
}
```

```
check_number 15
echo "Return code was: $?"
```

Output: Return code was 0

```
check_number 5
echo "Return code was: $?"
```

Output: Return code was 1

Debugging

Use **exit**, **return**, and **\$?** to handle success or failure in scripts and functions, and to debug by checking what happened last.

Useful debugging tools:

- **set -x** : Print each command before it runs (trace mode)
- **set +x** : Turn off trace mode
- **echo** : Print variable values manually
- **\$?** : Check the result of the previous command
- **bash -n script.sh** : Check for syntax errors without running
- **trap** : Handle unexpected errors or cleanup on script exit. It lets us clean up resources, log actions, or gracefully handle interruptions when a script exits or gets interrupted.

Debugging

Example

```
set -x  
NAME="Thomas"  
echo "Hello, $NAME"  
set +x
```

Debugging

Example

```
set -x  
NAME="Thomas"  
echo "Hello, $NAME"  
set +x
```

Outputs: Bash will print every command it executes, along with its expanded arguments

```
+ NAME=Thomas  
+ echo 'Hello, Thomas'  
Hello, Thomas  
+ set +x
```


Labs

Lab 1: Read lines from a file (10 min)

- Create a file with random text, called it “myfile.txt”
- Script readline.sh:
- Read line by line

Labs

Lab 1: Read lines from a file (10 min)

```
#!/bin/bash
```

```
# Step 1: Create a file with random text
```

```
cat > myfile.txt << EOF
```

```
apple banana cherry
```

```
dog elephant frog
```

```
grape house igloo
```

```
jackal kite lemon
```

```
monkey night owl
```

```
EOF
```

```
# Step 2: Read the file line by line
```

```
echo "Reading lines from myfile.txt:"
```

```
while read line; do
```

```
    echo "Line: $line"
```

```
done < myfile.txt
```

Labs

Lab 2: Print Positional Parameters

Script: args.sh

Instructions:

- Print the following:
 - \$# (number of arguments)
 - \$0 (script name)
 - \$1, \$2 (first and second arguments)
 - \$@ (all arguments)
- Run
 - ./args.sh A B C
 - ./args.sh "hello" "world"

Labs

Lab 2: Print Positional Parameters

```
#!/bin/bash  
echo "Script name: $0"  
echo "Number of arguments: $#"  
echo "First argument: $1"  
echo "Second argument: $2"  
echo "All arguments: $@"
```

Labs

Lab 3: Creating Users with a Function (20 min)

- Create script adduser.sh
- Define a function add_a_user
- Use positional parameters to:
 - Assign \$1 to USER
 - Assign \$2 to PASSWORD
 - Use shift to remove the first two arguments
 - Assign the rest (\$@) to COMMENTS
- Simulate creating users with echo commands
- Call the function 2 times with different arguments

Labs

Lab 3: Creating Users with a Function (20 min)

```
#!/bin/sh
add_a_user() {
    USER=$1
    PASSWORD=$2
    shift; shift
    COMMENTS=$@
    echo "Adding user $USER ..."
    echo useradd -c "$COMMENTS" $USER
    echo passwd $USER $PASSWORD
    echo "Added user $USER ($COMMENTS) with pass $PASSWORD"
}
```

```
# Main script starts here
echo "Start of script..."
add_a_user thomas tompwd Thomas deJaeger the Teacher
add_a_user student studentpwd Student Good students
echo "End of script..."
```

Labs

Lab 5: Greeting Function with Validation

- Create script greet.sh
 - Define a function greet_user that takes a name
 - If name is missing, show an error
 - Otherwise, print: "Hello, NAME. Today is DATE."

Ex:

`./greet.sh Thomas`

Result: Hello, Thomas. Today is Fri May 2 12:00:00 CEST 2025

Hint: You need to check if the argument exist or not, you can use:

- `-z STRING` is a test operator used inside `[ ... ]` to check whether a string is empty.

Labs

Lab 5: Greeting Function with Validation

```
#!/bin/bash
greet_user() {
  if [ -z "$1" ]; then
    echo "Error: No name provided"
    return 1
  fi
  echo "Hello, $1. Today is $(date)."
}

greet_user "$1"
```


Processes & Init Systems

Session 8

Processes & Init Systems

Session 8

Objectives:

- Understand the role of the init system and what systemd does
- List and monitor processes using tools like ps, top, and htop
- Start, stop, enable, disable, and check services with systemctl
- Understand process IDs, foreground/background jobs, and signals
- Use commands like kill, nice, jobs, &, and killall to manage processes

What is systemd?

systemd: is the first process (PID 1) launched by the Linux kernel after boot

- Manages services, logging, dependencies, and system states
- It replaces older SysV init and offers faster boot, better logging, and cleaner service control
- Use unit file to define and manage a resource or service on a Linux system.

Each unit file tells systemd:

- What to start
- How to start it
- When to start it
- How to manage restarts
- What dependencies exist

Ex:	Type	Description	Example
	.service	For running services or daemons	nginx.service
	.timer	For scheduling tasks (like cron)	backup.timer
	.mount	For managing mounted filesystems	home.moun

Controlling Services with systemctl

systemctl: is the Swiss army knife for systemd. You'll use it to start, stop, restart, enable, and inspect services.

Command

`systemctl status`

`systemctl status <service>`

`systemctl start <service>`

`systemctl stop <service>`

`systemctl restart <service>`

`systemctl enable <service>`

`systemctl disable <service>`

`systemctl list-units --type=service`

What it does

Show system-wide status summary

Show detailed status of a service

Start the service **now**

Stop the service **now**

Restart the service

Enable it to start on **boot**

Disable autostart at boot

Show all currently active services

Controlling Services with systemctl

Example: Let play with the ssh service which is **network service** that allows users to securely access a computer remotely over a network (more details in future lecture).

1. Check if ssh is running:

- `systemctl status ssh` (**maybe need sudo apt install openssh-client and apt install openssh-server**)

Output:

ssh.service - OpenBSD Secure Shell server

Loaded: loaded (/lib/systemd/system/ssh.service; **enabled**; vendor preset: enabled)

Active: active (running) since Fri 2025-05-02 10:43:21 CEST; 2h 16min ago

Main PID: 791 (sshd)

Tasks: 1 (limit: 4575)

Memory: 2.1M

CPU: 35ms

CGroup: /system.slice/ssh.service

└─791 /usr/sbin/sshd -D

Controlling Services with systemctl

Example: Let play with the ssh service which is **network service** that allows users to securely access a computer remotely over a network (more details in future lecture).

1. Verify if is it enable at boot:
 - `systemctl is-enabled ssh`
Output:
Enabled
2. Disable it (just for testing)
 - `sudo systemctl disable ssh`
3. Verify :
 - `systemctl is-enabled ssh`
Output:
Disabled
4. Enable it again :
 - `sudo systemctl enable ssh`

Controlling Services with systemctl

Example: Let play with the ssh service which is **network service** that allows users to securely access a computer remotely over a network (more details in future lecture).

1. Stop ssh:

- `sudo systemctl stop ssh`

2. Verify

- `systemctl status ssh`

Output:

ssh.service - OpenBSD Secure Shell server

Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)

Active: inactive (dead) since Fri 2025-05-02 13:00:12 CEST; 2s ago

3. Start it again :

- `sudo systemctl start ssh`

Viewing & Managing Processes

What is a process: A process is a running instance of a program. Every command you run (even ls) becomes a process.

Each process has:

- A PID (Process ID)
- A PPID (Parent Process ID)
- A status (running, sleeping, zombie, etc.)
- A user, CPU and memory usage

Viewing & Managing Processes

Commands:

Command	Description
ps aux	Show all running processes
ps -f	Full-format listing with parent info
top	Real-time monitoring
htop	Graphical process viewer
kill PID	Send signal to terminate a process
kill -9 PID	Forcefully stop a process
killall name	Kill all processes with a given name

Viewing & Managing Processes

EX: `ps aux | head -5`

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.1	22536	1208	?	Ss	10:00	0:01	/sbin/init
user	1547	0.1	1.0	52864	10532	?	Sl	10:01	0:12	/usr/bin/code
user	2000	0.0	0.2	12000	1500	?	S	10:10	0:00	bash

ps	The process status command : used to list running processes
aux	Options to show all processes in a detailed, non-truncated format
head -5	Displays only the first 5 lines of the output

Viewing & Managing Processes

EX: ps aux | head -5

```
USER  PID %CPU %MEM  VSZ  RSS TTY   STAT START  TIME COMMAND
root   1  0.0  0.1 22536 1208 ?    Ss   10:00  0:01 /sbin/init
user  1547  0.1  1.0 52864 10532 ?    Sl   10:01  0:12 /usr/bin/code
user   2000  0.0  0.2 12000 1500 ?    S    10:10  0:00 bash
```

Column Meaning

USER	The user who owns the process
PID	Process ID
%CPU	CPU usage
%MEM	Memory usage
VSZ	Virtual memory size
RSS	Resident memory size
STAT	Process state
START	Time the process started
TIME	CPU time consumed
COMMAND	Command used to start the process

Viewing & Managing Processes

EX: Code explanation for the program status

Code	Meaning
R	Running: currently using CPU
S	Sleeping: waiting for event (e.g., input, time)
D	Uninterruptible sleep: usually waiting for I/O (disk, network)
Z	Zombie: completed but not cleaned up by parent
T	Stopped: paused (e.g., Ctrl+Z)
X	Dead: defunct, should not appear
I	Idle: (kernel thread) only in some systems like htop

Flag	Meaning
s	Session leader (usually a shell)
l	Multi-threaded process (uses clone())
+	Process is in the foreground process group

Viewing & Managing Processes

EX: Kill a program

1. Start a dummy background program
sleep 300 &
2. `ps aux`
3. How to find directly your program?

Viewing & Managing Processes

EX: Kill a program

1. Start a dummy background program
sleep 300 &
2. `ps aux`
3. How to find directly your program?
 - `ps aux | grep sleep`
4. `kill <PID>`
5. Rerun `ps aux` and check that there is nothing

top vs ps

Feature	ps aux	top
Snapshot	One-time snapshot of processes	Real-time, continuously updating view
Output	Printed list (can be piped or saved)	Interactive full-screen interface
Usage	Useful for scripting, filtering	Great for live monitoring
Refresh	Does not update unless re-run	Auto-refreshes every 1–3 seconds
Sorting	Manual (sort) or via ps options	Built-in: press keys to sort by CPU, memory, etc.

In top:

Press k to kill a process

Press P to sort by CPU

Press M to sort by memory

Press q to quit round process group

Foreground & Background Jobs

Key Concepts:

- A foreground job runs in the terminal and blocks it until it finishes.
- A background job runs "behind the scenes" while you still use the terminal.
- You can pause, resume, or bring back background jobs.
- Jobs are tracked with job numbers like %1, %2, etc
- What is a Linux Daemon?
 - A system daemon in Linux is typically a background system process that awaits a specific set of conditions before jumping into action

Foreground & Background Jobs

Key Commands:

Command	Description
command &	Run command in the background
jobs	List current background and stopped jobs
fg %n	Bring job number n to the foreground
bg %n	Resume a stopped job in the background
CTRL+Z	Suspend (pause) the current foreground process
CTRL+C	Terminate the foreground process
kill %n	Send a signal to a job using its job number

Foreground & Background Jobs

EX

1. Start a dummy background program
sleep 300 &
Output: [1] 3456
Job [1] has process ID 3456.
2. Check Background Jobs
jobs
Output: [1]+ Running sleep 300 &.
3. Move it to the foreground
fg %1
4. Suspend a Foreground Job
CTRL+Z
Output [1]+ Stopped sleep 300

Signals & Process Priorities

- Every process can receive signals: special instructions from the OS or user
- Some signals can stop, pause, or restart a process
- Linux processes also have a priority (called nice value) that affects CPU scheduling

Signals

Signals: Common Examples

Signal	Code	Meaning
SIGTERM	15	Terminate the process gracefully
SIGKILL	9	Force kill the process immediately
SIGINT	2	Sent by CTRL+C in the terminal, to stop a program
SIGSTOP	19	Pause a process (like CTRL+Z)
SIGCONT	18	Resume a paused process

Signals & Process Priorities

Signals: How to send a signal

Command	Description
kill -15 PID (default)	Graceful termination
kill -9 PID	Force kill using SIGKILL
kill -l	List all available signals
killall name	Kill all processes with that name
trap	Catch or react to signals in a script

```
trap 'commands_to_run' SIGNAL_NAME
```

e.g: `trap 'echo "Goodbye!"; exit' SIGINT`

SIGINT = **interrupt signal**, sent when you press **Ctrl-C**., so if you press Ctrl-C you will see Goodbye

```
trap 'rm -f /tmp/myfile; echo "Cleaned up!'" EXIT
```

 other example useful, clean all the temp files when exiting

Signals

Examples:

sleep 300 &
ps aux | grep sleep

- Graceful Kill
 - kill -15 <PID>
- Force Kill
 - kill -9 <PID>

Process reaction

Can handle it: clean up, save data, exit

vs

Cannot handle it: terminated instantly

Process Priorities

Priority affects who gets CPU time first. Use nice to be polite with long-running background jobs.

Concept

nice

renice

Priority

Default

Description

Set process priority when launching it

Change priority of a running process

From -20 (highest) to 19 (lowest priority)

Most user processes start at priority 0

Process Priorities

Exemples: Runs sleep with lower priority

```
nice -n 10 sleep 1000 &  
ps aux | grep sleep
```

Check:

```
ps -o pid,ni,cmd -p <PID>
```

Rechange priority:

```
renice 5 <PID>
```


Creating a Custom systemd Service

- A service is a background program managed by systemd
- You can create your own service to automatically run your script at boot or on demand
- A unit file defines how the service should behave

Why?

You've written a script that logs temperature data every 30 seconds to a CSV file:

```
#!/bin/bash
while true; do
  echo "$(date), $(read_temp_sensor)" >> /var/log/temps.csv
  sleep 30
done
```

Problem:

If you run this script manually:

- It stops when the terminal closes
- It doesn't restart after reboot
- If it crashes, it doesn't come back

Why?

Solution: Turn it into a systemd service

- It starts automatically at boot
- Runs in the background continuously
- Restarts itself if it crashes
- Logs errors with journalctl

[Unit]

Description=Temperature Logger

[Service]

ExecStart=/usr/local/bin/log_temp.sh

Restart=always

[Install]

WantedBy=multi-user.target #“Enable this service when the system reaches **multi-user mode**

Example

- **Step 1 – Create a script**

Create /usr/local/bin/hello_loop.sh:

```
#!/bin/bash
while true; do
    echo "Hello from systemd at $(date)" >> /tmp/hello.log
    sleep 30
done
chmod +x /usr/local/bin/hello_loop.sh
```

- **Step 3 – Reload systemd and start the service**

```
sudo systemctl daemon-reload
sudo systemctl start hello.service
sudo systemctl enable hello.service
```

- **Check status and logs**

```
systemctl status hello.service
tail -f /tmp/hello.log
```

- **Step 2 – Create the service unit file**

Create /etc/systemd/system/hello.service

```
[Unit]
Description=My Hello Logging Service
```

```
[Service]
ExecStart=/usr/local/bin/hello_loop.sh
Restart=always
```

```
[Install]
WantedBy=multi-user.target
```

Labs

Lab 1: View and Inspect Running Processes

Objective: Learn to view running processes and understand key fields.

Instructions:

- Run `ps aux` to list all processes.
- Identify:
 1. Your shell process (`bash`)
 2. A background service (e.g. `systemd`, `ssh`)
 3. Any process with high CPU usage
- Use `ps -o pid,ppid,ni,stat,cmd` to see parent/child and priority info.
- Run `top`, sort by memory and CPU (press `M`, then `P`), then quit (`q`)

Labs

Lab 1: View and Inspect Running Processes

Solutions:

```
ps aux  
ps -o pid,ppid,ni,stat,cmd  
Top
```

Ps output:

```
USER  PID  PPID  NI  STAT  CMD  
user 2945 2891  0  Ss   bash  
root  1    0    0  Ss   /sbin/init  
user 3120 2945  0  R+   top
```

Top output

```
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND  
3120 user 20  0 162m 12m 5m  R  3.0  0.2  0:01.23 top
```

Labs

Lab 2: Foreground and Background Jobs

Objective: Practice managing foreground and background jobs.

Instructions:

- Run `sleep 120 &` to launch a background job.
- Use `jobs` to check the job status.
- Bring the job to foreground with `fg %1`.
- Press `CTRL+Z` to suspend it, then use `bg %1` to resume it in background.

Labs

Lab 2:: Foreground and Background Jobs

Solutions:

```
sleep 120 &  
jobs  
fg %1  
# (press CTRL+Z to suspend)  
bg %1
```

Output:

```
[1] 3355  
[1]+  Running      sleep 120 &  
[1]+  Stopped       sleep 120  
[1]+  sleep 120 &
```


Labs

Lab 3: Kill and Signals

Objective: Send signals to processes.

Instructions:

- Start `sleep 200 &`, get its PID using `ps`.
- Use `kill -15 <PID>` for graceful termination.
- Restart it, then use `kill -9 <PID>` to force kill it.
- Try `killall sleep` to terminate all sleep commands.

Labs

Lab 3: Kill and signal

Solutions:

```
sleep 200 &  
ps aux | grep sleep  
kill -15 <PID>  # graceful  
kill -9 <PID>   # force  
killall sleep   # by name
```

Output:

kill -15: process exits cleanly (preferred)

kill -9: kills even stuck processes

killall: kills all sleep processes

Labs

Lab 4: Nice and Renice

Objective: Adjust process priority.

Instructions:

- Run `nice -n 10 sleep 300 &`
- Use `ps -o pid,ni,cmd` to check the nice value.
- Use `renice 5 <PID>` to change it.
- Try renicing a root-owned process (should fail).

Labs

Lab 4: Nice and renice

Solutions:

```
nice -n 10 sleep 300 &  
ps -o pid,ni,cmd  
renice 5 <PID>
```

Output:

```
3333: old priority 10, new priority 5
```

```
renice: failed to set priority for 3333: Permission denied
```

Labs

Lab 5: Trap Signals in a Script

Objective: Catch and handle a signal in a script.

Instructions:

- Create trap_test.sh to catch SIGINT.
- Use trap to print a message when CTRL+C is pressed.
- Add a loop that prints “Running...” every 2 seconds.
- Run it and press CTRL+C.

Labs

Lab 5: Trap Signals in a Script Solutions:

```
#!/bin/bash
trap "echo 'SIGINT received. Exiting...'; exit" SIGINT
while true; do
  echo "Running..."
  sleep 2
Done
```

Output:

Running...

Running...

^C

SIGINT received. Exiting...

Labs

Lab 6: Create a Custom systemd Service

Objective: Create a service that logs the current time.

Instructions:

- Write a script logging time to /tmp/log.txt every 10s.
- Make it executable.
- Create /etc/systemd/system/timerlog.service.
- Run:
 - `sudo systemctl daemon-reload`
 - `sudo systemctl enable timerlog`
 - `sudo systemctl start timerlog`
- Check logs with `systemctl status` and `tail -f /tmp/log.txt`

Labs

Lab 6: Create a Custom systemd Service

Solutions:

```
#!/bin/bash
while true; do
  echo "$(date)" >> /tmp/log.txt
  sleep 10
done
chmod +x /usr/local/bin/timerlog.sh
```

[Unit]

Description=Log time every 10 seconds

[Service]

ExecStart=/usr/local/bin/timerlog.sh

Restart=always

[Install]

WantedBy=multi-user.target

```
sudo systemctl status timerlog
```

```
tail -f /tmp/log.txt
```


Users, Groups & Permissions

Session 9

Users, Groups & Permissions

Session 9

Objectives:

- Understand file and directory permissions (rwx, ls -l)
- Use chmod (symbolic and numeric modes)
- Modify ownership using chown and chgrp
- Understand how users and groups are stored in the system
- Create, delete, and modify users and groups
- Use sudo and su to escalate privileges

Password

Linux Login and Password Authentication:

- To enter a UNIX/Linux system, you need an **account**, a **username**, and a **password**



An account represents a **unique user identity** on the system. It contains all the user-related information and permissions.

A **username** is a **human-readable identifier** for the user account. **username** helps the system recognize which **account** you are trying to access.

A **password** is used for **authentication** — it ensures that only **authorized users** can access the system or their specific account.

Password

- Open `/etc/passwd`. What do you see?



Password

- Open `/etc/passwd`

`username:password:UID:GID:GECOS:home_directory:shell`



- **username**: The name of the user.
- **password**: A placeholder for the user's password (usually an 'x' or '*' if the password is stored elsewhere, like `/etc/shadow`).
- **UID**: User ID — a unique numeric identifier for the user.
- **GID**: Group ID — a numeric identifier for the user's primary group.
- **GECOS**: Optional information, often containing the real name or a description of the user (can be empty).
- **home_directory**: The path to the user's home directory.
- **shell**: The path to the user's default shell (e.g., `/bin/bash`).

Modify password

- Use the command **passwd**:

passwd <username>



- Adding a New User using **useradd**

useradd <username>

You can add also some information:

useradd -m -s /bin/bash thomas_dejaeger

- m: Creates the home directory for the user.
- s: Specifies the shell to use (e.g., /bin/bash)

Understanding File Permissions

Each file or directory has:

- **A type:** - for regular file, d for directory
- **A set of permissions:**
 - 3 for the owner (user): The owner of the file
 - 3 for the group: Members of the file's group
 - 3 for others: Everyone else (not user or group)

Ex: `ls -l`

Understanding File Permissions

`-rwxr-xr--`

First char: file type

Next 3: user permissions

Next 3: group permissions

Last 3: others

r = read

w = write

x = execute

Understanding File Permissions

-rwxr-xr--

- = regular file

rwx = user: read, write, execute

r-x = group : read, execute

r-- = other: read only

Symbolic chmod (change permissions)

There's a utility called **chmod**, which can modify permissions on files:

Legend:

- u = user
- g = group
- o = other
- a = all
- + = add, - = remove, = = set

Examples:

- `chmod u+xw file` # add execute & write for user
- `chmod g-w file` # remove write for group
- `chmod o=r file` # set other to read only

Numeric chmod and Permission Calculation

Use numbers from 0–7:

Permission	Value
r	4
w	2
x	1

For example:

`chmod 755 file` → `rwxr-xr-x`

`chmod 644 file` → `rw-r--r—`

7 = `rw`x, 5 = `r`-x, 4 = `r`--

chmod Demo (Before & After)

```
touch demo.txt
```

```
ls -l demo.txt
```

```
chmod 600 demo.txt
```

```
ls -l demo.txt
```

Output:

```
-rw----- 1 user user 0 demo.txt
```

Special Permissions: setuid, setgid, and Sticky Bit

What are Special Permissions?

Beyond basic rwx, Linux supports 3 special permission bits:

Bit	Applies To	Effect
setuid	Executable files	Run as the file's owner, not the executing user
setgid	Executable files or directories	Run as file's group (or inherit group on files)
sticky	Directories	Only file owner can delete, even if others have write access

Symbol	Meaning	Example
s	setuid or setgid is active and file is executable	-rwsr-xr-x (setuid on file) drwxrwsr-x (setgid on dir)
S	setuid or setgid is set, but file is not executable →	-rwSr--r-- or -rw-r-Sr--
t	Sticky bit is set (only owner can delete files)	drwxrwxrwt (e.g. /tmp)

Misusing them can open serious security holes — always review `ls -l` carefully before setting them

Default Permissions and umask

What is umask?

When you create a new file or directory, it starts with default permissions, and **umask** determines what gets subtracted from those defaults

By default:

Type	Default Mode	Meaning
Files	666	rw-rw-rw-
Directories	777	rwxrwxrwx

Default Permissions and umask

What is umask?

When you create a new file or directory, it starts with default permissions, and **umask** determines what gets subtracted from those defaults

By default:

Type	Default Mode	Meaning
Files	666	rw-rw-rw-
Directories	777	rwxrwxrwx

The **umask** value subtracts permissions from the default
It's a **4-digit** octal number (e.g., 0022 or 0077)

Default Permissions and umask

Digit position What it means

0 (1st) Special bits: setuid, setgid, sticky (rarely used in umask)

0 (2nd) Permissions removed from user (owner)

2 (3rd) Permissions removed from group

2 (4th) Permissions removed from others

What is the first digit?

It's a convention in Unix to write file modes and masks in 4-digit octal (base 8)

Some tools (like install, umask, chmod) expect or display it that way

Helps avoid confusion when using special bits like setuid

Default Permissions and umask

Type: umask

Output: 0022

Create a file:

`touch myfile.txt`

`ls -l myfile.txt`

Output:

`-rw-r--r--` Why?

Default: 666 (rw-rw-rw-) –(0022) -> 644 (rw-r--r--)

Default Permissions and umask

```
umask 0077  
touch private.txt  
ls -l private.txt
```

Output:

```
-rw----- Why?
```

Default: 666 (rw-rw-rw-) –(0077) -> 600 (rw-----)

Default Permissions and umask



IMPORTANT

Do not put random number for umask!!

For example I decided to type:

umask 0111

Create a folder

mkdir testdir

ls -ld testdir

What I will obtain?

Default Permissions and umask



IMPORTANT

Do not put random number for umask!!

For example I decided to type:

umask 0111

Create a folder

mkdir testdir

ls -ld testdir

What I will obtain?

777-111=666 -> drw-rw-rw-

No x!!! So you can not cd to this directory

cd: testdir: Permission denied

Solution?

Default Permissions and umask



IMPORTANT

Do not put random number for umask!!

For example I decided to type:

umask 0111

Create a folder

mkdir testdir

ls -ld testdir

What I will obtain?

777-111=666 -> drw-rw-rw-

No x!!! So you can not cd to this directory

cd: testdir: Permission denied

Solution

chmod +x testdir

ls -ld testdir

drwxrwxrwx

Ownership: chown and chgrp

Change file ownership with chown

```
ls -l file.txt
```

```
➔ -rw-r--r-- 1 alice editors 1234 May 2 14:10 file.txt
```

alice: owner user

editors=owner group

#Change user

```
chown newuser file.txt
```

#Change both, user and group

```
chown newuser:newgroup file.txt # change both
```

Change only file group using chgrp

```
chgrp newgroup file.txt
```

Ex:

```
touch demo.txt
```

```
sudo chown alice:editors demo.txt
```

```
ls -l demo.txt
```

Notes:

You need root privileges
(sudo) to change ownership

Only the owner or root can
change group (depending on
group membership)

Users and Groups: How Linux Stores Them

How does Linux manage users?

Linux stores user and group information in text-based system files. These are read by commands like `id`, `whoami`, `login`, `passwd`, etc.

File	Purpose	Example Line
<code>/etc/passwd</code>	Stores user accounts (username, UID, shell)	<code>alice:x:1001:1001:Alice:/home/alice:/bin/bash</code>
<code>/etc/shadow</code>	Stores encrypted passwords (restricted access)	<code>alice:\$6\$...:19000:0:99999:7:::</code>
<code>/etc/group</code>	Stores group info (name, GID, members)	<code>team:x:1002:bob,carol</code>

Useful commands:

<code>whoami</code>	# Print current username
<code>id</code>	# UID, GID, and all groups of current user
<code>groups</code>	# All groups of current user

Exemples

```
id bob -> uid=1002(bob) gid=1002(bob) groups=1002(bob),1003(team)
groups bob -> Show all groups user 'bob' belongs to
```

User and Group Management

What can you manage?

As a sysadmin (or a sudoer), you can:

- Create users and assign them to groups
- Set or change passwords
- Modify user properties (home, shell, groups)
- Remove users or groups

Examples:

- Create user

```
sudo useradd bob
```

- Set user's password

```
sudo passwd bob
```

- Create new group

```
sudo groupadd team
```


User and Group Management

Other examples:

(remember, with id bob:

uid=1002(bob) gid=1002(bob) groups=1002(bob))

- **Add User to a Secondary Group**

sudo usermod -aG team bob

-a = append (Don't overwrite)

-G = specify secondary group(s)

Check: groups bob -> bob : bob team

BE CAREFUL: sudo usermod -G team bob **OVERWRITES ALL THE GROUP MEMBERSHIPS**

- **Delete User or Group**

sudo userdel bob (if you want to remove also home directory add option -r)

sudo groupdel team

SU vs SUDO

su: Switch User (full session):

- You become root until you exit
- Requires the root user's password

You can switch to another user too: su Thomas, if you type whoiam the answer will be Thomas

sudo: Run One Command as Another User

- Executes one command as root
- Asks for your password, not root's
- Logs the command for security

Ex. sudo apt update

Who is sudo?

Type: getent group sudo

SU vs SUDO

Which should you use?

Use su when...

- You want a full root shell
- You know the root password
- You're in rescue/recovery mode

Use sudo when...

- You just need to run one admin command
- You're in the sudo group
- You're on a multi-user secure system

Add a user to the sudoers group:

```
sudo usermod -aG sudo bob
```

Controlling sudo Access with /etc/sudoers

Use sudo visudo (It checks for syntax errors and prevents you from breaking sudo (which could lock you out!))

Format:

username ALL=(ALL:ALL) ALL

Part	Meaning
username	The user allowed to use sudo
ALL	From any host (on multi-host setups, usually just ALL)
(ALL:ALL)	Can run commands as any user and as any group
ALL (last)	Can run any command

bob ALL=(root) /usr/bin/apt, /bin/mkdir, bob will be able to run sudo only for apt and mkdir command

LABS

Labs 1: Understand basic ownership and permission display.

- Create three files: file1.txt file2.txt file3.txt
- Who owns the files?
- What are the group permissions?
- Which files are writable?
- Clean everything

LABS

Solution 1: Understand basic ownership and permission display.

```
touch file1.txt file2.txt file3.txt  
ls -l file*
```

Output

```
-rw-r--r-- 1 user user 0 file1.txt  
-rw-r--r-- 1 user user 0 file2.txt  
-rw-r--r-- 1 user user 0 file3.txt
```

Owner: user

Group: user

Writable: Yes, for the owner (rw-)

Cleanup:

```
rm file1.txt file2.txt file3.txt
```

LABS

Labs 2:

Step 1: Explore Your Home Directory

- Open a terminal and go to your home directory.
- List your files and folders.
- Identify owners, groups, and permission symbols (rwx, etc.).

Step 2: Analyze and Copy a System File

- Examine the file /etc/passwd.
- Try to move it to your home directory.
- Try to copy it instead.
- Reflect on which command works and why.

Step 3: Inspect Ownership and Permissions

- After copying the file as datafile.txt, inspect:
- Who owns it?
- What are its access rights?

LABS (Solution)

Labs 2:

Step 1: Explore Your Home Directory

- `cd ~`
- `ls -l`
- File entries begin with -, directories with d. Owner and group appear in columns 3 and 4.

Step 2: Analyze and Copy a System File

- `ls /etc/passwd.`
- `mv /etc/passwd .`
- `cp /etc/passwd .`
- You can't move a system file (owned by root), but you can copy it.

Step 3: Inspect Ownership and Permissions

- `cp /etc/passwd datafile.txt`
- `ls -l datafile.txt`
- You are now the owner of the copied file. Permissions depend on your current umask

LABS

Labs 2:

Step 4: Test Read Restrictions

- Try reading datafile.txt.
- Then remove your own read access.
- Try reading it again — what happens?

Step 5: Make the File Read-Only

- Change permissions so the file is readable but **not writable** by anyone.

Step 6: Directory Access Test

- Create a folder (project/) and a file inside it.
- Remove execute permission from the folder.
- Try listing and accessing the file

Step 7: Modify Access Again

- Remove read permission, but restore execute permission.
- Attempt to enter the folder and open a file.

LABS (Solution)

Labs 2:

Step 4: Test Read Restrictions

- `cat datafile.txt.`
- `chmod u-r datafile.txt`
- `cat datafile.txt` # Permission denied

Step 5: Make the File Read-Only

- `chmod 444 datafile.txt (ls datafile.txt -r--r--r--)`

Step 6: Directory Access Test

- `mkdir project`
- `echo "test" > project/file.txt`
- `chmod -x project`
- `ls project` # Permission denied

Step 7: Modify Access Again

- `chmod u+x,u-r project`
- `cd project, ls, cat file.txt`

LABS

Labs 2:

Step 8: Restore Full Access

- Grant yourself full rights (rwx) on the project/ directory again.

Step 9: Lock Down a Documents Folder

- Create a folder structure: documents/versions/ with text files.
- Set permissions so only you can modify, but others may read or traverse.

Step 10: Read-Only Files

- Change all the files in documents/versions/ to read-only for everyone (including yourself).

Step 11: Script Directory Access

- Create a scripts/ folder and sample script files.
- Give yourself full access.
- Allow your group to execute, but block all access for others.

Step 12: Restrict Execution

- Set script files so that **only you and your group** can run them, with no access for others.

LABS (Solution)

Labs 2:

Step 8: Restore Full Access

- `chmod u=rwx project/`

Step 9: Lock Down a Documents Folder

- `mkdir -p documents/versions/`
- `touch documents/versions/file1.txt documents/versions/file2.txt`
- `chmod -R 755 documents/`

Step 10: Read-Only Files

- `chmod 444 documents/versions/*.txt`

Step 11: Script Directory Access

- `mkdir scripts`
- `touch scripts/script1.sh scripts/script2.sh`
- `chmod 750 scripts`
- `chmod 750 scripts/*.`

Step 12: Restrict Execution

- `chmod 750 scripts/*`

LABS

Labs 2:

Step 13: Group Ownership Practice

- Create folders: course/labwork/ and course/tests/.
- Change their group ownership to a common group (e.g., users).

Step 14: Set Permissions for Course Materials

- For the course/ structure:
 - Main folder: only owner and group can access
 - labwork/: group can read and enter
 - tests/: no access to group or others

Step 15: Secure Test & Lab Files

- Create files under labwork/ and tests/.
- Set them so only you can read and write them.

LABS (Solution)

Labs 2:

Step 13: Group Ownership Practice

- `mkdir -p course/labwork course/tests`
- `sudo chgrp users course/labwork course/tests`

Step 14: Set Permissions for Course Materials

- For the course/ structure:
 - `chmod 770 course`
 - `chmod 750 course/labwork`
 - `chmod 700 course/tests`

Step 15: Secure Test & Lab Files

- `touch course/labwork/lab1.txt course/tests/test1.txt`
- `chmod 600 course/labwork/lab1.txt course/tests/test1.txt.`

LABS

Step 16: Edit Draft Permissions

- Create a drafts/ folder and sample files.
- Remove all access for group and others on the folder.
- Then remove read access from group on the individual files

Step 17: Open Access to Project Files

- Grant read access to everyone on files inside the project/ directory

Step 18: Check Your umask

- Run the umask command.
- Write down the value you see.
- What does it mean?

Step 19–21: Experiment with umask

- Create files before and after changing the umask value (e.g., to 000).
- Observe how default permissions differ.

(Optional) Cleanup

- Optionally delete all files and folders created during the lab.

LABS (Solution)

Step 16: Edit Draft Permissions

- `mkdir drafts`
- `touch drafts/file1.txt drafts/file2.txt`
- `chmod 700 drafts`
- `chmod g-r drafts/file1.txt drafts/file2.txt`

Step 17: Open Access to Project Files

- `chmod a+r project/*`

Step 18: Check Your umask

- `umask.`
- 0022, folder will have by default `rw-r-xr-x` and files `rw-r--r--`
- **Step 19–21: Experiment with umask**
- `touch file1.txt`
- `umask 000`
- `touch file2.txt`
- `ls -l file1.txt file2.txt.`

(Optional) Cleanup

- `rm -r course drafts project`

Text Editors: vi/vim

Session 10

Text Editors: vi/vim

Session 10

Objectives:

- Understand the purpose and value of CLI text editors
- Use basic and advanced navigation in Vim
- Edit, copy, delete, and manipulate blocks of text using Vim modes
- Perform searches and global substitutions
- Choose between Vim and Emacs depending on context

What's a Text Editor? Why Use CLI Tools?

Content:

- Not word processors (no formatting)
- Fast, portable, no GUI needed
- Used for configs, logs, scripts, coding

What is VI/VIM?

Content:

- A screen-oriented text editor
- Released in 1991
- Improved version of “vi” released in 1976
- A “programmer’s editor”
- No mouse or ‘GUI’

Why is VI/VIM?

- **Efficiency:** Keyboard-only = very fast
- **Ergonomics:** Vim is designed to have your fingers always stay near the home row. This reduces hands fatigue and improves performance
- **Always available** (vim is POSIX)
- **It's Everywhere!** Vim exists on almost all machines. It's heavily used for system administration, programming, working with markup languages, and more!

VIM modes

vim is known for a modal editing scheme.

What is modal editing? In vim, various things that you typically do while editing text is split into different modes.

Normal mode (Esc)

Editor commands are used in this mode. The default mode.

Command-line mode (:)

Single line input, used mostly for file navigation. Exits mode after command

Insert mode (i)

Editing a text buffer. Similar to modern text editors.

Select mode (gh)

Printable characters replace text and Vim enters insert mode.

Visual mode (v)

Used for highlighting areas for text.

Ex mode (Q)

Similar to command-line mode, but does not exit mode after command

VIM modes

Mode	Purpose	Enter With	Example / Behavior
Normal	Move, delete, copy, paste, etc.	Press Esc	dd deletes a line, yy copies a line, p pastes below
Insert	Type text like in any editor	i, a, o, O	i – insert <i>before</i> cursor a – insert <i>after</i> cursor o – new line <i>below</i> O – new line <i>above</i>
Visual	Select text (char/line/block)	v, V, Ctrl+v	v – select chars, then y to copy V – select whole lines Ctrl+v – block/column Ctrl+shift+v
Command	Save, quit, search, replace	: (in Normal mode)	:w to save, :q! to quit without saving, :%s/foo/bar/g to replace text globally

Normal mode: Navigation

So how do we move around without a mouse?



Normal mode: Navigation

So how do we move around without a mouse?

Command

h j k l

w, b, e

0, ^, \$

gg, G

%

What It Does

Move left/down/up/right

Move to next, previous, end of word

Start of line, first char, end

Go to top/bottom of file

Jump to matching (), {}

Additionally:

Use numbers before motions to repeat:

5j → move down 5 lines

3w → skip 3 words

Search and Replace in Vim

Command

/word

?word

n

N

:%s/foo/bar/g

:%s/foo/bar/gc

Description

Search forward for the word "word"

Search backward for the word "word"

Jump to the next match

Jump to the previous match

Replace all "foo" with "bar" in the file

Replace with confirmation (y/n/a/q)

Copy paste in VIM

Command	Action	Description
yy	Yank (copy) current line	Use p to paste after, P to paste before
y\$	Yank to end of line	Copy from cursor to end
yaw or yiw	Yank one word	Copy one word from the cursor
p	Paste after cursor	Common after yy or yw
P	Paste before cursor	Useful in visual mode
dd	Delete current line	Also stores line in register (can paste later)
dw	Delete one word	From cursor to start of next word
d\$	Delete to end of line	From cursor to end
x	Delete one character under cursor	Simple char deletion
U	Undo last change	Step-by-step undo
Ctrl+r, :earlier	Redo last undone change	Opposite of u

Live demo

- Create a file:
 - vim demo.txt

You are now in **Normal Mode** (default on launch), which is for navigating and controlling the editor.

- Press Esc to ensure Normal Mode.
- Type i to be in the **Insert Mode**, type Hello and then <Esc> to come back to the **Normal Mode**
- Now we will try:
 - dd " delete current line
 - u " undo
 - Ctrl+r " redo.
- Write "Hello!" In the file.
- Then try:
 - i "BEFORE" → Esc
→ Result: BEFORE Hello!
 - Move to ! (\$), press a "AFTER" → Esc
→ Result: BEFORE Hello! AFTER
 - oThis is a new line below. → Esc
 - OThis is a new line above. → Esc

This is a new line above.
BEFORE Hello! AFTER
This is a new line below.

Live demo

Visual Mode (v) Use Visual Mode to select and operate on blocks.

v → use l to select → y to yank

V → select line → d to delete

Ctrl+v → block selection → > to indent

Practice:

- v → move to select Hello → press y to yank it
- Move cursor to BEFORE → press p to paste after

Result: BEFOREHello Hello! AFTER

- V → select full line → press y
- Move below → press p to paste copied line
- Use Ctrl+v → select 3 columns across 2 lines → press > to indent

- :w " Save
- :q! " Quit without saving
- :wq " Save and quit

**This is a new line above. BEFOREHello
Hello! AFTERThis is a new line below.
BEFOREHello Hello! AFTER**

Networking Essentials

Session 11

Networking Essentials

Session 11

Objectives:

- Understand key networking tools in Linux
- Test connectivity using common commands
- Review IP addressing concepts (IPv4/IPv6)
- Gain basic troubleshooting skills
- Understand DHCP and manual configuration tools

Networking

Why Networking?

- Enables communication: sharing files, accessing the internet, using cloud resources
- Essential for remote work and system administration
- Underpins security: firewalls, open ports, etc.

Almost everything today depends on network connections—whether you're sending emails, coding a web app, or logging into a server on the other side of the world. Even for data science, remote data storage and compute clusters require networking.

TCP/IP Overview

- TCP/IP is the foundational protocol suite for modern networking.
- It ensures end-to-end communication, routing, and application-level services.
- It is structured into four abstraction layers:

TCP/IP Overview

- TCP/IP is the foundational protocol suite for modern networking.
- It ensures end-to-end communication, routing, and application-level services.
- It is structured into four abstraction layers:
 - Application Layer:
 - Provides services to applications like web browsers, email clients, terminals.
 - Examples of protocols: HTTP, HTTPS, FTP, SSH, DNS, SMTP.
 - **is what you interact with: web browsers, SSH clients, email, etc**

TCP/IP Model
Application Layer
Transport Layer
Internet Layer
Link Layer

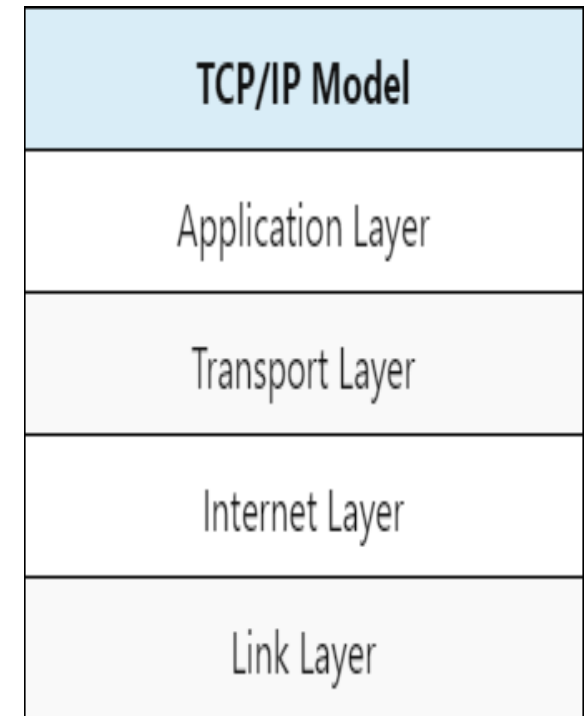
TCP/IP Overview

- TCP/IP is the foundational protocol suite for modern networking.
- It ensures end-to-end communication, routing, and application-level services.
- It is structured into four abstraction layers:
 - Transport Layer:
 - Ensures communication between host systems.
 - TCP: reliable, connection-oriented; ensures ordered delivery.
 - UDP: lightweight, connectionless; no delivery guarantees.
 - **delivers your data: either reliably (TCP) or fast but with less control (UDP).**

TCP/IP Model
Application Layer
Transport Layer
Internet Layer
Link Layer

TCP/IP Overview

- TCP/IP is the foundational protocol suite for modern networking.
- It ensures end-to-end communication, routing, and application-level services.
- It is structured into four abstraction layers:
 - Internet Layer:
 - Responsible for logical addressing and routing across multiple networks.
 - The IP protocol routes packets from source to destination.
 - Also includes ICMP (used for diagnostics like ping, traceroute).
 - Packets are identified and routed by source/destination IP addresses.
 - **finds the route to your destination with IP addresses.**

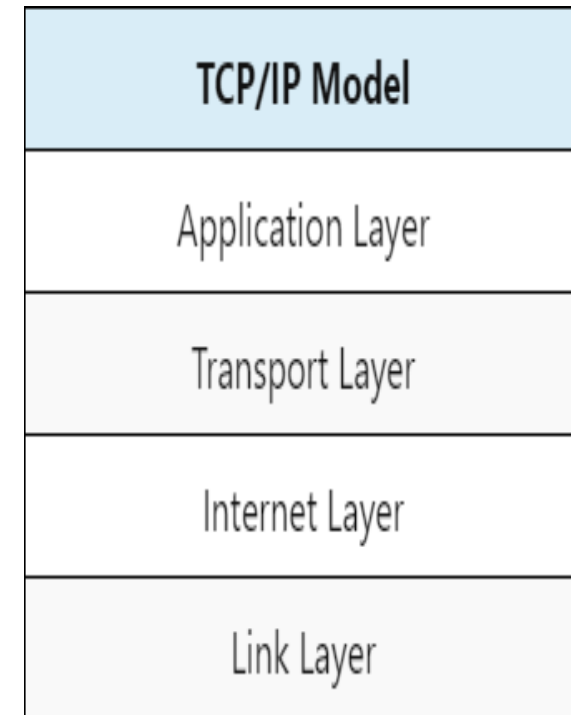


TCP/IP Overview

- TCP/IP is the foundational protocol suite for modern networking.
- It ensures end-to-end communication, routing, and application-level services.
- It is structured into four abstraction layers:
 - Link Layer:
 - Handles communication with the physical network.
 - Defines how data is encoded for a specific type of link (e.g., Ethernet, Wi-Fi).
 - Responsible for MAC addresses, frame transmission, and error checking.
 - This layered model allows interoperability, modularity, and reliable communication over diverse networks.
 - **moves the data through cables or WiFi, using things like MAC addresses.**

Think of it as the postal system:

- **The letter (data) goes in an envelope (TCP/UDP),**
- **gets an address (IP),**
- **and travels via different roads and delivery people (Ethernet/WiFi).**



Real-World Protocol Examples

Layer	Example Protocols	Example Use
Application	HTTP, SSH, FTP, DNS, SMTP	Web, remote login
Transport	TCP (reliable), UDP (fast)	File transfer, VoIP
Internet	IP, ICMP	Routing, ping
Link	Ethernet, WiFi	LAN, wireless

Browsing the web uses HTTP at the Application layer, which uses TCP to ensure your page loads correctly. DNS lets you use domain names instead of remembering IP addresses. SSH allows secure remote login.

What Is a Network?

- A network is a collection of connected devices that share resources and data.
- Networks allow communication between computers and services, locally or globally.
- Two major types:
 - LAN (Local Area Network) – e.g., your home Wi-Fi
 - WAN (Wide Area Network) – e.g., the Internet
- Requires protocols to establish rules (e.g., TCP/IP)

A network is just a collection of devices connected together. At home, that's your phone, laptop, and maybe a smart TV—using WiFi (a LAN). The Internet connects billions of devices together into a WAN. But for these devices to “talk,” they have to agree on the language and rules. That's the job of protocols like TCP/IP.

What is a IP adress?

- An IP address is a unique identifier for a device on a network.
- IPv4: 32 bits, written in dotted decimal (e.g., 192.168.1.1)
- IPv6: 128 bits, written in hexadecimal (e.g., 2001:0db8:85a3::8a2e:0370:7334)

IPv4 is what we still mostly use today. It's easier to read, but its address space is running out. IPv6 is the modern replacement, already widely used in smartphones, servers

- IP addresses can be statically assigned (manually configured by an admin), or dynamically assigned using a protocol like DHCP.
 - Static IP: **Useful for servers, printers, or devices requiring predictable addresses.**
 - Dynamic IP (via DHCP): Automatically assigned on demand; suitable for most client machines. For example, your laptop gets a different IP when you reconnect to WiFi.
- The IP address includes:
 - Network ID (defines which network)
 - Host ID (defines which device on the network)

Having a look at your IP

- In a terminal type ip a

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
inet 127.0.0.1/8 scope host lo
```

```
    valid_lft forever preferred_lft forever
```

```
inet6 ::1/128 scope host noprefixroute
```

```
    valid_lft forever preferred_lft forever
```

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:bf:7b:b6 brd ff:ff:ff:ff:ff:ff
```

```
inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
```

```
    valid_lft 85436sec preferred_lft 85436sec
```

```
inet6 fd17:625c:f037:2:678f:aae7:5d8c:bcdd/64 scope global temporary dynamic
```

```
    valid_lft 86285sec preferred_lft 14285sec
```

```
inet6 fd17:625c:f037:2:a00:27ff:febf:7bb6/64 scope global dynamic mngtmpaddr
```

```
    valid_lft 86285sec preferred_lft 14285sec
```

```
inet6 fe80::a00:27ff:febf:7bb6/64 scope link
```

```
    valid_lft forever preferred_lft forever
```


Having a look at your IP

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
    valid_lft forever preferred_lft forever
```

- The lo interface (loopback) has IPv4 127.0.0.1, **Used for internal communication within the same machine**
- /8: means that the first 8 bits are the network part, covering all addresses from 127.0.0.0 to 127.255.255.255.
- Inet6::1/128 is the loopback IPv6 address, similar to 127.0.0.1 in IPv4
- **IPv4 address: Always 127.0.0.1 (or in the range 127.0.0.0/8).**
- Typical use: When software on your machine wants to talk to itself (e.g., a web server tested locally with localhost).

Having a look at your IP

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:bf:7b:b6 brd ff:ff:ff:ff:ff:ff
inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
    valid_lft 85436sec preferred_lft 85436sec
inet6 fd17:625c:f037:2:678f:aae7:5d8c:bcdd/64 scope global temporary dynamic
    valid_lft 86285sec preferred_lft 14285sec
inet6 fd17:625c:f037:2:a00:27ff:febf:7bb6/64 scope global dynamic mngtmpaddr
    valid_lft 86285sec preferred_lft 14285sec
inet6 fe80::a00:27ff:febf:7bb6/64 scope link
    valid_lft forever preferred_lft forever
```

- 2: enp0s3: — This is the second listed interface, named enp0s3. **Connects your computer to a physical network (LAN, Internet, etc.)**
- link/ether 08:00:27:bf:7b:b6 — This is the **MAC address** of the interface (a unique identifier assigned to a network interface card)
- inet 10.0.2.15/24 — IPv4 address assigned via DHCP.
- scope global — Global IPv4 address assigned dynamically (DHCP)
- valid_lft, preferred_lft — DHCP lease durations: time left before renewal required.
- inet6 fd17:... scope global temporary dynamic — A temporary IPv6 address for privacy, assigned dynamically.
- inet6 fd17:... scope global dynamic mngtmpaddr — A stable, managed global IPv6 address.
- inet6 fe80::... scope link — Link-local IPv6 address, only reachable on the local network

Public vs Private IP Addresses

- **Private IP:**
 - used inside LANs
 - not accessible directly from the internet
 - IPv4: 192.168.x.x, 10.x.x.x, 172.16.x.x – 172.31.x.x
- **Public IP:**
 - Routable
 - assigned by ISP
 - visible online NAT (Network Address Translation)

Type:

- ip a → show Private IP
- curl ifconfig.me → show your public IP
visible by website

curl stands for **Client URL:**

- used to transfer data from or to a server using supported protocols
- curl ifconfig.me sends an HTTP request to a remote web service that responds with your public IP address.

DNS

What is DNS?

DNS

What is DNS?

- DNS stands for Domain Name System.
- It is the Internet's "phonebook" : it translates human-readable names like google.com into machine-readable IP addresses like 142.250.184.110.
- Without DNS, you would have to remember and type IP addresses instead of domain names.
- Your system queries a DNS to resolve names.(e.g. google: 8.8.8.8)

Try
ping google.com
ping 8.8.8.8

How IPs Are Assigned – DHCP

- **DHCP = Dynamic Host Configuration Protocol**
- Automatically assigns IP address, subnet mask, gateway, and DNS settings to clients.
- **NO need for manual IP configuration.**
- Works in four steps:
 - DHCPDISCOVER – client broadcasts to find DHCP server
 - DHCPOFFER – server responds with available IP and settings
 - DHCPREQUEST – client accepts the offer
 - DHCPACK – server confirms the lease
- Common scenarios: connecting a laptop to Wi-Fi, booting a VM on NAT
- Indicators of DHCP usage:
 - “ip a” shows scope global dynamic
 - valid_lft, preferred_lft timers on IP lease

DHCP: Automatic IP Assignment

On modern Linux systems, there are two main ways to manage network interfaces:

1. **Netplan (For static config on servers):**

- Declarative YAML files (e.g., /etc/netplan/ 01-network-manager-all.yaml) Example:
network: version: 2
ethernets: enp0s3:
dhcp4: true
- **Apply changes with:** sudo netplan apply

2. **NetworkManager GUI tool or CLI via nmcli, nmtui**

- Common on Ubuntu Desktop, Fedora, etc.
 - Configuration is managed internally or via /etc/NetworkManager
 - Interfaces will not appear in netplan YAML if managed by NetworkManager only.
-
- Search and open the YAML files:
network: version: 2
renderer: NetworkManager

Using nmcli

nmcli is the command-line tool for NetworkManager

Allows you to view or configure network settings without a GUI

- **Commands**
 - nmcli device show # Show interface details (IP, DNS, etc.)
 - nmcli con show # List all saved connections
 - nmcli device status # Connection state of all devices
 - nmcli con up id <name> # Activate a connection
 - nmcli con edit # Interactive conf

Ex: nmcli device show enp0s3

Expected output:

- Interface name and MAC address
- IP addresses (IPv4/IPv6)
- DNS servers
- DHCP status

Ports

- A port is a logical **communication endpoint on a device**.
- Think of an IP address as a street address, and a port as the apartment number.
- **Ports let one machine run multiple services but only one process can be bound to a port at a time.**

Port numbers range from 0 to 65535:

- 0–1023: Well-known ports (require root privileges to use)
- 1024–49151: Registered/user ports
- 49152–65535: Dynamic/private ports

Common ports:

22 → SSH

80 → HTTP

443 → HTTPS

53 → DNS

25 → SMTP (email)

Tools to list open ports:

- `ss -tuln`
- `lsof -i` → see which processes use which ports

Basic Troubleshooting

Steps if networking fails

Is the interface up?

- Run: `ip a`
- Look for state UP on `enp0s3` or similar
- If DOWN: `sudo ip link set enp0s3 up`

Do you have an IP address?

- Check: `ip a`
- If no inet line → request IP with: `sudo dhclient -v`

Can you reach your gateway?

- View default route: `ip r`
- Test: `ping 192.168.1.1` (or whatever your gateway is)

Can you resolve domain names? (DNS)

- Try: `ping google.com`
- If that fails but `ping 8.8.8.8` works → DNS problem
- Check: `cat /etc/resolv.conf`

Use traceroute to find the failure point

- `traceroute google.com` or `tracepath google.com`

Are services listening?

- `ss -tuln` → shows open ports
- `lsof -i` → shows which apps use the ports

Example Why can't I SSH into a server?

Is the interface up?

- Run: `ip a`
- What to check:
 - Is your main interface (enp0s3, eth0, wlan0, etc.) UP and does it have an IP address?
 - If not:
 - You may not be connected to the network. Check cables, or enable WiFi.

Can You Reach the Server?

- `ping <server_ip>`
- What to check:
 - Do you get replies, or does it time out?
 - If ping fails:
 - The server may be down, the IP may be wrong, or there's a network problem (firewall, routing)

Is DNS Working? If you use a hostname instead of an IP address, check if DNS is working.

- `ping <server_hostname>`
- What to check:
 - If `ping <IP>` works, but `ping <hostname>` fails, you have a DNS problem.

Example Why can't I SSH into a server?:

Is SSH Service Running on the Server?

- `sudo systemctl status ssh`
- `ss -tuln | grep :22`
- What to check:
 - Is sshd running? Is port 22 open?
 - If not:
 - Start the SSH service: `sudo systemctl start ssh`

Is a Firewall Blocking SSH??

- `sudo ufw status`
- `sudo firewall-cmd --list-all`
- What to check:
 - Is port 22 allowed?

Are You Using the Right Port?

- `ssh -p <port> user@server`

Check Your Credentials

- Are you using the correct username and password or SSH key?

Example Why can't I SSH into a server?:

Step	What to Check	Command(s)
Network Connection	Interface up & has IP	<code>ip a</code>
Server Reachable	Can ping server IP	<code>ping <server_ip></code>
DNS Works	Hostname resolves	<code>ping <hostname></code>
SSH Running	Service up, port 22 open	<code>systemctl status ssh` ss -tuln</code>
Firewall	Port 22 allowed	<code>ufw status</code> or <code>firewall-cmd</code>
Right Port	Correct SSH port	<code>ssh -p <port> ...</code>
Credentials	Correct user & key permissions	Check username, <code>chmod 600</code>
SSH Verbose	See debug output	<code>ssh -v ...</code>

Remote Access, Downloads & Shell Customization

Session 12

Remote Access, Downloads & Shell Customization

Session 12

Objectives:

- **Use SSH to connect to remote systems**
- **Download and extract content using wget, curl, tar, and unzip**
- **Customize their shell with alias, environment variables, and .bashrc modifications**

SSH: Secure Shell

What is SSH?

A cryptographic network protocol that allows secure communication between two machines, even over untrusted networks.

It is commonly used for:

- Secure remote login to servers
- Safe file transfers
- Running commands on remote systems

SSH: Secure Shell

ssh [username]@[hostname or IP address]

- [username]: your remote server username
- [hostname or IP address]: the server's hostname or IP address.

Ex: ssh thomas@192.168.56.1

SSH: Secure Shell

On Windows

- check ip from ipconfig on windows or check ip from the wifi (properties)
- Optional features ->
 `Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH.Server*'`
 `Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0`
- Try `Get-Service sshd`, should see running otherwise: `Start-Service sshd`
- *`net user thomas thomaspwd /add`*
- *`net user thomas /active:yes`*

On your VM (Linux)

- *`apt install openssh-server,`*
- *`systemctl enable ssh`*
- *`systemctl start ssh`*

SSH: Secure Shell

On MAC:

- Check `/etc/ssh/sshd_config` and you should see `PasswordAuthentication yes`
If not change the line and then run
`sudo launchctl stop com.openssh.sshd`
`sudo launchctl start com.openssh.sshd`
- In general, network allow sharing

SSH: Secure Shell

Password input?

SSH does not allow `--password` on the command line (for security)

Use **sshpass** only for demo/testing:

```
sshpass -p 'pwd' ssh thomas@192.168.56.1
```

Secure Copy: scp

Secure file transfer is an essential skill for Linux system administration. The **scp** command (Secure Copy Protocol) allows you to **safely copy files** over a network by using **SSH encryption**.

Unlike basic tools that send data in plain text, scp encrypts everything—your password, files, and commands: protecting them from eavesdropping or interception.

```
scp file.txt user@host:/path/  
scp -r folder/ user@host:/home/user/
```

Demo: Send a file from Linux VM to Windows scp file.txt thomas@192.168.56.1:~/Desktop/

rsync

rsync is a powerful command-line tool used to **synchronize files and directories** between two locations — locally or over a network via SSH..

It only transfers differences between source and destination — making it much faster than scp for repeated transfers or large directories.

Feature	scp	Rsync
Secure transfer	SSH	SSH
Sync only changed files	No	Yes
Resume support	No	Yes
Directory structure	Yes	Yes
Compression	No	YES (-z)

rsync

Syntax:

rsync [options] source destination

Option

Meaning

-a	Archive mode: preserve permissions, timestamps, symbolic links, etc.
-v	Verbose: show progress
-z	Compress data during transfer (faster over network)
-r	Recursive: copy subdirectories
--delete	Delete files from destination that are no longer in the source

-Sync folder to remote machine:

rsync -avz my_project/ thomas@192.168.1.39:~/projects/

-Pull files from a remote machine:

rsync -avz thomas@192.168.1.39:~/projects/ .

rsync

demo:

- create two folders test1 and test2*
- create 3 files into test1*
- rsync all the files from test1 to test2*
- modify one file and redo rsync*

mkdir test1 test2

cd test1;touch file1.txt file2.txt file3.txt

rsync -av test1/ test2/

gedit file2.txt

rsync -av test1/ test2/

Downloading Files: wget and curl

wget – For simple file downloads

wget gnu.org/software/wget/manual/wget.txt

wget -O lecture.pdf <https://example.com/file123.pdf> #DL and rename

curl – For more control (headers, APIs). Working with REST APIs or inspecting HTTP headers

ex: curl -O <https://example.com/file.zip> # save file

curl <https://example.com/data.csv> -o data.csv

Extracting Archives

Extract .tar and .zip archives

`tar -czvf archive.tar.gz folder/` # Create

`tar -xvf archive.tar.gz` # Extract

`unzip archive.zip` # Extract .zip file

Option

-c

-x

-v

-f

-z

-j

-t

Meaning

Create a new archive

Extract files from archive

Verbose: show file names during processing

File: specify archive file name

Use **gzip compression** (for .tar.gz)

Use **bzip2 compression** (for .tar.bz2)

List files inside an archive (without extracting)

Extracting Archives

Demo:

```
mkdir testdir && echo "Hello" > testdir/hello.txt  
tar -czvf testdir.tar.gz testdir/  
tar -tvf testdir.tar.gz  
tar -xvf testdir.tar.gz
```

Alias

Alias command useful to create custom shortcuts for frequently used commands.

- Temporary:

```
alias ll='ls -lrt'
```

- Permanent:

Open ~/.bashrc and add this line: `alias ll='ls -lrt'`

Then close and reopen terminal OR source ~/.bashrc

Others examples:

```
alias projects='cd ~/project'
```

PATH variable

Why can you run “ls” anywhere?

PATH variable

Why can you run “ls” anywhere?

Your own script (e.g. myscript.sh) only runs if you're in the same directory your \$PATH.

But system commands like ls, cd, work from anywhere.

PATH variable

Why can you run “ls” anywhere?

Your own script (e.g. myscript.sh) only runs if you're in the same directory your \$PATH.

But system commands like ls, cd, work from anywhere.

Because the shell automatically searches in all directories listed in \$PATH when you run a command without a path.

PATH variable

echo \$PATH

-> usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games

When you run a command:

- ➔ If it finds the command in one of those directories → it runs it
- ➔ If not → it shows “command not found”

PATH variable

demo:

Create a simple shell script
gedit hello.sh

```
#!/bin/bash  
echo "Hello!!."
```

Make the script executable
chmod +x hello.sh

Add the line to your .bashrc:

```
export PATH=$PATH:$HOME/XXXXXX  
source ~/.bashrc
```

Run it from ANYWHERE